Governance And Constitution Meta

§ . . — File-layout discipline

Project files MUST be organised by purpose, not by historical accident. Source code goes under canonical project roots. Tests go under canonical test directories. Logs and forensic artifacts go under operator-controlled directories (`~/Documents/` , `qa-results/` , `/tmp/` , OS-equivalent) — NEVER scattered at the repo root, NEVER inside working source trees, NEVER tracked unless they are reference assets explicitly required for evidence-replay.

Discovered drift is fixed by **moving** files to their canonical location and updating every caller — never by adding redirect shims or by leaving the misplaced copy "for backwards compatibility".

Carve-out (User mandate 2026-05-20). The 5 canonical tracker documents — `docs/Issues.md` , `docs/Issues_Summary.md` , `docs/Fixed.md` , `docs/Fixed_Summary.md` , `docs/CONTINUATION.md` — sit at `docs/` root by design. They are architectural constants of the project layout, analogous to AOSP's `Makefile` , `Android.bp` , `OWNERS` files at repo root. Their location is encoded as literal path strings in §11.4.12 + §11.4.15 + §11.4.16 + §11.4.19 + §11.4.44 + §11.4.53 propagation gates plus the helper-script constellation that regenerates them (`generate_issues_summary.sh` , `generate_fixed_summary.sh` , `sync_issues_docs.sh` , `commit_docs.sh` , `colorize_progress_html.py` , `export_progress_docs.sh`). Moving them would require coordinated amendment of those 6 sister anchors plus 5 pre-build gates plus ~20 helper scripts plus 42 consumer files (parent + 10 owned submodules + nested + HelixQA dependencies) in a single PWU. Per §11.4.66, that scope is operator-blocked until explicitly authorised. Audit-snapshot files (`docs/audit/anti_bluff_audit.md` , `docs/audit/PRE_SONOS_TAG_READINESS.md` , `docs/audit/`

`D1_WIFI_FAIL_CLASSIFICATION.md`, plus any future audit snapshots) DO move under `docs/audit/` per the §11.4.11 general principle — they are not part of the tracker constellation and carry no propagation-gate path-string burden.

§ . . — Universal-vs-project classification of new rules (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-14):

> "Adding any new rules or mandatory constraints or anything relevant which should be added into Constitution, CLAUDE.MD, AGENTS.MD and relevant files MUST BE determined as reusable / universal VS project specific. If it is universal and reusable it MUST BE added into our root / main constitution Submodule, otherwise into our project level Constitution, AGENTS.MD and CLAUDE.MD (or Submodule if it is Submodule related)."

Before any new rule, mandatory constraint, covenant clause, gate declaration, or "MUST"-bearing statement is added to a project's Constitution / CLAUDE.md / AGENTS.md or to a child submodule's equivalents, the author MUST first classify it along this axis:

Classification	Definition	Destination
Universal (reusable)	The rule applies to ANY project — independent of language, framework, target platform, or domain. Examples: anti-bluff covenant, no-guessing mandate, credentials-handling, host-session safety, item-status tracking, multi-upstream push, file-layout discipline, deep-web-research-before-implementation.	This constitution submodule's `Constitution.md` + `CLAUDE.md` + `AGENTS.md`. Propagates to every consuming project via inheritance.
Project-specific	The rule references a specific hardware target, vendor, model, SoC, vendor-fork, project-internal package name, deployment topology, or company-internal asset. Examples (illustrative — these are intentionally NOT in this universal file): the SoC's specific power-management quirks; a particular Android/iOS/Linux SDK behaviour; a specific app or service bundled with the project.	The project's own `Constitution.md` / `CLAUDE.md` / `AGENTS.md` (top-level), OR the affected submodule's equivalents (when the rule scopes to that submodule).

A rule that mentions hardware part numbers, vendor names, specific package identifiers, geographic regions, or company-internal asset names is **project-specific by definition** — it cannot be lifted into the universal layer without genericising those references first. A universal rule MAY reference patterns (e.g., "USB-HID multitouch panels") but MUST NOT name specific vendors.

Anti-bluff: the universal-vs-project classification is itself an auditable artefact. Every new rule's commit message MUST include a one-line classification statement (e.g., `Classification: universal — applies to any project tracking items through an Issues/Fixed lifecycle`). A commit that adds a rule without classification justification is a §11.4.17 violation.

Authors who are uncertain SHOULD default to project-specific (narrower scope). Lifting a project-specific rule to universal later is cheap (one merge); generalising a universal rule that turned out to leak project-specific assumptions is expensive (every consumer must update).

Pre-build gate `CM-UNIVERSAL-VS-PROJECT-CLASSIFICATION` audits the last N commits for rule additions and asserts each one carries a classification statement. Paired mutation strips the classification literal and asserts the gate FAILs.

§ . . — Constitution-Submodule Update Workflow Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE commited and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!"

Operative rule. Before ANY agent or operator modifies the `constitution/Constitution.md`, `constitution/CLAUDE.md`, or `constitution/AGENTS.md` files of a project that consumes this Constitution as a submodule, the agent or operator MUST execute the following pipeline in order, with NO step skipped:

1. **Fetch + pull first.** From inside the `constitution/` submodule worktree, run `git fetch <every-configured-remote>` followed by `git pull --ff-only origin <branch>` (or `--rebase` if non-FF-mergeable; **never** `--strategy=ours` / `--allow-unrelated-histories` without explicit operator authorization). The submodule MUST be at upstream tip BEFORE any local edit is applied.

2. **Apply the change.** Edit the relevant file(s). The edit MUST classify itself per §11.4.17 (universal vs project-specific) — only universal additions belong in the constitution submodule; project-specific clauses belong in the consuming project's own governance files. The edit MUST include the verbatim user mandate (if it originated from one) as a forensic anchor.
3. **Validate before commit.** Run the constitution submodule's `meta_test_inheritance.sh` (or equivalent) to confirm the inheritance chain still resolves. Verify no governance file was left with merge-conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`). Verify all three governance files (Constitution + CLAUDE + AGENTS) cross-reference the new clause consistently.
4. **Commit + push to ALL upstreams.** Stage only the governance files (NEVER `git add -A` inside the constitution submodule — stray local artefacts MUST NOT enter governance). Commit with a message that cites the user mandate (verbatim quote) + the classification line per §11.4.17. Push to **every** configured remote of the constitution submodule. A commit that lives on one upstream but not others is a §11.4.26 violation equivalent to a §2.1 multi-upstream-push violation.
5. **Conflict resolution.** If `pull --ff-only` reports non-fast-forward, the merge MUST be performed carefully: inspect both sides, preserve the union of governance content (no clause silently dropped), re-classify per §11.4.17, validate per step 3. Force-push to "make conflicts go away" is FORBIDDEN (§9.2). Nothing about the constitution may be broken, made faulty, corrupted, or rendered unusable by the merge.
6. **Post-merge validation + verification.** After the push lands, re-clone (or `git submodule update --remote --init`) in a throwaway worktree and re-run the consumer project's inheritance-cascade verifier (e.g. `scripts/verify-governance- cascade.sh`) to confirm the new clause reaches every owned submodule per CONST-047. Any cascade gap MUST be closed in the same change-window.
7. **Update the consuming project's pointer.** The consuming project's `.gitmodules`-tracked submodule pointer MUST be bumped to the new constitution HEAD in the SAME commit as any downstream cascade work; out-of-sync submodule pointers are a §11.4.26 violation.

Operational scope. This workflow applies regardless of who initiates the change (operator, primary agent, subagent per §11.4.20, automated lint suggestion). The workflow CANNOT be shortcut by "I'll fetch later" or "I'll push to the other upstreams in the next commit". The constitution is the **single source of truth** for every project that imports it; allowing it to fragment across upstreams is the structural equivalent of a §11.4 PASS-bluff at the governance layer.

Cross-cutting reach. §11.4.26 composes with: §2 (single- entrypoint commit wrapper), §2.1 (multi-upstream push), §3 (submodule changes propagate through submodule commits first), §9.1 / §9.2 (data-safety + force-push authorization), §11.4.17 (universal-vs-project classification), §11.4.22 (lightweight doc-sync), §11.4.25 (full-automation coverage — the post-merge validation step is itself a form of automation coverage). It does NOT supersede them.

Classification: universal (per §11.4.17). No escape hatch. A constitution-submodule change that violates §11.4.26 is a release blocker for every consuming project, equivalent in severity to a force-push without §9.2 authorization.

2026 05 15

§ . . — Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory — we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! We MUST take it into the account, analyze it, extend it, create missing tests, do full testing of it, fill the gaps (if any), fix any issues that we discover or they pop-up, write and extend the documentation, user guides, manulas, diagrams, graphs, SQL definitions, Website(s) and all other relevant materials! We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE acessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project — directly from project's root project_name/ submodule_name or some more proper structure project_name/ submodules/submodule_name! This MUST BE heavily enforced and respected so no chaos and mess is created with various dependencies we may have!"

Operative rule. Three cooperating invariants govern every consuming project's relationship with its owned-by-us submodules (those whose upstream `origin` lives under one of the operator- listed orgs — `vasic-digital`, `HelixDevelopment`, `red-elf`, `ATMOSphere1234321`, `Bear-Suite`, `BoatOS123456`, `Helix-Flow`, `Helix-Track`, `Server-Factory` — or any additional org the operator subsequently authorises, the canonical list discoverable at any time via `gh org list` / `glab` / the orgs' public APIs):

(A) Equal-codebase invariant. Every owned-by-us submodule is an **equal part** of the consuming project's codebase. The consuming project's engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manual, guides, diagrams, graphs, SQL definitions, website pages, any other authoring surface) — applies to each owned submodule on equal basis. A round of work that improves only the main project while leaving an owned-submodule deficiency

unaddressed is a §11.4.28 violation, severity-equivalent to a §11.4 PASS-bluff at the project-scope layer. Coverage ledgers (§11.4.25) MUST list every owned submodule as an in-scope target. CONST-047 (recursive cascade) governs the propagation of governance changes; §11.4.28 is the engineering-content counterpart that mandates the same attention to non-governance content.

(B) Decoupling / reusability invariant. Owned submodules MUST remain **fully decoupled** from any specific consuming project. No project-specific context, hardcoded paths, hostnames, asset names, naming schemes, or runtime assumptions may be introduced into an owned submodule's source tree. Every owned submodule MUST be:

- **Project-not-aware** — its code, tests, and docs make no reference to which parent project consumes it.
- **Fully reusable** — any future Helix-family or third-party project must be able to consume the submodule unmodified.
- **Modular** — its public surface is the only documented integration contract; internal layout may evolve without breaking consumers.
- **Completely testable** — every public surface has standalone tests (per the §11.4.27 100%-test-type matrix) that pass when the submodule is checked out as a standalone repo, without any parent-project rigging.

A commit that adds `project_name/...` strings, hostnames belonging to a specific deployment, or other parent-project context to a submodule's source tree is a §11.4.28 violation. The honest path when a submodule needs information from the parent project is configuration injection (env var, config file, constructor parameter) — never a hardcoded reach into the parent's tree.

(C) Dependency-layout invariant. Every dependency that an owned submodule itself consumes MUST be accessible **from the root of the parent project** at one of two canonical paths:

```
<project_root>/<submodule_name>/          # flat layout
<project_root>/submodules/<submodule_name>/   # grouped layout
```

Nested-submodule chains are FORBIDDEN. A submodule MUST NOT have its own `.gitmodules` entries that pull in further owned- by-us repos (transitive own-org submodule recursion). Every dependency required by submodule X MUST be

added to the parent project at the canonical path above; X reaches it via documented import / SDK path / runtime resolver — never via its own nested submodule pointer.

EXCEPTION — constitution-submodule-anchored reusable engines (operator Option-2 decision, 2026-07-07; universal per §11.4.17). The single scoped carve-out to the nested-chain prohibition: THE CONSTITUTION SUBMODULE ITSELF (the canonical root per §11.4.35, where reusable engines are anchored) MAY host its own reusable- engine submodules under `constitution/submodules/<name>/`, PROVIDED each such nested submodule (i) ships a §11.4.31 `helix-deps.yaml` at its root so consumers can reconstruct the dependency graph, AND (ii) declares ZERO further own-org submodules of its own — DEPTH-1 ONLY, never a recursive chain (a nested engine that itself nests an own-org submodule is still FORBIDDEN). This EXCEPTION applies to NO OTHER repository: arbitrary consumer submodules remain forbidden from nesting and MUST use the root-flattened `helix-deps.yaml` bridge above. The rationale for the carve-out is that the constitution submodule is the deliberate home of reusable, project-agnostic engines (§11.4.28(B) / §11.4.74) — anchoring them as depth-1 nested submodules there, each with a manifest, keeps the version-drift risk the base rule guards against bounded (one authoritative home, one manifest, no transitive recursion) while satisfying the operator's intent that the engines be *derived from / anchored by* the constitution.

Rationale: nested own-org submodule chains cause version-drift chaos (two consumers of `LLMsVerifier` end up at different SHAs because each parent picked a different transitive path). The flat / grouped layout makes the consuming project's submodule graph a tree-of-depth-1, which any developer can audit at a glance via `git submodule status` from the project root.

Third-party submodules (not under our orgs) are exempt — they MAY appear at any depth as the upstream's structure dictates. The invariant applies only to our owned set.

Audit + enforcement.

- Gate `CM-OWNED-SUBMODULE-EQUAL-ENGINEERING` (project-side): every release-gate sweep verifies each owned submodule has current test runs, coverage entries (§11.4.25), and documentation freshness on par with the main project. Stale submodules surface as §11.4.28 violations (Status: Operator-blocked or In progress per §11.4.21 — never "ignored").
- Gate `CM-OWNED-SUBMODULE-DECOUPLING` (submodule-side): every owned submodule's pre-commit hook (or equivalent) greps the staged diff for parent-project names / hostnames / asset names. Hits abort the commit until refactored to configuration injection.
- Gate `CM-OWNED-SUBMODULE-LAYOUT` (project-side): the parent project's pre-merge sweep verifies (i) every owned submodule sits at `<root>/<name>/` or `<root>/submodules/<name>/`, (ii) no owned submodule contains a nested `.gitmodules` entry whose upstream is in our org list — EXCEPT the constitution submodule, which MAY carry depth-1 nested own-org engine submodules under `constitution/submodules/<name>/` PROVIDED each ships a `helix-deps.yaml` AND declares zero further own-org submodules (the §11.4.28(C) carve-out; a depth-≥2 chain, or a nested engine missing its manifest, still FAILs), and (iii) every dependency declared by an owned submodule has a corresponding parent-project submodule entry at the canonical path. Paired §1.1 mutation: add a SECOND-level own-org `.gitmodules` entry inside a constitution-anchored engine (a depth-2 chain) → gate FAILs; OR drop the engine's `helix-deps.yaml` → gate FAILs; restore → PASSes.
- Paired mutations (§1.1) for all three gates: plant the forbidden pattern → gate FAILs; restore → gate PASSes.

Workflow integration. Honoring §11.4.28 in practice:

- Engineering rounds plan in submodule-aware tranches: "improve feature X in main; in same round, audit + improve the X- related surface in submodule Y; cascade governance per CONST-047 as usual".
- The §11.4.25 coverage ledger row format is extended with a `submodule` column so coverage rolls up across the whole owned set.

- Cross-submodule refactors that touch shared types or interfaces ship as a single change-window: parent + every affected owned submodule advanced in lockstep (§3 submodule- first-commit discipline + §2.1 multi-upstream push).

Classification: universal (per §11.4.17). No escape hatch. Composes with: §1 (four-layer test floor reaches submodules too), §1.1 (false-positive immunity), §3 (submodule changes propagate through submodule commits first), §11.4.17 (universal-vs-project classification), §11.4.20 (subagent delegation for the cross-submodule audit sweep), §11.4.25 (full-automation coverage ledger expanded to submodules), §11.4.26 (constitution-update workflow's submodule-pointer-bump step), §11.4.27 (100%-test- type coverage applies to every submodule's standalone surface), CONST-047 (recursive governance cascade). A round of work that violates §11.4.28 is a release blocker for the consuming project, severity-equivalent to a §11.4 PASS-bluff at the codebase-completeness layer.

§ . . — Lowercase-Snake_Case-Naming Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of '_' character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! However, since this will most likely break some of the functionalities renaming we do MUST BE applied to all references to particular Submodule or directory! Everywhere where particular Submodule directory are referenced proper updates MUST BE applied
>
> - all configuration files, documentation and relevant materials, links to Submodules and directories, source code that points to them, etc. There MUST BE reasonable exceptions for this rules - source code for programming languages or Submodules which apply different naming convention - Android, Java, Kotlin and others. Root directory for such applications, services or Submdoules can follow OUR convention, but EVERYTHING inside still MUST follow language / technology specific rules! We apply this rules per common sense basis and it MUST NOT be the cause of bigger issues such as technology breaking! Upstreams directory which all of our projects and Submodules have MUST BE renamed to the lowercase letters too, however root project containing the install_upstreams system command (it is exported in out paths in our .bashrc or .zshrc) MUST BE updated to fully work with both Upstreams and upstreams directory. That change if it is not already applied MUST BE done, commited and pushed! ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them! Every change done MUST BE covered with all supported test types, full automation tests for validation and verifiifcation and fully applied and followed anti-bluff policy and all anti-bluff rules!"

Operative rule. Every directory, submodule, and file under the parent project's working tree MUST use a **lowercase, snake_case** name (ASCII letters / digits / underscores, words separated by `_`). Existing names that violate the rule (`ExampleModule/`, `Challenges/`, `Containers/`, `ExampleAgent/`, `HelixQA/`, `Security/`, `Github-Pages-Website/`, `Upstreams/`, `Dependencies/`, etc.) MUST be renamed as part of the migration window opened by this clause. Every reference in the codebase MUST be updated atomically with

the rename: configuration files, documentation, user manuals, diagrams, scripts, source-code imports, links, governance files. **Reference drift after a rename is a §11.4.29 violation** of equal severity to the rename itself.

Exceptions (common-sense scope). The rule MUST NOT break language-/technology-specific conventions:

- **Programming-language source roots** that mandate a specific case (Java / Kotlin package paths, Android resource folders, Apple framework directories, C# / Swift project layouts) keep their language-mandated names. The submodule's root directory follows our convention; the language-specific subtree inside follows its own.
- **Vendor / upstream submodules** (third-party orgs not in our owned set) keep their upstream-mandated names — we MUST NOT rename a third party's repo.
- **Build-tooling artefacts** (`node_modules/` , `__pycache__/` , `.git/` , `target/` , `build/` , `bin/`) keep their tool-mandated names.

When in doubt, the test "does renaming break the technology?" trumps the snake_case rule. The §11.4.29 spirit is operator- ergonomics + reference-discoverability, not pedantic uniformity at the cost of technology compatibility.

`Upstreams/` → `upstreams/` transition. The constitution submodule's installer (`install_upstreams.sh`) — exported on operator paths via `.bashrc` / `.zshrc` — MUST support **both** `Upstreams/` and `upstreams/` directory layouts during the migration window, so existing checkouts keep working while consuming projects rename at their own pace. The installer reads whichever directory exists; if both exist the lowercase wins. After every project under this Constitution has migrated, the uppercase fallback MAY be retired by a deliberate amendment, but the migration window remains open as long as ANY owned project still ships the uppercase form.

Project-Toolkit Upstreamable submodule synchronisation. The Upstreamable / Project-Toolkit machinery that propagates governance into every consuming project MUST be fetched + pulled before any rename batch, and MUST itself comply with this rule. Any Upstreamable submodule lacking BOTH-directory support is a release blocker for the rename program.

Test coverage of renames. Every batch of renames MUST ship with: (i) a regression test that verifies every reference to the renamed entity now resolves to the new name (no stale references left); (ii) a full CONST-050(B) test-type matrix run against the post-rename tree; (iii) anti-bluff (CONST-035) wire-evidence captured during the runtime verification. A rename batch without all three is a §11.4.29 violation.

Cascade reach. This rule applies recursively through every owned-by-us submodule layer (per CONST-047) and applies equally to the consuming project and its owned submodules (per CONST-051). Per CONST-051(C) — dependencies at the parent root — the renamed paths MUST be the only canonical location; old-name aliases remain only as transitional symlinks if absolutely required, and those symlinks MUST be removed on next-N-cycle review (project- configurable, recommended ≥3 release cycles).

Phased execution. Because the rename touches every reference, the execution MUST be planned as fine-grained phases per the operator's explicit instruction: comprehensive brainstorming, phase-divided plan, fine-grained tasks/subtasks with enormous detail, every change covered by every applicable test type. The phases run in parallel with mainstream work (§11.4.20 subagent delegation is the natural fit for cross-submodule rename sweeps).

Classification: universal (per §11.4.17). No escape hatch beyond the explicit common-sense exceptions enumerated above. Severity-equivalent to a §11.4 PASS-bluff at the reference-integrity layer — a half-completed rename that leaves broken references is worse than no rename at all because it silently breaks consumers.

Composition. §11.4.29 composes with §1 (four-layer floor for every rename batch), §1.1 (paired mutation: rename without updating a reference → gate FAILs), §11.4.12 (regenerate auto- generated docs on rename), §11.4.17 (universal — no project- specific assumptions), §11.4.18 (script-doc-sync after renamed script paths), §11.4.20 (subagent delegation for cross-cutting rename sweeps), §11.4.25 (every renamed path appears in coverage ledger), §11.4.26 (constitution-submodule rename pipeline), §11.4.27 (rename-touch test types apply across the matrix), §11.4.28 (owned submodule renames cascade per CONST-047), CONST-047 (recursive governance reach).

§ . . — Submodule-Dependency-Manifest Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file with expected Submodules Git ssh urls perhaps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to veryone. This MUST be analyzed and applied. We MUST apply the best strategy for this which can be easily executed just by following our root Constitution, AGENTS.MD and CLAUDE.MD! Process this, extend out Constitution, AGENTS.MD and CLAUDE.MD with mandatory instructions and then process project root and all Submodules deep recursively so proper configuration Submodules dependency files are created! Document EVERYTHING and cover with all supported test types! Any kind of bluff is strictyl forbidden! All wotk MUST come with mechanism for validation and verification by creating proper proofs for all critical points!"

Operative rule. Every owned-by-us submodule MUST ship a machine-readable, version-controlled **dependency manifest** at the canonical path `helix-deps.yaml` (or `helix-deps.json` / `helix-deps.toml` if the submodule's ecosystem strongly prefers a different serialisation — but only one canonical file per submodule, and its name MUST appear in the submodule's `README.md` + governance trio).

Manifest schema (CONST-051(C)-aligned):

```yaml
# helix-deps.yaml
# Declares this submodule's own-org dependencies. Consuming projects
# read this manifest recursively and add each declared dependency to
# their root at <root>/<name>/ or <root>/submodules/<name>/ per
# CONST-051(C). Nested own-org submodule chains are FORBIDDEN — this
# manifest is the bridge. (Sole exception per §11.4.28(C): the
# constitution submodule MAY host depth-1 reusable-engine submodules
# under constitution/submodules/<name>/, each shipping its own
# helix-deps.yaml — never a recursive chain.)

schema_version: 1
deps:
  - name: Challenges                    # canonical name (will become snake_case per §11.4.29)
    ssh_url: git@github.com:vasic-digital/Challenges.git
    ref: main                           # branch or pinned tag
    why: "Cross-cutting Challenges + Panoptic browser harness"
    layout: flat                        # 'flat' = <root>/<name>/; 'grouped' = <root>/submodules/
                <name>/
  - name: HelixQA
    ssh_url: git@github.com:HelixDevelopment/HelixQA.git
    ref: main
    why: "Anti-bluff QA orchestration + autonomous-session driver"
    layout: flat

transitive_handling:
  # Each declared dep itself ships a helix-deps.yaml. The incorporator
  # tooling MUST recurse — top-level project gets the union of all
  # transitively-declared deps, flattened to root.
  recursive: true

  # When two submodules declare the same dependency at different
  # refs, CONFLICT — operator MUST resolve before incorporation
  # proceeds. The incorporator aborts on conflict (never silently
  # picks one).
  conflict_resolution: operator-required

language_specific_subtree: false      # set true for Android/Kotlin/Apple
                                      # roots (per §11.4.29 exception);
                                      # excludes inner subtree from
                                      # snake_case enforcement.
```

Tooling contract. A consuming project MUST be able to bootstrap its entire own-org dependency graph by:

1. Running `incorporate-submodule <ssh-url>` (canonical name; lives in the constitution submodule's `scripts/` or in a parent project's bin path).
2. The script:
    - Adds the supplied submodule at its declared canonical path (per CONST-051(C) `flat` vs `grouped` layout).
    - Reads the newly-added submodule's `helix-deps.yaml`.
    - For each declared dep, checks if it already exists at the consuming project's root; if missing, recurses (incorporate- submodule on the dep's ssh_url).
    - On conflict (same name declared at different ref by two submodules), aborts with a directed error pointing the operator at the conflicting declarations.
    - Emits a final manifest-of-manifests file at `<root>/.helix-manifest.yaml` listing every submodule + its declared deps, for audit + reproducibility.

Anti-bluff guarantee. Every manifest MUST be paired with a **verification proof**: a Challenge script (per §11.4.27 + CONST-050(B)) that:

- Bootstraps a throwaway consuming project from scratch in a temp directory.
- Runs `incorporate-submodule` against the manifest under test.
- Verifies the produced submodule layout matches the manifest's declarations (every dep present at its declared layout path; no extras; no missing).
- Runs the submodule's own test suite against the bootstrapped layout; asserts pass.
- Captures wire evidence (per §11.4.2) of every step.

A manifest without this verification proof is a §11.4.31 violation of equal severity to a §11.4 PASS-bluff at the dependency-graph layer.

Cascade requirement. Every owned-by-us submodule MUST ship `helix-deps.yaml` at its root, recursively (sub-submodules of submodules ship their own). The constitution submodule itself ships a manifest declaring its own deps (currently empty for the universal Constitution; project consumers may have project-specific extensions). The verifier (`scripts/verify-governance-cascade.sh` or its successor) MUST check every owned submodule for manifest presence.

Composition. §11.4.31 directly enables §11.4.28 / CONST-051(C) flat-layout enforcement: nested own-org submodule chains can be mechanically flattened because each submodule declares what it needs, and the incorporator places those deps at the root. The §11.4.28(C) constitution-submodule carve-out does not weaken this: a permitted depth-1 constitution-anchored engine still ships its own `helix-deps.yaml`, so the incorporator reconstructs the graph identically — the manifest is the bridge in both cases. Composes with §1 (manifest schema is itself tested for parse + validation), §3 (submodule-first commit discipline applies to manifest changes too), §11.4.12 (manifest changes regenerate any derived docs/ diagrams), §11.4.17 (universal — no project-specific assumptions in the manifest format), §11.4.18 (manifest documented in script-doc + external user guide), §11.4.20 (subagent delegation for cross-submodule manifest authoring), §11.4.25 (manifest presence in coverage ledger), §11.4.26 (constitution-update workflow when extending manifest schema), §11.4.27 (manifest test matrix), §11.4.28 (this rule is its operational complement), §11.4.29 (manifests use snake_case names + canonical paths), §11.4.30 (manifest is tracked source — not a build artefact), CONST-047 (manifests cascade recursively).

Classification: universal (per §11.4.17). No escape hatch. Severity-equivalent to a §11.4 PASS-bluff at the dependency-graph layer.

2026-05-15

§11.4.32 — Post-Constitution-Pull Validation Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!"

Operative rule. Whenever a consuming project's constitution submodule is fetched + pulled with **any** content change (rule addition, rule revision, gate addition, anchor reference, version bump), the consuming project MUST execute a full-project + recursive-submodule validation sweep BEFORE the new constitution HEAD is treated as canonical for any other work.

Validation sweep contract. The sweep is implemented as `scripts/verify-all-constitution-rules.sh` (canonical name) which:

1. Re-runs the existing governance-cascade verifier (`scripts/ verify-governance-cascade.sh`) covering every §11.9 + CONST-* anchor across every owned submodule (recursive per CONST-047).
2. For each rule whose enforcement gate is implementable programmatically (e.g., CONST-053 `.gitignore` -pattern audit; CONST-051(C) nested-own-org-chain audit; CONST-052 case-conformance audit; CONST-050(A) mock-path-from-production- code audit; CONST-035 anti-bluff smoke-scan), the sweep runs the corresponding gate against the post-pull tree.
3. Any failure produces a directed FAIL entry naming the rule (e.g., `FAIL: CONST-053 — *.log tracked at the-project/foo.log`)
    - the canonical fix.
4. Failures populate the project's Issues tracker per §11.4.15 with Status: `Reopened` (since a previously-passing audit now fails) and Type: `Bug` (since real codebase state violates the constitution).
5. The agent or operator MUST resolve every FAIL before treating the new constitution HEAD as canonical — closure of each reopened item per the existing §11.4 anti-bluff covenant (positive-evidence-only, captured wire evidence).

Pull-time invocation. `git submodule update --remote constitution` MUST trigger the sweep automatically (post-update hook OR the operator's commit wrapper invokes it as part of any commit that advances the constitution submodule pointer). Operator-explicit manual invocation MUST also be available (`./scripts/verify-all-constitution-rules.sh`).

Anti-bluff guarantee. A sweep that exits PASS without actually running every implementable gate is a §11.4.32 violation. The sweep's own meta-test (paired mutation, §1.1) MUST plant a known violation of each enforced gate and assert the sweep reports FAIL for the planted gate.

Composition. §11.4.32 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule. Without it, new rules cascade as anchors but never get enforced in the codebase. Composes with every rule that has a programmatic gate: §1, §1.1, §11.4.10, §11.4.12, §11.4.15, §11.4.16, §11.4.18, §11.4.20, §11.4.22, §11.4.24, §11.4.25, §11.4.26, §11.4.27, §11.4.28, §11.4.29, §11.4.30, §11.4.31, CONST-035, CONST-038, CONST-042, CONST-043, CONST-044, CONST-045, CONST-046, CONST-047, CONST-048, CONST-049, CONST-050, CONST-051, CONST-052, CONST-053, CONST-054.

Classification: universal (per §11.4.17). No escape hatch. Severity-equivalent to a §11.4 PASS-bluff at the constitutional- enforcement layer — without §11.4.32, every other rule is a decorative anchor rather than an enforced gate.

§ . . — Canonical-root inheritance clarity (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-15):

> "Parent AGENTS.MD or CLAUDE.MD are located under constitution directory (Submodule) containing these parent (root) which files we are inheriting inside the project! Pay attention to this and make sure you ALWAYS follow this rule! Same applies to the root Constitution file!"
>
> "If needed (and we think it is needed it seems) add into the root Constitution, AGENTS.MD and CLAUDE.MD located in constitution directory (Submodule) which we are inheriting that the root (parent) Constitution, AGENTS.MD and CLAUDE.MD are not the ones in root of the project but inside the constitution directory (Submodule). We are inheriting it and if project specific rules have to be added or project specific constraints which are not universal and reusable, then they go into the Constitution, CLAUDE.MD and AGENTS.MD of the project directory itself!"

Classification: §11.4.17-classified **universal** — every project that consumes this constitution as a Git submodule MUST unambiguously distinguish the **canonical root** (the constitution submodule's three files) from the **consumer extensions** (the project's own three files).

The defect this anchor closes. Loose terminology — "parent CLAUDE.md", "root CLAUDE.md", "main constitution" — used without a file-path anchor causes a quiet but persistent confusion between the inheriting copy and the source-of-truth. AI agents have already been observed editing the consumer-side `CLAUDE.md` when the User's intent was to extend the canonical layer (or vice-versa), causing universal rules to leak into project-specific copies and project-specific rules to be falsely promoted as universal. The defect compounds: once a rule is misfiled, future propagation gates treat the misfile as canonical and silently spread it.

The mandate. The three files in **this constitution submodule** (`constitution/Constitution.md` , `constitution/CLAUDE.md` , `constitution/AGENTS.md`) are the **canonical root** — also called the **parent** files. They contain only universal rules per §11.4.17 — rules that any project consuming this submodule benefits from.

The three files in **the consuming project's repository root** (`<project-root>/CLAUDE.md` , `<project-root>/AGENTS.md` , optionally `<project-root>/Constitution.md` or equivalent) are the **consumer extensions**. They MUST start with the inheritance pointer per the existing constitution/CLAUDE.md "How inheritance works" section. They contain only project-specific rules per §11.4.17 — rules that reference particular hardware, vendor names, regulatory regions, internal asset names, or project-private conventions.

Operative invariants that follow:

1. **When in doubt about which file to edit:** if the rule is reusable across any project, edit the constitution submodule's file. If the rule references project-private specifics (a hardware revision, a vendor SDK version, a regional constraint, a particular service), edit the consumer's file. Default to consumer-side when uncertain (per §11.4.17 — narrower scope is cheap to widen later, the reverse is expensive).
2. **Terminology anchor.** When prose in any file in this constitutional family references "the parent CLAUDE.md" or "the root Constitution," the referent is the constitution-submodule file at `constitution/<filename>` , never the consumer's file. When it references "the project CLAUDE.md" or "this project's

AGENTS.md," the referent is the consumer-side file at `<project-root>/<filename>`. AI agents reading this constitution MUST resolve ambiguous pronouns ("the CLAUDE.md", "the constitution") via this rule.

3. **Universal rules added to the consumer side are misfiled.** Per §11.4.17 commit-time check, before adding a `MUST` to the consumer file, the author assesses whether it would benefit other projects that consume this constitution. If yes, lift it to the constitution submodule first.

4. **Project-specific rules added to the constitution submodule are misfiled.** Per §11.4.17, before adding a `MUST` to the constitution submodule, the author assesses whether it references project-private specifics. If yes, demote it to the consumer file (or genericise it to a universal form first).

5. **Propagation gates target both layers but their contents differ.** The existing `CM-COVENANT-114-N-PROPAGATION` gate family verifies that anchor TEXT for §11.4.N exists in EVERY `CLAUDE.md` and `AGENTS.md` across the project. The constitution-submodule files carry the **canonical** text; the consumer files carry **compact propagation pointers** that reference the canonical authority by file path. Both forms count as valid presence; the gate is satisfied when any form is found.

6. **`@constitution/CLAUDE.md` import (Claude-Code-style) and the pointer-block fallback (Aider/Codex/Gemini-style) are equivalent.** The consumer's CLAUDE.md MUST start with one of:

   - The native import: `@constitution/CLAUDE.md`
   - The portable pointer-block per the `## INHERITED FROM constitution/CLAUDE.md` heading defined in `constitution/CLAUDE.md` "How inheritance works".

7. **No silent demotion or silent promotion.** Moving a rule between layers MUST be a visible commit — `git mv` of a section if it's a clean clone, or an explicit "Lifted from to constitution per §11.4.35" / "Demoted from constitution to per §11.4.35" line in the commit message. AI agents MUST NOT silently re-author a §11.4.X anchor in the wrong layer and call it propagation.

Pre-build gate (recommended, per consuming project):

- **`CM-CANONICAL-ROOT-CLARITY`** — verifies (a) consumer's `CLAUDE.md` opens with the inheritance pointer (either `@import` or `## INHERITED FROM constitution/CLAUDE.md` heading), (b) the constitution submodule's three files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution submodule's own files (those ARE the source-of-truth, not consumers).

Propagation. Composes with §11.4.17 (universal-vs-project classification — §11.4.35 defines the file-layer split that §11.4.17 classifies INTO), §11.4.18 (script documentation discipline — same canonical-vs-consumer file-layer rule applies to docs/scripts/). Reading order: this anchor first, then §11.4.17 for the classification rule.

No escape hatch. Misfiling a rule between layers IS a §11.4.35 violation regardless of intent. The fix is `git mv` + commit per invariant 7, not "leave both copies in place to be safe" — duplicates silently diverge over time.

2026-05-20 (§11.4.74)

§ . . — Submodule-catalogue-first discovery + extend-don't-reimplement (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times if they are already done as some of existing universal, reusable general development purpose Submodules! For any missing features that some Submodules we incorporate may be missing we MUST IMPLEMENT the properly and extend those Submodules furter! We do control all of the and we CAN and MUST maintain and extend the regularly! All development cycle rules we have MUST BE applied to them and fully respected!"

The mandate. Before scaffolding ANY new module, package, helper, or utility, the contributor (human or AI agent) MUST:

1. **Survey the canonical Submodule catalogue.** Two organisations are authoritative:
   - `vasic-digital` on GitHub (`https://github.com/vasic-digital`) AND GitLab (`https://gitlab.com/vasic-digital`).
   - `HelixDevelopment` on GitHub (`https://github.com/HelixDevelopment`) AND GitLab (`https://gitlab.com/HelixDevelopment`). Both organisations mirror across all four supported hosts per §2.1 — surveying one mirror MUST yield the full catalogue.
2. **Inventory existing Submodules.** The canonical repo INDEX is `submodules-catalogue.md` in this submodule (landed 2026-05-20 per user follow-up mandate). It enumerates every owned-by-us repository under `vasic-digital` + `HelixDevelopment` (142 repos at landing time) with a one-line capability description, categorised so consuming projects can locate "is there already a Submodule for X?" in one glance. The catalogue MUST be re-generated quarterly (or after any non-trivial repo create/rename) per its §3 regeneration recipe.
3. **Reuse before reimplement.** If a Submodule already provides the functionality (or 80%+ of it), the consuming project MUST add the existing Submodule as a Git submodule, not write the functionality fresh.
4. **Extend in-place when 80%+ matches but features are missing.** When an existing Submodule is close-but-not-complete, the missing features MUST be added TO THAT SUBMODULE (PR upstream + bump consuming project's submodule pointer) — never as a separate consuming-project helper that duplicates the 80% already in the Submodule.
5. **Same development-cycle rules apply.** Every Submodule under the two orgs is subject to: §11.4 anti-bluff covenant, §1.1 paired-mutation tests, §11.4.10 credentials handling, §11.4.61 metadata table + ToC, §11.4.65 universal Markdown export, §11.4.73 spec versioning (if the Submodule has its own spec), §2.1 multi-mirror push, §3 propagation order. Maintenance + extension of those Submodules MUST follow these rules without exception.

6. **Document the survey result.** Each new feature's tracker entry (Issues.md row, ADR, PR description, or equivalent) MUST contain a `Catalogue-Check:` field with one of three values:
    - `Catalogue-Check: reuse <org/repo>@<sha>` — the existing Submodule covers this.
    - `Catalogue-Check: extend <org/repo>@<sha>` — extending an existing Submodule; reference the upstream PR.
    - `Catalogue-Check: no-match <date>` — surveyed both orgs on the named date; no existing Submodule covers this functionality; the new code is justified to live in-project. Operators reviewing the PR re-validate this claim.

Anti-bluff captured-evidence gate (planned). `CM-SUBMODULE-CATALOGUE-CHECK`:

- For every new module / package introduced in a non-trivial PR (heuristic: ≥ 200 LOC of new non-test code), the gate scans the PR description and any linked tracker row for a `Catalogue-Check:` line. Absence is a FAIL.
- The gate's paired mutation strips the line from a PR and asserts the gate FAILs.

Composition. Composes with §3 (submodule commits propagate first), §4 (every tag on the main repo MUST be mirrored on every owned submodule), §11.4.36 (mandatory `install_upstreams` on clone/add), §11.4.31 (Submodule-Dependency-Manifest), §11.4.32 (post-pull validation), §11.4.28 (Submodules-As-Equal-Codebase).

Why this matters. Without the catalogue-check, consuming projects re-implement UUIDv7 helpers, RLS-tenant-context wrappers, Pandoc-multi-format exporters, install_upstreams shell scripts, Markdown ToC generators, fnv1a32 hash wrappers, etc. — fragmenting the catalogue and forcing the next project to choose between three subtly-different implementations of the same thing. The mandate makes the catalogue load-bearing.

Classification: universal (per §11.4.17). Applies to every project that consumes this Constitution.

Canonical authority: constitution submodule `Constitution.md` §11.4.74.

Non-compliance is a process violation; severe cases (duplicate implementation landed without catalogue check) are release blockers.

2026 05 20

---

§ . . — Mechanical Enforcement Without Exception (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "Why do these violations still happen!? This is a serious problem! We cannot rely on stability nor consistency if we cannot respect our Constitution, mandatory rules and constraints! Is there a way to make this always respected, followed and applied without exception fully and unconditionally!? WE MUST HAVE THIS WORKING FLAWLESSLY!!! Do investigate the root causes of such problems! Once all problems are identified WE MUST apply proper mechanisms for this not to happen NEVER EVER AGAIN!"

§-slot history note. This anchor was originally drafted as §11.4.74 but renumbered to §11.4.75 per the §11.4.41 merge-first mandate after upstream landed a different §11.4.74 (Submodule-catalogue- first discovery) concurrent with this work. Same renumber pattern as Phase 39.DH §11.4.41 collision resolution (commit `8ad51547115` of the parent consuming project's repo).

The §11.4 covenant ("tests pass while feature broken-for-end-user") historically relied on agent and operator vigilance to enforce. Three forensic incidents in 2026-05-19→20 demonstrated the failure: subagent `a84d2c86f90fdc297` committed `docs/research/workstation/Configurations.md` without HTML+PDF siblings (§11.4.65 violation) when it stalled at the 600s watchdog before its pandoc-export step; subagent `a6b38eedf4ef796e8` dispatched to remediate ALSO stalled at the same operational seam; the parent CLAUDE.md / AGENTS.md drift after §11.4.66 propagation required a dedicated catch-up commit `f93b25a92eb`. The common failure mode: late-binding enforcement that fires at `pre_build_verification.sh` time — hours-to-days after the violator commit reached every remote.

The mandate. Constitutional invariants MUST be enforced mechanically at FIVE independent layers — bypassing any single layer does not bypass the discipline. The five layers are:

1. **Local `pre-commit` git hook** — refuses staged `.md` lacking sibling `.html` + `.pdf` (and other staged-only invariants).
2. **`commit_all.sh` integration** — the canonical project commit script invokes the same checks + auto-runs `sync_all_markdown_exports.sh` to self-repair before the commit.
3. **Local `pre-push` git hook** — re-runs siblings + propagation gate subset on every commit in the push.
4. **`post-commit` auto-repair hook** — detects orphan `.md` in the just-committed manifest, auto-generates siblings via pandoc + weasyprint, creates a `chore(§11.4.75): auto-export ...` follow-up commit. Idempotent + recursion-guarded.
5. **Local-only equivalent** (Phase 39.GF, User mandate 2026-05-20). Remote CI surfaces (GitHub Actions, GitLab pipelines, Jenkins, CircleCI, etc.) are DISABLED per User mandate 2026-05-20 — the workflow file is preserved at `.github/workflows/constitution-compliance.yml.disabled-local-only` so its contents survive for re-enable but GitHub Actions does NOT execute it (only `.yml` / `.yaml` files under `.github/workflows/` are honoured). Layer 5 enforcement migrated to the LOCAL pre-build- verification + meta-test run that the operator MUST execute before tagging: `bash device/rockchip/rk3588/tests/pre_build_verification.sh`
    - `bash scripts/testing/meta_test_false_positive_proof.sh`. Layers 1-4 remain authoritative; Layer 5 is now the operator's local final- gate ritual. A future re-enable PWU may re-establish remote CI.

Helper contracts (mandatory):

- `scripts/install_git_hooks.sh` — idempotent installer that copies hooks from tracked `scripts/git_hooks/` into `.git/hooks/`. Hooked into `scripts/setup.sh` so fresh clones auto-install. Verifies itself with `--verify` flag.
- `scripts/git_hooks/pre-commit` — Layer 1 implementation.
- `scripts/git_hooks/pre-push` — Layer 3 implementation.

- `scripts/git_hooks/post-commit` — Layer 4 implementation.
- `scripts/git_hooks/commit-msg` — Bypass-rationale enforcement.
- `_constitution_sibling_check` function in `scripts/commit_all.sh` — Layer 2 implementation.

Bypass policy. No layer is perfectly bypass-proof in isolation — that is the design. Bypassing ALL FIVE simultaneously is mechanically impossible because:

- Layers 1 + 3 can be bypassed by `--no-verify`. Layer 5 (now local pre-tag ritual after Phase 39.GF) re-runs the same check before tag creation; tags are NEVER created without it per §11.4.40.
- Layer 2 can be bypassed by direct `git commit`. Layer 1 catches that.
- Layer 4 cannot be bypassed by `--no-verify` (runs after commit lands).
- Layer 5 is now local-only (Phase 39.GF — User mandate 2026-05-20). The operator runs `pre_build_verification.sh` + meta-test before every tag per §11.4.40 full-suite-retest mandate. Skipping the local Layer 5 ritual blocks tag creation.

Legitimate bypasses MUST be audited. When `--no-verify` is detected via touch-file marker `.git/ATMO_LAST_BYPASS_ATTEMPT`, the `commit-msg` hook REQUIRES a `Bypass-rationale: <reason>` footer in the commit message. `docs/audit/bypass_events.md` accumulates the audit trail per §11.4 captured-evidence requirement.

Captured-evidence enforcement. Five pre-build gates with paired §1.1 meta-test mutations:

- `CM-COVENANT-114-75-PROPAGATION` — anchor literal across canonical files.
- `CM-GIT-HOOKS-INSTALL-SCRIPT` — installer present + executable.
- `CM-GIT-HOOKS-SOURCE-DIR` — 4 hook bodies present + executable.
- `CM-COMMIT-ALL-SIBLING-CHECK` — `_constitution_sibling_check` in `commit_all.sh`.
- `CM-CI-WORKFLOW-PRESENT` — CI workflow + ≥3 required jobs.

Composes with §1.1 (paired meta-test mutations), §9 (data safety), §11.4 (end-user-quality covenant — Layer 5 CI enforcement is the §11.4 promise made mechanical), §11.4.1 (FAIL-bluffs forbidden), §11.4.4 (test-interrupt-on-discovery), §11.4.5 (audio + video quality), §11.4.6 (no-guessing), §11.4.41 (pre-force-push

merge-first — this very anchor's renumber from §11.4.74 → §11.4.75 was driven by §11.4.41 merge-first discipline), §11.4.43 (TDD-fix), §11.4.51 (live-ADB-first), §11.4.52 (autonomous-validation), §11.4.58 (PWU pipeline), §11.4.65 (universal Markdown export — Layer 1+3+4 are §11.4.65's mechanical seam), §11.4.66 (interactive-clarification), §11.4.67 (target-shell- parseability — hooks parse under bash AND mksh), §11.4.69 (universal sink-side positive-evidence), §11.4.70 (subagent-driven execution), §11.4.71 (pre-push fetch + integrate — Layer 3 + 5 honour the merge- first pipeline), §11.4.72 (audio top-priority — hooks do not race against in-flight audio commits because they hold no lock themselves), §11.4.73 (main-spec versioning — when spec exists, hooks include it), §11.4.74 (submodule-catalogue-first — hooks can be added to the canonical catalogue for cross-project reuse).

No escape hatch. No `--skip-hooks`, `--bypass-enforcement`, `--allow-orphan-md`, `--ci-not-applicable`, `--mechanical-enforcement-not-needed` flag exists. The `--no-verify` route IS the deliberate audit-trail bypass; §11.4.75 makes the audit trail mechanical via the `commit-msg` footer requirement.

The mandate exists because the User mandate of 2026-05-20 is unambiguous: violations MUST NEVER EVER AGAIN happen. Reliance on vigilance has demonstrably failed three times in 24 hours. Only mechanical enforcement at every layer can deliver the discipline the project requires.

Propagation gate `CM-COVENANT-114-75-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.75.

Non-compliance is a release blocker regardless of context. 2026 05 20

§ . . — Containers-submodule mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemy / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: https://github.com/vasic-digital/containers (git@github.com:vasic-digital/containers.git). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! All these details MUST BE added to our root constitution (constitution Submodule) - Constitution.md, AGENTS.md, CLAUDE.md, QWEN.md - and followed and respected! No bluff work is allowed of any kind!"

§-slot history note. This anchor was originally drafted as §11.4.75 but renumbered to §11.4.76 per the §11.4.71 fetch-before-push mandate after a concurrent upstream commit (`0a70083`) landed §11.4.75 (Mechanical Enforcement Without Exception). Same collision-resolution pattern as §11.4.75's own renumber-from-§11.4.74.

The mandate. For ANY containerized workload — Docker, Podman, Qemu, Kubernetes, container-backed emulators (Android emulator, VM-based testbeds, etc.) — every consuming project MUST:

1. **Use the canonical Containers Submodule.** `https://github.com/vasic-digital/containers` (`digital.vasic.containers`) is the authoritative library for runtime auto-detection, endpoint discovery, lifecycle/health management, compose orchestration, cross-build, emulator integration, and on-demand service boot. Mirrored across the four §2.1-canonical hosts (GitHub + GitLab + GitFlic + GitVerse).
2. **Install as a Git submodule.** Container-orchestration concerns enter the consuming project as a `git submodule add git@github.com:vasic-digital/containers.git <path>/containers` . Project go.mod (or equivalent dependency manifest) consumes via `replace digital.vasic.containers => ./<path>/containers` during development; production builds resolve through pinned commit SHAs.

3. **Boot infrastructure on demand.** Tests, CLI doctor commands, and local-development workflows that depend on Postgres / Redis / OTel / message-brokers / emulators MUST invoke the Containers Submodule's `pkg/boot` + `pkg/compose` + `pkg/health` APIs to bring infra up automatically. Operators MUST NOT be required to manually start `podman machine` / `docker compose up` before tests — the boot is part of the test entry point. This is the **on-demand-infra invariant**.
4. **Extend the Containers Submodule, never reimplement.** Per §11.4.74's extend-don't-reimplement rule applied specifically to containerization: if a runtime (e.g., a new emulator type) or lifecycle primitive (e.g., a new health-check kind) is missing, the consuming project MUST add it to `vasic-digital/containers` (PR upstream + bump the consuming project's submodule pointer) — never as a parallel implementation inside the consuming project.
5. **Anti-bluff captured-evidence requirement.** Every integration test that claims to exercise a containerized component MUST actually boot that component via the Containers Submodule. Tests that pass without containers actually running (e.g., short-circuit fakes that bypass the boot path) are a §11.4 bluff violation. A passing test MUST imply the infra was up.
6. **Composition with §11.4.74.** The Containers Submodule itself is one of the Submodules surveyed by `Catalogue-Check`. A project tracker row touching containerization MUST record `Catalogue-Check: extend vasic-digital/containers@<sha>` (or `reuse`) — `no-match` is only valid if the consuming project demonstrates the Containers Submodule's API cannot model the workload (rare).

Anti-bluff captured-evidence gate (planned). `CM-CONTAINERS-USED`:

- For every PR that touches `Dockerfile*`, `*compose*.yml`, `*podman*.yml`, `Vagrantfile`, or any code under a directory matching `(test|integration|e2e)/.*infra` heuristic, the gate scans for an import of `digital.vasic.containers/...`. Absence in a non-trivial container-touching PR is a FAIL.
- The gate's paired mutation strips the import and asserts the gate FAILs.

Composition. Composes with §11.4.74 (catalogue-first discovery), §11.4.75 (mechanical enforcement — this clause IS one of the things mechanical enforcement protects), §3 (submodule propagation), §11.4.31 (Submodule-

Dependency-Manifest), §11.4.36 (`install_upstreams` on clone), §11.4.28 (Submodules-As-Equal-Codebase: the Containers Submodule itself follows all dev-cycle rules), §11.4.65 (multi-format export for docs maintained inside the Containers Submodule), §1.1 (paired-mutation gates protect the Containers Submodule's own boot / health primitives).

Why this matters. Without the on-demand-infra invariant, integration tests silently degrade to "pass without infra running" (a §11.4 bluff) or break with cryptic "connection refused" errors that operators waste hours debugging. The Containers Submodule turns infra-boot into a typed, programmable, anti-bluff-tested capability rather than a sequence of imperative shell commands. Consuming projects converge on a single shared lifecycle/health model instead of each project hand-rolling their own.

Classification: universal (per §11.4.17). Applies to every project that runs code inside any containerization or virtualization runtime.

Canonical authority: constitution submodule `Constitution.md` §11.4.76.

Non-compliance is a process violation; landing containerized infra without consuming the Containers Submodule (i.e., reinventing compose orchestration in-project) is a release blocker.

2026 05 20

§ . . — Regeneration-mechanism-required mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content! Add this mandatory safety rule / constraint into ours root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md, QWEN.md or any other required document / file from the constitution Submodule! Do not forget to fetch and pull Submodule first! Commit and push all Submodule changes to all upstreams!"

Forensic incident (2026-05-20T15:00Z, parent project, a consuming project). The parent's audio Tier 1 `commit_all.sh` stalled four hours on `git add -A` while scanning 274 GiB of untracked `.git-backup-20260520T084610Z-pre-orphan-kill/` + 159 GiB `RKTools/linux/` + 167 GiB `qa-results/` content — all of which should never have been tracked. The natural remediation is to add those paths to `.gitignore`, BUT doing so without a regeneration mechanism would silently orphan fresh clones: a new operator running `git clone && cd <project>` would find missing RKTools, missing test infrastructure, missing build outputs they cannot reproduce. §11.4.77 codifies the universal rule that closes this gap.

The mandate. Every `.gitignore` entry that excludes either (a) more than ~100 MiB of content, OR (b) any artifact essential to building / running / testing the project, MUST be accompanied by a documented + automated mechanism that re-obtains or re-generates the excluded content on a fresh clone. The closed-set choices are:

1. **Re-obtain** — download from an authoritative source (vendor tarball, SDK installer, npm / pip / cargo / go-mod registry, container image registry, dedicated git submodule, S3, GCS, vendor-internal artifact server) via a script that runs deterministically on a fresh clone.
2. **Re-generate** — produce the content from tracked source code via a script (build pipeline, code generation, asset rendering, captured-evidence replay, container build, kernel build, image packaging) that runs deterministically on a fresh clone.

Required artefacts per qualifying `.gitignore` entry (each is independently mandatory):

1. **Metadata file at `.gitignore-meta/<entry-slug>.yaml`** (or equivalent project-canonical location declared in the project's `CLAUDE.md`) carrying: the gitignore-pattern verbatim, `mechanism-type: re-obtain | re-generate`, `script-path: <repo-relative path>` (the deterministic regeneration entrypoint), `expected-disk-usage: <bytes-or-human-readable>`, `vendor-url-or-source: <URL or in-tree path>` when applicable, `integrity: { algorithm: sha256 | md5, value: <hex> }` when the content is fetchable from a known-stable mirror, `requires-network: true | false`, `requires-credentials: true | false` (and which credentials slot per §11.4.10).

2. **Entry in the post-clone bootstrap script** (`scripts/setup.sh` or the project-canonical equivalent). The script MUST run the regeneration mechanism non-interactively on a fresh clone or be invocable as an idempotent step (`scripts/setup.sh --regenerate <entry-slug>`).
3. **Pre-build gate** that verifies either the regenerated content is present at the expected path OR the regeneration script ran successfully (e.g., a stamp file under `.gitignore-meta/.regenerated/<entry-slug>.ok` with mtime ≥ source freshness window).
4. **Documentation update** — README plus the relevant `docs/guides/*.md` guide describing the mechanism, including the manual fallback (vendor URL, alternative download mirrors), the time + disk-usage budget, and per-§11.4.10 credentials requirements.

No escape hatch. Adding a bare `.gitignore` entry that excludes significant or essential content without the accompanying mechanism is itself a §11.4 PASS-bluff variant — the codebase appears complete to the casual eye, every pre-build / post-build gate goes green, but a fresh clone cannot build / run. There is no `--skip-regen-mechanism` , `--gitignore-is-enough` , `--operator-already-has-content` flag.

Composition with sister anchors:

- **§11.4.6 (no-guessing)** — the mechanism MUST be verified working end-to-end on a sandbox clone before the `.gitignore` entry lands. `LIKELY works on a fresh clone` is the exact bluff §11.4.6 forbids.
- **§11.4.65 (universal Markdown export)** — generated HTML / PDF siblings are an instance of `re-generate` ; the auto-regeneration helper (`sync_all_markdown_exports.sh`) is the §11.4.77 mechanism for that gitignored content where applicable.
- **§11.4.66 (interactive clarification)** — if the agent is uncertain whether a candidate `.gitignore` addition qualifies under §11.4.77, the agent MUST ASK the operator via the platform's interactive question mechanism rather than guess.
- **§11.4.71 (pre-push fetch + integrate)** — the regeneration mechanism's integrity hash and vendor URL MUST be re-validated against upstream before pushing the §11.4.77-governing commit; stale hashes are a §11.4.6 violation.

- **§11.4.74 (catalogue-first + extend-don't-reimplement)** — if the regeneration mechanism can be implemented by extending a Submodule already in `vasic-digital/HelixDevelopment` (e.g., a reusable downloader / decompressor helper), the project MUST extend rather than reimplement.
- **§11.4.75 (Mechanical Enforcement Without Exception)** — the local `pre-commit` hook (Layer 1) MUST refuse a commit that adds new `.gitignore` lines without a corresponding `.gitignore-meta/*.yaml`; the CI workflow (Layer 5) MUST replay the gate on every push. Paired meta-test mutation: strip a metadata YAML entry → gate FAILs.
- **§11.4.76 (Containers-submodule mandate)** — container images excluded from VCS are regenerated via `vasic-digital/containers` `pkg/boot` + Dockerfiles tracked in-tree; the `.gitignore-meta/<entry-slug>.yaml` references the Containers Submodule build path.
- **§9 / §9.2 (zero-risk data safety)** — the regeneration mechanism MUST be pre-tested in a sandbox clone with a hardlinked-backup safety net BEFORE relying on it for live operator clones; a regeneration script that silently corrupts content on a fresh clone is a §9 violation.
- **§3 (propagation order)** — submodules consuming this constitution submodule inherit §11.4.77; their own `.gitignore` entries are subject to the same mechanism requirement, propagated via their own `CLAUDE.md` / `AGENTS.md` / `QWEN.md`.

Anti-bluff captured-evidence gate (planned). `CM-GITIGNORE-REGEN-MECHANISM`:

- For every PR that adds a line to `.gitignore` (or to any `*.gitignore` file in a sub-tree), the gate scans for a matching entry in `.gitignore-meta/`, verifies the `script-path` exists and is executable, verifies the metadata YAML parses and carries the required keys, and verifies the post-clone bootstrap references the mechanism.
- Bare-line additions (e.g., `echo 'qa-results/' >> .gitignore`) without the metadata sibling are a FAIL.
- The gate's paired §1.1 mutation strips one required YAML key (e.g., `script-path:`) and asserts the gate FAILs.

Why this matters. Without §11.4.77, the natural `.gitignore` ergonomics — "this is too big to track, exclude it" — leads to fresh-clone orphanage: operators or CI runners (or new developers, or end-user mirror sites) clone the project and discover essential content is missing, with no documented way to obtain it. The cost is paid at every fresh-clone event, every CI bootstrap, every operator onboarding, every disaster-recovery exercise. §11.4.77 forces the cost to be paid once, at the `.gitignore`-addition commit, where it is bounded and visible.

Forensic anchor crosswalk to today's incident. Without §11.4.77, the natural fix to today's `commit_all.sh` 4-hour-stall — adding `.git-backup-*`, `RKTools/linux/`, `qa-results/` to `.gitignore` — would have orphaned every fresh clone of a consuming project. With §11.4.77, each of those entries now requires: (a) a `.gitignore-meta/<slug>.yaml` declaring the mechanism (e.g., `.git-backup-*` is a transient artefact safe to re-create per §9.2 hardlinked-backup; `RKTools/linux/` is re-obtainable from Rockchip's vendor tarball; `qa-results/` is re-generatable by re-running the on-device test cycle); (b) bootstrap-script entries; (c) gates; (d) docs. The bare `.gitignore` addition is now mechanically blocked.

Classification: universal (per §11.4.17). Applies to every project that excludes content from VCS for size / regenerability / vendor-provenance reasons.

Canonical authority: constitution submodule `Constitution.md` §11.4.77.

Non-compliance is a release blocker regardless of context.

2026 05 20

---

§ . . — CodeGraph code-intelligence mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "All gathered knowledge and steps MUST BE added as future reusable experience into our constitution Submodule on setting up / configuring and using codegraph with all supported CLI agents we may use for our codebase! Make codegraph MANDATORY CHOICE for this purpose for all of our project and add proper comprehensive details about everything codegraph related into our root constitution (constitution Submodule) Constitution.md, AGENTS.md, CLAUDE.md, QWEN.md and other constitution Submodule relevant files! All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!"

The mandate. Every consuming project under this Constitution MUST install, initialize, and use **CodeGraph** (`https://github.com/colbymchenry/codegraph` , npm package `@colbymchenry/codegraph`) as its code-intelligence layer for AI coding agents. CodeGraph builds a local SQLite semantic knowledge graph of a codebase and exposes it to agents over the Model Context Protocol (MCP) — 100% local, no cloud, no external API (compatible with the Local-Only CI/CD posture). A project worked on by AI coding agents that has not yet installed and wired CodeGraph MUST do so; using it is not optional.

1. **Install.** `npm install -g @colbymchenry/codegraph` (Node.js 18+). The npm global prefix MUST be user-writable — installation MUST NOT require `sudo` / `su` (§9 / §12 host-safety + the no-privilege-escalation posture). Native modules (`better-sqlite3` , `tree-sitter`) compile on install with a WASM fallback.
2. **Initialize + index.** `codegraph init` at the project root creates `.codegraph/config.json` (tracked) and `.codegraph/codegraph.db` (a regenerable build artifact — gitignored per §11.4.30 / §11.4.77, with `codegraph index` as its declared regeneration mechanism). The `config.json` `exclude` list MUST exclude (a) other-owned submodules and vendored trees — index the consuming project's own domain code — and (b), non-negotiably, every credential/secret path (`.env*` , keystores, signing keys, service-account JSON) per §11.4.10. A secret reaching the index is a §11.4.10 violation. `codegraph index` builds the graph; `codegraph sync` keeps it fresh.

CRITICAL — filter mechanism. CodeGraph v1.x is zero-config: it uses BUILT-IN skip lists (`node_modules` , `vendor` , `dist` , `build` , `target` , `.venv` , `Pods` , `.next`) PLUS the project root `.gitignore` to decide which files to index. The `.codegraph/config.json` `include` / `exclude` fields are **not used** for file filtering — all filtering happens through the ignore matcher seeded with the built-in defaults + root `.gitignore` patterns. To control what gets indexed, use root-anchored `.gitignore` patterns (start with `/` to match only the top-level directory) and `!` negations to re-include owned paths under a default-skipped parent (e.g., `!/vendor/widevine_*` re-includes owned vendor code that CodeGraph's built-in `vendor` skip would otherwise block).

MUST-EXCLUDE classes (actual rubbish — build outputs, caches, credentials, binaries):

```
**(a) Build outputs and regenerable artifacts:** `/out/`, `/prebuilts/
`, `/rockdev/`, `/RKTools/`, `.compressed/`.

**(b) Caches, dependencies, tooling:** `.gradle/`, `__pycache__/`,
`.venv/`, `flash_logs/`, `logs/`.

**(c) Credentials, secrets, QA/recording artifacts:** `/secrets/`,
`scripts/testing/secrets/`, `**/*.env`, `/qa-results/`, `/recordings/`.

**(d) Config.json include/exclude (DEPRECATED — not enforced by v1.x
engine):** maintained as documentation-only for future compatibility.
```

ALL SOURCE CODE MUST REMAIN INDEXABLE, including AOSP platform trees (`frameworks/` , `external/` , `art/` , `cts/` , `bionic/` , `packages/` , `system/` , `kernel-5.10/` , `hardware/*/` , `vendor/` , `device/*/`) — these are tracked source and MUST NOT be gitignored for CodeGraph purposes. Only the rubbish classes above may be excluded. A project with 1M+ tracked files (AOSP fork) will produce a correspondingly large index (30-40 GB for the first init, with subsequent `codegraph sync` operations fast because they use `git diff`). The large initial size is expected and correct — the index represents the full codebase the AI agent needs to understand cross-references.

Universal mandatory-exclude baseline set (§11.4.78 + §11.4.79 refinement — binds EVERY consuming project). Independently of project, the CodeGraph index MUST NEVER include the following file-classes anywhere —

they are indexed-rubbish in every project; a consuming project MAY add its own project-specific rubbish paths on top per §11.4.35 but MUST NOT drop any baseline class: (a) **build outputs / regenerable artifacts** — `/out/`, `/build/`, `/dist/`, `/target/`, `bin/`, `obj/`, `.compressed/`, and any generator-produced tree; (b) **caches / dependency / tooling dirs** — `**/ccache/`, `**/.ccache/`, `.gradle/`, `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`, `node_modules/`, `.next/`, `.cache/`, `.terraform/`, `.venv/`, `logs/`; (c) **credentials / secrets** — `.env`, `.env.*`, `**/*.env`, `secrets/`, `**/*secret*`, keystores / signing-keys / service-account JSON, and every credential-bearing path per §11.4.10 (a secret reaching the index is a §11.4.10 violation, never negotiable); (d) **QA / recording / large regenerable corpora** — `**/qa-results/`, `**/recordings/`, and prebuilt-binary trees. Realized via root-anchored `.gitignore` patterns (CodeGraph v1.x zero-config filter = built-in skip list + root `.gitignore`, NOT `config.json` include/exclude). Own-org submodule sources stay INCLUDED per §11.4.79; third-party vendored trees EXCLUDED. This baseline is UNIVERSAL (§11.4.17) — dropping any class is a §11.4.78 / §11.4.79 violation, and no source-code tree may be excluded to satisfy it.

3. **Wire every supported CLI agent.** The CodeGraph MCP server (`codegraph serve --mcp`, stdio transport) MUST be registered with every AI coding CLI agent the project's developers use. Registration is project-scoped and committed where the agent supports it (e.g. Claude Code `.mcp.json`, OpenCode `opencode.json`, Qwen Code `.qwen/settings.json`, Crush `.crush.json`); host-local where the agent stores MCP config outside the repository (e.g. Kimi CLI `~/.kimi/mcp.json`). Every MCP config MUST reference the bare `codegraph` command resolved on `PATH` — never a hardcoded host path (the no-hardcoding posture). Claude Code is the canonical primary agent; the others MUST work too.

4. **Anti-bluff verification is mandatory.** CodeGraph integration MUST be covered by an anti-bluff verification suite (the canonical reference implementation is a consuming project's `scripts/verify-codegraph.sh` + `tests/codegraph/`, six layers): index reality, query correctness, MCP-protocol JSON-RPC, per-agent connectivity, per-agent end-to-end LLM drive, and a falsifiability rehearsal that mechanically proves the suite FAILs when the index is deliberately broken. The per-agent end-to-end layer MUST use an

unforgeable challenge — a fact obtainable only by calling a CodeGraph MCP tool (e.g. the index node count via `codegraph_status`) — so an agent answering from its own file-reading tools cannot produce a false PASS. An agent that genuinely cannot be driven end-to-end in the test environment (missing credentials, exhausted quota, environment incompatibility) is recorded as a documented SKIP gap per §11.4.3 — never a faked PASS.

5. **Comprehensive documentation.** Every project MUST carry a `docs/CODEGRAPH.md` (or canonical equivalent) describing install, initialization, indexing, per-agent wiring, the verification suite, and troubleshooting — kept in sync per §11.4.12 / §11.4.65.

6. **Catalogue-first.** CodeGraph is a third-party developer tool, not an owned submodule — it does NOT enter the project as a `git submodule` (per §11.4.74 it is consumed as the published npm package) and it does NOT add a Git remote.

Anti-bluff captured-evidence gate (planned). `CM-CODEGRAPH-WIRED`: for every consuming project, the gate verifies `.codegraph/config.json` exists with the §11.4.10 secret-exclusions present, every developer-used agent carries a CodeGraph MCP registration, and the verification suite exists and is executable. The paired §1.1 mutation removes a secret-exclusion from `config.json` and asserts the gate FAILs.

Composition. Composes with §11.4.3 (per-environment-topology SKIP for un-runnable agents), §11.4.10 (credentials never indexed), §11.4.12 + §11.4.65 (CODEGRAPH.md kept in sync + exported), §11.4.30 / §11.4.77 (the `.codegraph/codegraph.db` artifact is gitignored with `codegraph index` as its regeneration mechanism), §11.4.74 (catalogue-first — CodeGraph is consumed, not reimplemented), §11.4 (the verification suite is anti-bluff by construction — CI green is necessary, never sufficient), §1.1 (paired-mutation gate).

Why this matters. AI coding agents that explore a codebase by repeated file scanning consume tokens and tool calls wastefully and build a shallow, drift-prone mental model. A pre-indexed semantic graph gives every agent instant, consistent symbol / caller / callee / impact resolution. Standardising on one tool — CodeGraph — across every Helix project means the capability is wired once, verified anti-bluff once, and reused everywhere, instead of each project hand-rolling ad-hoc context tooling.

EXTENSION — scope, launcher, runner patches and verification are CONSUMED BY REFERENCE; the consumer supplies only DATA (research-derived, 2026-09-24). Measured this cycle on a multi-hundred-thousand-file consuming project: clause 2's filter text contradicted itself and §11.4.79 steps 2–3 (one sentence says the `.codegraph/config.json` `exclude` list is not used for filtering, another instructs editing it), the installed CLI version read a DIFFERENT config file that did not exist in the repository and silently fell back to zero-config, nothing derived the scope from the submodule graph, and the anti-bluff gate treated any database above a size threshold as a real index — so a stalled, partial index would have PASSED. This extension closes those gaps without deleting any text above; where it conflicts with the clause-2 filter wording, THIS extension governs (§11.4.227 extend-don't-re-mint).

1. **ONE MECHANISM, INHERITED BY REFERENCE.** The scope machinery (`<constitution>/scripts/codegraph/scope_render.py` , the runner-discovery bridge `scope_enumerate.js` , the baseline class list `scope_baseline.txt` , the DATA template `scope.example.yaml`), the single sanctioned writer entry (`codegraph_safe.sh` + `codegraph_safe_helper.py` + `codegraph_guard.sh` + `disk_tripwire.sh`), the version-keyed runner patcher (`fk_index_patch.py` + `runner_patches/`), the bulk-window hazard probe (`fk_cascade_probe.py`), the progress-proven watchdog (`index_watch.py`), the scope proof (`codegraph_scope_guard.py`) and the semantic-index scope generator (`<constitution>/scripts/lumen/gen_lumenignore.py`) MUST live in the constitution submodule and MUST be INVOKED BY REFERENCE from the consuming project (§11.4.28 / §11.4.80 / §11.4.177). A consumer copy, fork, vendored duplicate or hand-rolled equivalent is a violation (§11.4.227 — no parallel mechanism). Each tool MUST be documented (§11.4.18) and test-first with paired mutations (§11.4.224 / §1.1), and a tool lacking its own tests MUST NOT be pointed at a live index; at the time of writing `index_watch.py` and `fk_cascade_probe.py` (no test references of their own — the writer entry's unit test substitutes a stub for the probe), the runner patches `lockfix1` and `datafrag1` (no tests of their own — excluded from the default patch set for that reason), and the §11.4.18 companion guide of EVERY tool named here (none exists yet) are OWED, tracked per §11.4.197.

2. **THE CONSUMER SUPPLIES DATA, NEVER MECHANISM (§11.4.35).** A consuming project contributes exactly one declarative scope file (schema = the constitution's `scope.example.yaml`): its own-org list, its project-specific rubbish paths, its pathological inputs (files that stall the parser — declared, never retried to timeout), explicit re-include negations, per-submodule class overrides WITH a reason, the accepted indexed-file count and its tolerance, and its efficiency thresholds (§11.4.275(D)). It MUST NOT drop a baseline class (the universal baseline above still binds).
3. **CONFIG TRUTH = WHAT THE INSTALLED CLI ACTUALLY READS.** The scope source of truth is the configuration the INSTALLED CLI version loads, established from that version's own loader code (§11.4.99 latest-source, §11.4.6 never from memory); a config file the CLI does not read is dead documentation and MUST NOT be cited as the scope. The effective scope artifact is GENERATED by the renderer from the baseline classes + the §11.4.79 classification + the consumer DATA; hand edits are refused, and a missing or malformed config that the CLI silently falls back from is a FAIL, never a warning.
4. **SCOPE PROOF BEFORE "READY" (supersedes size-based evidence).** Before an index is called usable, the scope guard MUST enumerate files through the installed runner's OWN discovery code and assert, each with a control needle (§11.4.273): indexed-file count within the consumer tolerance of the accepted count; zero files under third-party roots; zero secret-class files (§11.4.10); at least one file under every own-org root (§11.4.79). Database size is NEVER evidence of a real index. Completeness (pending work == 0) is §11.4.275(C).

EXTENSION — host-adaptive V8 heap budget lands in the single writer entry, closing a real OOM crash on this consuming project's bulk index run (research-derived, 2026-09-25). Forensic FACT: on this constitution's own consuming project (584,328 tracked files, ~47.5M edges), a bulk `codegraph index` run reached 100% file parsing then crashed in the resolve phase with `FATAL ERROR: Reached heap limit ... JavaScript heap out of memory` (rc=134), discarding an approximately 85-minute run — on a host with 161 GiB of RAM available at the time. Root cause: a stock-launched Node.js indexer inherits V8's DEFAULT old-space heap ceiling (roughly 4 GiB) regardless of the host's real memory, since `codegraph_safe.sh` (the item-7 single sanctioned writer entry) never set one — a self-imposed software limit masquerading as real memory

exhaustion, exactly the class of un-measured resource quota §12.11 and §11.4.6 forbid. Fix: `preflight()` now computes a heap budget from MEASURED `MemAvailable` (half of it, floored at 8192 MiB so it always exceeds the stock default, capped at 65536 MiB so one process alone never threatens the §12.6 60%-of-total-RAM ceiling — never hardcoded, applying §12.11's dynamic-detection discipline to this bare-metal indexer process, not only to §12.11's own containerized-build scope) and exports it via `NODE_OPTIONS=--max-old-space-size=<budget>` before every `init` / `index` / `sync` launch; an unmeasurable budget REFUSES the run (`die 6`) rather than silently falling back to the unmeasured stock limit (§11.4.201 conservative-safe default). Landed with two new regression tests (`scripts/codegraph/tests/test_unit_safe.sh` U44/U45, item-7 test-first discipline) proving the exact floor/cap/proportional arithmetic, and independently VERIFIED LIVE (not merely unit-tested): the relaunched bulk run's real indexer process was confirmed via `/proc/<pid>/environ` to have genuinely received `NODE_OPTIONS=--max-old-space-size=65536` . Zero-regression proof: the full pre-existing test suite produced identical pass/fail counts before and after (git-stash comparison), isolating the delta to U44/U45 flipping FAIL→PASS. Companion doc updated per §11.4.18 (`docs/scripts/codegraph_safe.md`). Commit `941841d3e7235a081e3199a4d3073b409d41753b` . Composes §11.4.6 / §12.6 / §12.11 / §12.12 / §11.4.80(4)(6) (upgrade-gated compatibility probes and supervised long runs — this closes exactly the crash class those clauses anticipate for bulk runs) / §11.4.201 / §11.4.224 / §1.1.

CORRECTION (same day, operator mandate 2026-09-25 — "do not set caps for codegraph and lumen … indexed space exposed and used by all agents"): the "capped at 65536 MiB" clause above is SUPERSEDED, not deleted. The 64 GiB figure was an INVENTED ceiling, not a derivation of §12.6: on a 251 GiB host, 64 GiB is ~25% of total RAM, far below the actual §12.6 bound (60% ≈ 150 GiB) — an artificial cap that starved a legitimate large-repo resolve phase for no principled reason (observed live: the relaunched run's RSS climbed to ~58 GiB and plateaued against the invented ceiling, a genuine crash risk). Fixed: the ceiling is now `MemTotal * 60 / 100` (in MiB), computed fresh every preflight run from `/proc/meminfo` — the REAL §12.6 bound, never a fixed number. Mathematically this ceiling can never bind an ordinary run (half-of-available is always < half-of-total, which is always < 60%-of-total), so in practice the heap budget is now effectively unconstrained short of the one absolute, no-escape-hatch host-safety invariant this constitution carries — exactly matching the operator's mandate without weakening

§12.6. New regression test `test_unit_safe.sh` U46 proves the cap is real and independently computed (not inert, not the old hardcoded number) by constructing a scenario where it overrides even the 8192 MiB floor. Full-suite zero-regression proof unchanged in kind: `cases=46 failed=31`, same 31 pre-existing unrelated failures. Commit `8a5f47ab2e0e7a05d1c1652ba5a3475751652062`.

EXTENSION — CodeGraph is now wired as a genuine, project-tracked, first-class MCP tool for this consuming project (not only the bare-`codegraph`-on-PATH posture clause 3 above describes), closing a "tool exists but no agent can discover it" gap found under the same operator mandate (forensic incident + fix, 2026-09-25). Audit finding: prior to this extension, this project's `.mcp.json` was `{"mcpServers": {}}` — EMPTY — despite CodeGraph shipping a full first-class MCP query surface (`codegraph_explore`, `codegraph_node`, and more, confirmed from the installed CLI's own `--help` text) and its own installer (`codegraph install`) built for exactly this purpose. The gap was total: an agent had no way to discover CodeGraph's query tools short of already knowing to hand-invoke the raw CLI via Bash — the opposite of "regularly used for all major needs." Separately, the semantic (Lumen) tool's own "use this instead of grep" nudge, observed firing reliably all session, was traced to a Claude Code PLUGIN hook (`lumen/<version>/cmd/hook.go`) — entirely UNTRACKED by this repository; it works only by accident of which plugins happen to be enabled on a given host/account, and would not follow this project to a fresh clone, a different agent, or another track. `grep -rl "codegraph\|lumen" constitution/scripts/hooks/ scripts/hooks/` returned nothing — zero project-tracked enforcement of either tool's use existed anywhere.

Fix landed: (1) `.mcp.json` now registers a `codegraph` MCP server whose command is `bash constitution/scripts/codegraph/codegraph_mcp_serve.sh` — a new, project-tracked, §11.4.177-decoupled wrapper, NOT the bare `codegraph` command clause 3 above names. This is a REFINEMENT of clause 3's "no-hardcoding posture," not a violation: clause 3 forbids a HARDCODED HOST PATH, and the wrapper hardcodes nothing — it dynamically resolves the SAME validated, patched runner `codegraph_safe.sh` itself uses (via `fk_index_patch.py --print-bin`, re-derived every launch, correct automatically across a future `codegraph upgrade`) rather than trusting whatever `codegraph` happens to resolve first on `$PATH` (the STOCK, unpatched build — this project's own `codegraph_safe.sh` NEVER trusts stock for a write operation, per the

2026-09-23 incident item 7 above already documents; the wrapper extends that same discipline to MCP serving). Consuming projects with no equivalent patched-runner requirement MAY use clause 3's plain bare-`codegraph` form; this wrapper is the pattern for a project that, like this one, cannot safely assume stock-on-PATH is correct.

A SEPARATE, MORE SERIOUS forensic incident surfaced while building and testing this wiring, and is now a permanent, tested safety property of the wrapper. `codegraph serve --mcp` is NOT a passive read-only query server — with its file watcher enabled it auto-syncs (WRITES) the database on detected file changes, and `serve --mcp` daemonizes (re-parents to init, outliving its own launching process or a `timeout`-based kill — this is CodeGraph's own documented persistent-background-service architecture). A first version of the wrapper omitted `--no-watch`; a single ~5-second manual test of it against THIS project's own live checkout spawned a detached daemon that held `.codegraph/codegraph.db` / `-wal` / `-shm` open with an active write-capable file watcher for roughly two minutes before being found and killed — CONCURRENTLY with an in-flight bulk `sync` continuation also writing to the SAME database, which then hard-crashed with `Failed to index: database is locked`, discarding a further ~85 minutes of resolve-phase progress on top of the earlier heap-crash loss the same day. Root cause: two writers, one database, zero coordination — `codegraph_safe.sh`'s own single-writer flock/lock discipline governs only ITS OWN launches; an MCP server started via `.mcp.json` launches completely outside that mechanism and was never taught to respect it. Recovery: the stray daemon was killed, `PRAGMA integrity_check` found exactly 3 rows missing from one index (`idx_nodes_lower_name`, a mid-transaction-crash artefact, not page-level corruption), repaired via `REINDEX` (a data-presence check on the affected rows confirmed the underlying table data was intact before and after), and the bulk operation was resumed via `codegraph_safe.sh … sync` (BULK-classified on `pending>=threshold`, correctly continuing the interrupted resolve phase rather than re-parsing 584K files from scratch).

The durable fix: the wrapper now ALWAYS passes `--path <project> --no-watch` — `--no-watch` is not optional, ever, per the CLI's own documented behaviour ("Disable the file watcher (no auto-sync)"), making this a TRUE query-only reader that can run PERMANENTLY (as a registered MCP server necessarily does) alongside future bulk index/sync operations without risk of this collision class recurring; a permanently-registered MCP server WILL eventually overlap with a

future bulk run in an actively multi-tracked project — the two-writer collision is not a one-off testing artefact, it is the exact risk profile permanent registration creates. An occasionally-stale query view (never auto-refreshing) is the correct trade against a database corruption/crash risk; `codegraph sync` (or a fresh query once an agent knows a sync completed) is how staleness is cleared, deliberately, never automatically. Verified DECISIVELY, not by argv inspection alone (the daemon's own child process does not necessarily inherit its launching invocation's literal argv — verified empirically instead): against a real, freshly-`init`'d scratch project (never this repository's own checkout), the index database's mtime was read before and after editing a tracked file with the wrapper's server running, and found byte-for-byte unchanged after a 4-second settle — proving no auto-sync write occurred.

Six new regression tests landed (`scripts/codegraph/tests/test_codegraph_mcp_serve.sh`, item-7 test-first discipline): T01 (resolved runner is always the SAME path `fk_index_patch.py --print-bin` reports, never independently computed), T02 (`--path` + `--no-watch` are always present, never optional), T03 (`--path` targets exactly the resolved project, never cwd/guessed), **T04 — the incident itself, replayed as a repeatable regression test** (a real init'd scratch project, real `serve --mcp` launch, real file mutation, real DB-mtime proof of zero writes), T05 (`$CLAUDE_PROJECT_DIR`, when set, is used verbatim, never overridden by git-toplevel guessing), T06 (fails closed — non-zero exit — when patched-runner resolution cannot proceed, never silently falls back to an unverified binary). All 6/6 PASS. T01/T03/T05 initially failed on an unrelated wrapper bug (the wrapper hardcoded `python3 "$PATCH_TOOL"`, force-feeding a `.sh`-fixture stub to the Python interpreter in tests — fixed by reusing `codegraph_safe.sh`'s own `run_tool()` dispatch, which the production path never exercised since `$PATCH_TOOL` defaults to a real `.py` script there); found and fixed before landing, not after. A second, independent bug (the submodule-vs-parent project-root fallback used a plain `git rev-parse --show-toplevel`, which from inside a submodule resolves to the SUBMODULE's own root, not the consuming project's — fixed with `--show-superproject-working-tree`, tried first) was likewise found and fixed during this same investigation, not discovered later.

Honest boundary (§11.4.6). `.mcp.json` registration takes effect on a FRESH Claude Code session — this session, already running, cannot self-restart to verify the `codegraph_explore` / `codegraph_node` tools are genuinely callable end-to-end from inside it; that verification is OWED as a tracked §11.4.197 follow-up on the next fresh session in this project, not claimed here. This extension delivers and

tests the MECHANISM (a project-tracked, always-present MCP registration pointing at a verified-correct, incident-hardened, non-writing wrapper) — it does NOT, and cannot, mechanically guarantee that every agent ALWAYS chooses to call it over a raw grep/find/glob on every occasion; that remains a soft property of tool-selection behaviour, same honest limit §11.4.275(A)'s own accessibility clause already lives with for Lumen. What changed today is that the mechanism itself is no longer an accident of host/plugin configuration — it is now a committed, tested, portable part of this repository, present for any agent, any track, any future clone.

Composes §11.4.6 / §11.4.14 (cleanup on every exit path — every test daemon this investigation spawned was found and killed, none left running) / §11.4.35 / §11.4.78(3) (refines the bare-`codegraph`-on-PATH posture for a patched-runner project) / §11.4.78(7) (the wrapper is a new tool under the same inherited-by-reference / §11.4.18-documented / test-first discipline item 7 already mandates) / §11.4.102 (systematic-debugging root-cause-first — the incident was investigated to a proven, evidenced cause before any fix, not patched blind) / §11.4.108 (runtime-signature verification — the heap-budget correction was proven live on the real running process before being called done) / §11.4.177 (decoupled, inherited by reference) / §11.4.197 (the fresh-session verification is tracked, not silently dropped) / §11.4.201 (fail-closed on unresolvable runner state) / §11.4.224 (test-first) / §11.4.263 (process-group signal safety — every daemon kill in this investigation, manual and automated, validated real `/proc` cmdline identity before signaling, never a bare `pkill -f` carrier-match) / §11.4.273 (control-needle-proven measurement — the incident's own root cause was confirmed via `/proc/<pid>/cmdline` + daemon registry JSON, never assumed) / §11.4.275(A) (universal agent/subagent accessibility — this is its CodeGraph-side completion) / §1.1.

Classification: universal (per §11.4.17). Applies to every project that is worked on by AI coding CLI agents.

Canonical authority: constitution submodule `Constitution.md` §11.4.78.

Non-compliance is a process violation; a project worked on by AI agents without CodeGraph installed, wired, and anti-bluff-verified is in breach of this mandate.

§ . . — Own-org submodules MUST be included in the CodeGraph index (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-21):

> "All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!"

The mandate. This extends §11.4.78 step 2's exclude-list refinement with a per-submodule-ownership split. Every consuming project's `.codegraph/config.json` MUST distinguish:

Submodule class	Treatment in CodeGraph index
Own-org — full write access via the project's CLIs (canonical orgs: `vasic-digital` on GitHub/GitLab/GitFlic/GitVerse + `HelixDevelopment` on GitHub)	**MUST be INCLUDED.** Per-submodule sources MUST be indexed so AI coding agents resolve symbols, callers, and impact across the whole repo graph — not just the consuming project's domain code.
Third-party — no write access, vendored only (the §11.4.74 `no-match → vendor` path, e.g. an external SDK like `gopkg.in/telebot.v3`)	**MUST be EXCLUDED.** Indexing third-party code wastes the local index budget and creates symbol-noise the project's contributors cannot fix.

This refines, NOT contradicts, §11.4.78 step 2 (which spoke of "other-owned submodules" generically). The classifier is **write-access via the project's own CLIs**, not "internal vs external" subjectively — own-org submodules are the project's own code under another path; third-party submodules are upstream code Herald extends without owning.

Operational steps for every consuming project:

1. **Fetch + pull latest of every submodule** before re-indexing — `git submodule update --remote --merge` from the project root, then verify the project still builds. Third-party submodule pins (e.g. `gopkg.in/telebot.v3` pinned at `v3.3.8` to keep the Go import path stable) MUST be respected and rolled back if a `--remote` advances past the load-bearing pin.
2. **Adjust `.codegraph/config.json` `exclude` to keep own-org submodules in scope.** Third-party submodule paths MUST be explicitly listed in `exclude` (per §11.4.78 step 2's per-credential exclusion principle); own-org submodule paths MUST NOT appear in `exclude`.
3. **Re-index.** Run the project's canonical CodeGraph initialiser (e.g. `scripts/codegraph_setup.sh`) so `codegraph index` (or `codegraph sync`) rebuilds the index with the corrected exclude list.
4. **Verify symbols across own-org submodules.** The project's `scripts/codegraph_validate.sh` (per §11.4.78 anti-bluff verification) MUST probe at least one symbol that lives ONLY inside an own-org submodule (e.g. `bg.TaskQueue` from `submodules/background/interfaces.go`). A successful query proves the include path actually reached the submodule's sources — not a §107 PASS-bluff where validate runs against a stale index.
5. **Paired §1.1 mutation**: temporarily add the own-org submodule path to `exclude`, re-index, run validate — it MUST FAIL on the cross-submodule probe. Restore. This mutation proves the validate is not bluffing about the include reach.

Composition. Composes with:

- §11.4.74 (catalogue-first): the `own-org` classifier matches the `reuse/extend` catalogue-check disposition; `third-party` matches `no-match → vendor`.
- §11.4.78 (CodeGraph mandate): this §11.4.79 refines step 2's exclude-list contract without weakening any other step.
- §11.4.10 (credentials never tracked): the exclude list MUST still cover every `.env*`, keystore, signing key, service-account JSON regardless of submodule ownership.
- §1.1 (paired mutation): every validate probe added per step 5 above MUST land with a mutation pair.

- §107 (end-user usability): an indexed graph that lies about which symbols are reachable is a §107 PASS-bluff against the AI coding agents that consume it.

Why this matters. When AI coding agents work across a project that vendors own-org capability modules (e.g. `commons_constitution` consuming the Helix-stack 9-module surface under `submodules/`), excluding those modules from the index forces the agents back into shallow file-scan exploration of half the codebase — exactly the failure mode §11.4.78 exists to prevent. The agents lose call-graph resolution across the seam between domain code and own-org infra code. Including own-org submodules is the difference between agents seeing the project as a unified graph and agents tripping over an invisible boundary at the submodule directory.

EXTENSION — classification is DERIVED, not hand-maintained (research-derived, 2026-09-24). Measured this cycle: with no generator deriving scope from the submodule graph, dozens of third-party submodule roots (tens of thousands of files) were indexed while tens of thousands of tracked first-party source files were silently dropped by the engine's built-in skip list — neither error surfaced anywhere. Steps 1–5 above remain in force; the following clauses are added:

1. **Mechanical classification.** Own-org vs third-party is DERIVED by the constitution's renderer (§11.4.78(7)) by walking `.gitmodules` RECURSIVELY (nested gitlinks included) and matching each submodule's remote URL organisation against the consumer's own-org DATA list (§11.4.78(8)). Third-party code nested INSIDE an own-org submodule is third-party. A per-submodule override is DATA with a recorded reason, never an unexplained edit.
2. **Stale render fails.** Adding, removing or re-pointing a gitlink without re-rendering the scope makes the render stale; the scope guard (§11.4.78(10)) MUST FAIL on a stale render — the receipt carries the input hashes it was built from.
3. **Steps 2–3 re-pointed.** "Adjust `.codegraph/config.json` `exclude`" in step 2 now means: edit the consumer scope DATA file and re-render through the constitution renderer; step 3's re-index runs ONLY through the single writer entry of §11.4.80(5). Tracked first-party source that the engine skips by default is re-included by GENERATED negations, never by hand.

Classification: universal (per §11.4.17). Applies to every project under this Constitution that has CodeGraph configured and own-org submodules vendored.

Canonical authority: constitution submodule `Constitution.md` §11.4.79.

Non-compliance is a process violation; an AI-agent-worked project whose own-org submodules are excluded from CodeGraph (when they could be included) is in breach of this mandate. Severe cases (own-org submodules silently excluded WITHOUT an audit trail in `.codegraph/config.json` comments) are release blockers.

§ . . — CodeGraph regular-update + sync automation mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-21):

> "We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed on the host machine! After update executes with success use the following commands (and validate and verify them using codegraph help) to sync all: [list of codegraph commands]. Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule since they MUST BE available to all projects which do respect and follow our constitution rules and mandatory constraints! Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents (another area / context to regularly update and sync) and regularly export them like all other Status docs into the PDF and HTML!"

The mandate. Every consuming project under this Constitution MUST regularly check for CodeGraph npm-package updates, install the latest stable version, and re-sync its local index. The automation lives in the constitution submodule itself so every consuming project gets it via the inherited tree without reimplementation. Three deliverables:

1. **`<constitution>/scripts/codegraph_update.sh`** — checks the installed `@colbymchenry/codegraph` version against npm's latest, runs `npm update -g @colbymchenry/codegraph` (or `npm install -g @colbymchenry/codegraph@latest`) if a newer version exists, captures the old

+ new version + timestamp into the constitution's `docs/codegraph/Status.md` audit trail, then exits 0 only when `codegraph --version` produces the expected post-update output. Idempotent: re-runs on the same day are no-ops. Anti-bluff (§107): the "update succeeded" PASS MUST observe the new version, not just `npm update` exit code.

2. **`<constitution>/scripts/codegraph_sync.sh`** — after a successful update, runs the canonical sync sequence inside the consuming project (passed as an argument or auto-resolved via `find_constitution.sh` parent walk): `codegraph status` (baseline) → `codegraph sync .` (incremental update) → `codegraph status` (post-sync) → `scripts/codegraph_validate.sh` (the project's anti-bluff verifier, per §11.4.78 step 4). Each step's output is appended to the project's `docs/codegraph/Status.md` ledger. The sync MUST consult `codegraph help` BEFORE invoking any command to verify the subcommand still exists with the same shape (tools evolve). The canonical sync sequence covered by `codegraph help`:

```
codegraph                          # Run interactive installer
codegraph install                  # Run installer (explicit)
codegraph init [path]              # Initialize in a project (--index to
also index)
codegraph uninit [path]            # Remove CodeGraph from a project (--
force to skip prompt)
codegraph index [path]             # Full index (--force to re-index, --
quiet for less output)
codegraph sync [path]              # Incremental update
codegraph status [path]            # Show statistics
codegraph query <search>           # Search symbols (--kind, --limit, --
json)
codegraph files [path]             # Show file structure (--format, --
filter, --max-depth, --json)
codegraph context <task>           # Build context for AI (--format, --
max-nodes)
codegraph affected [files...]      # Find test files affected by changes
codegraph serve --mcp              # Start MCP server
```

3. **`<constitution>/docs/codegraph/Status.md` + `docs/codegraph/Status_Summary.md`** — append-only ledgers tracking every CodeGraph-related event across the consuming project: install events, npm-update events, sync runs, index regenerations, validate runs (with PASS/FAIL counts), HRDs

opened/closed against CodeGraph. Both ledgers MUST be exported to `.html` and `.pdf` siblings on every edit (per §11.4.65 multi-format export mandate). The `Status_Summary.md` is the operator-readable derived index (per §11.4.53 fixed-summary backfill convention).

Operational cadence. The automation MUST run at least weekly (per §11.4.45 status-digest cadence). Consuming projects MAY wire it more frequently (e.g. daily via cron or per-CI-run) but never less. Cadence is not contractually fixed because environments differ (development workstation vs CI vs production-locked host); the floor is "weekly" because slower than that risks accumulated drift.

Scripts are inherited by reference. Per §3 submodule inheritance, consuming projects do NOT copy `codegraph_update.sh` / `codegraph_sync.sh` into their own `scripts/` directory. They invoke the constitution submodule's scripts directly (e.g. `bash ${CONST_DIR}/scripts/codegraph_update.sh && bash ${CONST_DIR}/scripts/codegraph_sync.sh .`). This is the single-source-of-truth pattern — when the constitution updates the scripts, every consuming project picks up the change on its next submodule update.

Composition. Composes with §11.4.78 (CodeGraph parent mandate), §11.4.79 (own-org submodule inclusion), §11.4.10 (credential exclusions stay applied), §11.4.45 (status-digest cadence), §11.4.53 (Fixed_Summary backfill), §11.4.65 (multi-format export), §107 (each sync run's "ok" MUST carry observed-version evidence, not just exit code), §1.1 (paired mutation: temporarily downgrade installed codegraph version → script MUST detect drift and self-correct → restore).

Why this matters. A CodeGraph index built against an older version of the indexer may miss symbols that newer versions resolve correctly. Stale indexes silently lie to AI agents about what symbols exist and how they connect. Regular automated updates + sync close the loop: tooling stays current, the index reflects current source, AI agents see the truth. Manual "run codegraph index when I remember" is exactly the failure mode this mandate forbids.

EXTENSION — safe writes, gated upgrades, supervised long runs (research-derived, 2026-09-24). Measured this cycle (genericised): the update script installs the latest CLI with no compatibility probe, so every upgrade silently re-arms a known bulk-window stall; the sync script was never recorded running and exits 0 when its validator is absent; the vendor CLI deletes a lock by AGE, not by holder liveness, and a writable open recreates deliberately-dropped indexes without taking

the lock — so a second writer against a live index corrupts it; and a launcher that classified `sync` by the number of rows ALREADY stored rather than by the size of the PENDING backlog ran a bulk job of tens of millions of references on the stock runner, in the foreground, unsupervised. Deliverables 1–3 above remain in force; the following clauses are added:

1. **Upgrade gated on a compatibility probe.** An update MUST first probe the new version (the config filename + keys its loader reads, the bulk-window hazard probe, and applicability of every version-keyed runner patch). If the patched runner cannot be re-derived and re-proven for the new version, the upgrade REFUSES loudly and keeps the prior version (§11.4.201) — never a silent fall-back to the stock runner. A runner patch without its own RED→GREEN test, paired mutation and equivalence proof on a byte copy (never on the live database) MUST NOT be applied to a live index (§11.4.224 / §1.1).
2. **Exactly one writer entry.** Every `init` / `index` / `sync` goes through the constitution's single writer entry (§11.4.78(7)), which MUST (i) refuse while ANY live process whose real command line (§11.4.196(D)) targets the same root is running, WHATEVER the lock file's age; (ii) classify a run as BULK by the size of the PENDING work (unresolved backlog, files still to parse) — NEVER by the size of what is already stored; (iii) route bulk runs only through the proven patched runner, detached, under the watchdog of clause 6 and a disk tripwire. Readers of a live index open it read-only/immutable and never run full-table counts; experiments run on fixtures or copy-on-write copies.
3. **Supervised long runs.** Every long run is supervised by a PROGRESS-PROVEN watchdog (§11.4.232(C)); a process that is alive but not advancing is a STALL → loud stop + tracked item (§11.4.197), never "still running".
4. **Cadence is real and fail-closed.** The weekly floor above is realised by an actual timer/hook plus the constitution-pull hook (§11.4.164), each run leaving a ledger entry; a missing validator is a FAIL, never a warning or a silent exit 0.
5. **Live help is authoritative.** The embedded command list above is illustrative; the authoritative list is a live `codegraph help` diff against the previous run (for example, the current version also exposes a `telemetry` command the list omits). Telemetry MUST be off for the Local-Only posture — whether it is currently off is a per-host FACT to be captured, not assumed.

Classification: universal (per §11.4.17). Applies to every project under this Constitution that has CodeGraph configured per §11.4.78.

Canonical authority: constitution submodule `Constitution.md` §11.4.80.

Non-compliance is a process violation; severe cases (consuming project has not run `codegraph_update.sh` in >2 weeks AND has open AI-agent work) are release blockers — the agents' index reflects a stale tool version and may produce silently wrong reasoning.

§11.4.100 — RETIRED. Demoted to consumer project (a consuming project's video-color/visual-quality fidelity) per §11.4.17/§11.4.35 — project-specific (RK3588/MPV/Arvus), not universal. See the consuming project's Constitution/CLAUDE/AGENTS/QWEN.

---

§ . . — Mandatory Anti-Forgetting Enforcement: PreToolUse Guard Hook + Subagent Constitutional Preamble + Orchestrator Pre-Action Checklist (Operator mandate)

Short tag: `anti-forgetting-enforcement`.

Forensic anchor — operator mandate:

> UNCONFIRMED: pending operator's verbatim anti-forgetting mandate quote (tracked: PENDING item 3 in the current session's CONTINUATION.md handoff).

Background context (UNCONFIRMED quote omitted; factual description only): the motivating incident is documented in `docs/AGENT_GUARDRAILS.md` and in `scripts/hooks/guard-forbidden-commands.sh`'s in-source header. During an on-device-API build, emulator subagents ran raw host-direct `emulator` / `adb` commands instead of routing through the Containers submodule as required by the governing clauses (§6.X / §6.V / §6.AG in the consuming-project layer). The root cause was that the orchestrator **forgot to inject the rule into subagent prompts**. A rule the orchestrator forgets to paste is not enforcement — it is a social contract with the agent's memory, and that contract is broken by every cold-session start,

context-window exhaustion, and sub-agent dispatch. The mechanical fix is twofold: (1) a **PreToolUse guard hook** that blocks the forbidden command classes at the tool-call boundary unconditionally, and (2) a **canonical subagent preamble document** that the orchestrator pastes verbatim into every dispatch, plus a **pre-action checklist** for the orchestrator's own actions. Together these are the "anti-forgetting" enforcement layer: the hook is the floor (commands cannot be executed even if the agent forgot every rule), the preamble is the ceiling (the full live ruleset, not just what the hook can pattern-match). This two-layer model was introduced in a consuming project (commit family on `master`, 2026-06-01/02) and is explicitly designed to be universal — `scripts/hooks/guard-forbidden-commands.sh` carries `Classification: universal` in its header and is portable by design (no project-specific paths, no `jq` hard dependency).

Rule. Every HelixConstitution-consuming project MUST deploy the §11.4.109 anti-forgetting enforcement layer consisting of three components, all maintained together:

(A) PreToolUse guard hook — the mechanical floor.

A shell script MUST be wired as a `PreToolUse` hook in the AI agent runtime's project-scoped settings file (e.g. `.claude/settings.json` for Claude Code) to intercept every `Bash` tool call before execution and BLOCK the forbidden command classes at the tool-call boundary. The hook:

1. Reads the tool invocation as JSON from stdin (`.tool_name` + `.tool_input.command`).
2. Exits 0 for non-Bash tools (pass-through) and empty/missing commands.
3. For Bash tool calls, pattern-matches against the following mandatory blocked classes, each mapped to its governing clause:

Blocked class	Governing clause(s)	Examples
Raw host-direct Android emulator launch / APK install / instrumentation run	§11.4.76 (Containers mandate) + any project-layer emulator clause (e.g. §6.X / §6.V / §6.AG in a consuming project)	`emulator -avd …`, `$ANDROID_HOME/emulator/emulator …`, `adb install …`, `adb -s … install …`, `am instrument …`
Force-push / hook bypass / signature bypass	§11.4.41 (pre-force-push merge-first) + §11.4.71 (pre-push fetch) + project-layer §6.T.3 equivalent	`git push --force`, `git push -f`, `git push --force-with-lease`, `--no-verify`, `--no-gpg-sign`
Privilege escalation	§11.4 no-sudo (project-layer §6.U equivalent)	`sudo …`, `su`, `su -`, `su -l …`
Host power management	§12 host-session-safety (all consuming projects)	`systemctl suspend/hibernate/poweroff/reboot/halt/kill-user/kill-session`, `loginctl …`, `pm-suspend/hibernate`, `shutdown …`

4. Exits 2 (block) when a match is found; the stderr text printed is fed back to the agent as the refusal reason, citing the governing clause.
5. Supports a documented-exception escape hatch: a command containing the literal marker `# guardrails:allow <reason>` is WARNED (stderr) but NOT blocked — EXCEPT for host-power commands, which are categorically non-overridable.

The **canonical implementation** of this hook lives at `constitution/scripts/hooks/guard-forbidden-commands.sh` in this constitution submodule. Consuming projects MUST reference it at that path (inherited by reference per §11.4.80) — NEVER copy it locally (a copy diverges silently). The script is portable: no `jq` hard dependency (built-in awk fallback), no consumer-project-specific paths, no hardcoded credentials.

Anti-bluff requirement: a hermetic test suite (at minimum 20 cases) MUST verify every blocked class exits 2, every allowed command exits 0, the escape hatch fires for non-power classes, and the host-power class rejects even with the escape marker. Paired §1.1 mutation: remove the `emulator -avd` pattern from the hook → the emulator-gate case exits 0 → test FAILs → restore → test PASSes.

(B) Subagent Constitutional Preamble — the semantic ceiling.

A canonical document MUST exist at `docs/AGENT_GUARDRAILS.md` (or the project's equivalent governance preamble path, recorded in the project's `CLAUDE.md`) containing:

1. **The SUBAGENT CONSTITUTIONAL PREAMBLE block** — a verbatim, self-contained text block covering the top-10 mandatory constraints that cannot be pattern-matched by the hook alone (anti-bluff intent, resource caps, evidence honesty, continuation maintenance, hardcoding prohibition, distribute gate, remote policy, no-guessing vocabulary, etc.) plus the constraint classes that the hook already enforces (emulators, force-push, sudo, host-power) for completeness.
2. **Clear instruction that the orchestrator MUST paste this block verbatim into every subagent dispatch.**

The preamble is the semantic complement to the hook: the hook catches command-class violations; the preamble pre-informs the agent of all rules it must follow before it even issues a command. Together they eliminate the "agent forgot because the orchestrator didn't paste it" failure class.

Anti-bluff requirement: the preamble MUST be structured so it can be mechanically verified to be present in `docs/AGENT_GUARDRAILS.md` by `check-constitution.sh` (or the project-equivalent sweep); the document MUST contain the anchor literal `11.4.109` once this clause lands. Paired §1.1 mutation: remove the preamble's hook-reference sentence → constitution checker detects absence → FAIL.

(C) Orchestrator Pre-Action Checklist — guard against self-forgetting.

The same `docs/AGENT_GUARDRAILS.md` document MUST contain an **ORCHESTRATOR PRE-ACTION CHECKLIST** covering at minimum:

- Before any subagent dispatch: confirm the preamble (sub-rule B) is pasted verbatim.

- Before any emulator/device action: confirm the run routes through the Containers submodule CLI, not host-direct; confirm no live device is targeted; confirm the gate host is eligible (otherwise honestly BLOCKED).
- Before any distribute action: confirm Challenge Tests EXECUTED (not compiled) against the exact artifact; version code bumped; CHANGELOG entry present; debug-stage evidence present if two-stage distribute applies.
- Before any push / destructive git action: confirm no force flags without per-operation operator approval; remote is the approved set only; for history rewrite, hardlinked `.git` backup made first.
- Before any host-affecting command: confirm the command is not in the host-power blocked class.

Anti-bluff requirement: the checklist MUST be present in the preamble document and verifiable by `check-constitution.sh`. Each checklist item is traceable to a governing clause cited inline.

Mechanical enforcement.

Consuming-project `check-constitution.sh` (or `verify-all-constitution-rules.sh`) MUST verify ALL of:

1. `constitution/scripts/hooks/guard-forbidden-commands.sh` exists at the canonical path in the constitution submodule.
2. The consumer's AI-agent settings file (`.claude/settings.json` or equivalent) contains a `PreToolUse` hook entry referencing the canonical script path.
3. `docs/AGENT_GUARDRAILS.md` (or the equivalent preamble path) exists, contains the `SUBAGENT CONSTITUTIONAL PREAMBLE` heading, and contains the `ORCHESTRATOR PRE-ACTION CHECKLIST` heading.
4. A hermetic test for the hook exists (at minimum, a file under `tests/hooks/` or equivalent) and passes.
5. The anchor literal `11.4.109` is present in every per-scope governance file (`CLAUDE.md`, `AGENTS.md`, `QWEN.md`) of the consuming project and its owned submodules per §11.4.35 inheritance.

Pre-build gate `CM-ANTI-FORGETTING-ENFORCEMENT` enforces (1)–(4). Propagation gate `CM-COVENANT-114-109-PROPAGATION` enforces (5). Paired §1.1 meta-test mutations: (a) remove the `PreToolUse` hook entry from the agent settings file → gate (2) FAILs; (b) delete `docs/AGENT_GUARDRAILS.md` → gate (3) FAILs; (c)

remove the canonical hook script from the constitution submodule → gate (1) FAILs; (d) strip the `11.4.109` literal from a consumer `CLAUDE.md` → propagation gate FAILs.

Inheritance.

Applies recursively to every submodule, every feature, every new artifact, every project consuming this constitution. Submodule constitutions MAY add stricter rules (additional blocked command classes, additional checklist items, stricter test counts) but MUST NOT relax any clause. Per §11.4.35: this universal clause lives in the constitution submodule; consuming projects' `CLAUDE.md` / `AGENTS.md` / `QWEN.md` carry the inheritance pointer-block per §11.4.35 + the `11.4.109` literal per the propagation gate.

The principle this clause enshrines (in the fewest possible words): A constraint that depends on an agent remembering it is not a constraint — it is a hope. The PreToolUse hook converts the most dangerous command classes into hard-blocked failures regardless of agent memory. The preamble document converts the full ruleset into something the orchestrator can inject rather than recite from training. Together they are the minimum viable "anti-forgetting" enforcement posture for any AI-agent-assisted project under this constitution.

Composes with §11.4 + §11.4.1..§11.4.16 (anti-bluff covenant — the hook and preamble are the floor and ceiling of the anti-bluff enforcement layer), §11.4.6 (no-guessing — the escape hatch requires a `<reason>` exactly because undocumented exceptions are a guessing posture), §11.4.10 (credentials — the host-power class is categorically non-overridable, analogous to credential leaks), §11.4.75 (mechanical enforcement — §11.4.109 is the AI-agent-side specialisation of §11.4.75's git-hook mechanical enforcement; together they cover the full automated-and-agentic surface), §11.4.76 (Containers mandate — the emulator gate in the hook enforces §11.4.76 at the tool-call boundary), §11.4.78 / §11.4.79 / §11.4.80 (CodeGraph — the hook passes codegraph MCP calls through untouched; the preamble includes the anti-forgetting constraint about CodeGraph use), §11.4.81 (cross-platform parity — the hook script is portable bash/awk with no OS-specific primitives; per-OS blocked-class equivalents are addable via consuming-project extension), §11.4.84 (working-tree quiescence — the pre-action checklist instructs agents to grep for mutation markers before staging), §11.4.98 (full-automation mandate — the hook's test suite MUST itself be fully self-driving end-to-end), §11.4.102 (systematic-debugging — any hook violation that surfaces

should trigger the systematic-debugging arc, not a quick workaround), §12 (host-session-safety — the host-power class is non-overridable; §12 is the constitutional root of that prohibition).

Classification: universal (§11.4.17) — the PreToolUse guard hook + subagent preamble + orchestrator checklist pattern is vendor-neutral (defined against the JSON stdin contract that every Claude Code hook receives; the pattern ports to any agent runtime that exposes a pre-tool-call interception point) and reusable across ANY project that uses AI coding agents. The specific blocked command-class list at the UNIVERSAL layer covers the four classes that are unconditionally forbidden by this constitution (host-power, privilege escalation, force-push/bypass, raw emulator host-direct); consuming projects extend the list with project-specific additions per §11.4.35.

Non-compliance is a release blocker. No escape hatch — no `--skip-pretooluse-hook`, `--no-guardrails-doc`, `--anti-forgetting-optional`, `--single-layer-sufficient` flag exists. A project whose agent can forget a constitutional constraint and execute the forbidden command is constitutionally undefended, irrespective of how well the docs are written.

Canonical authority: this Constitution.md §11.4.109 in the HelixConstitution submodule; reference implementation `constitution/scripts/hooks/guard-forbidden-commands.sh` + reference preamble document `constitution/docs/AGENT_GUARDRAILS.md` (path within the submodule). Consuming projects inherit by reference per §11.4.80.

2026 06 09

§ . . — Universal action-prefix system (`ACTION_NAME ::`) (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-09):

> "When a user prompt STARTS with `ACTION_NAME ::` (e.g. `BACKGROUND :: IMPORTANT: do X...`) the agent MUST replace the `ACTION_NAME ::` prefix with that action's registered expansion text, then execute the rest. For `BACKGROUND ::` the expansion is: 'The following prompt that we will provide MUST BE executed in background in parallel with all main work streams using the subagents-driven development approach! All work done MUST PRODUCE rock solid evidence covered with hard physical proof(s) that all done is working as expected and as specified without any false results and without any bluff!'. The mechanism MUST be UNIVERSAL/extensible — more actions will be added later, each with its own expansion + rules. It MUST work with EVERY CLI agent, be fully decoupled + reusable by every project that includes the constitution submodule, and load + execute out of the box."

Every project under this Constitution MUST support a universal, extensible **action-prefix system**: when a user prompt's FIRST non-blank line starts with an uppercase action token followed by `" :: "` (grammar `^([A-Z][A-Z0-9_]*) ::` — anchored at line start, UPPERCASE-only token, exactly one space on each side of `::`), the agent MUST (1) look the token up in the shared **action registry** (tracked data file `constitution/actions/registry.yaml`, or `$HELIX_ACTION_REGISTRY`); (2) if the token is a registered action, REPLACE the `ACTION_NAME ::` prefix with that action's registered `expansion` text and apply its `rules`; (3) execute the REMAINDER of the prompt under the expanded instruction. The system is the C-preprocessor expand-then-rescan model applied to prompts (a line-anchored single-token macro): detect → substitute the registered expansion → execute the residual.

Two-layer architecture (both mandatory; LAYER 1 is the universal floor). (LAYER 1 — universal, always-on, out-of-the-box) the action-prefix recognition instruction MUST be mirrored into EVERY agent context carrier the constitution maintains (`CLAUDE.md`, `AGENTS.md`, `QWEN.md`, `GEMINI.md` per §11.4.35) so the agent itself recognises and applies the prefix on every CLI agent — Claude Code, Gemini CLI, Qwen Code, OpenAI Codex CLI, GitHub Copilot CLI, Cursor, Aider, Cline, Continue, Roo Code, and any future agent that reads one of those carriers. (LAYER 2 — mechanical, where the agent exposes a pre-submit seam) a prompt-preprocessing hook reading the SAME registry applies the expansion deterministically (Claude Code `UserPromptSubmit` / `UserPromptExpansion`

hook injecting the expansion via `additionalContext`; per-agent slash-command equivalents generated from the registry for Gemini/Qwen/Codex). LAYER 2 is the §11.4.109 anti-forgetting upgrade applied to prompt prefixes — the expansion holds even if model recall lapses; LAYER 1 guarantees the system works everywhere with zero extra setup. Honest §11.4.3 boundary (§11.4.6): transparent mechanical free-form `^PREFIX ::` interception is genuinely available ONLY on Claude Code today; on every other agent the free-form form is honoured by LAYER 1 (the agent self-applies) and the slash-command equivalent is the mechanical convenience — this split is documented, never papered over.

Grammar (mandatory, all hold). (1) Anchored at the start of the first non-blank line ONLY; mid-prose tokens never match. (2) UPPERCASE-only token `[A-Z][A-Z0-9_]*`; lowercase never matches. (3) Separator is exactly `" :: "` (space-colon-colon-space) — avoids C++ `Foo::Bar`, YAML `key: value`, URLs. (4) Stacked prefixes (`A :: B :: rest`) apply outer-to-inner, left-to-right (expand A, rescan, expand B, then the residual is the task). (5) A leading `\` escapes the prefix (literal, no expansion) so action names can be discussed. (6) An unknown token that matches the grammar shape but is NOT registered is NEVER silently expanded or silently dropped — the agent asks which registered action was meant (§11.4.66 + §11.4.105) or treats it literally; it NEVER invents an expansion (§11.4.6). (7) Any prompt not satisfying the grammar is an ordinary prompt; the system is a no-op. The carrier-block GRAMMAR_ADDENDUM (2026-06-09, extended with the arrow form 2026-07-02) additionally recognises FOUR EQUIVALENT forms of every action beyond the bare `::` — `PREFIX::ACTION_NAME :: <rest>` (namespaced `::`), `/ACTION_NAME <rest>` (bare slash), `/PREFIX::ACTION_NAME <rest>` (namespaced slash), and `ACTION_NAME ---> <rest>` (bare arrow, with the namespaced `PREFIX::ACTION_NAME ---> <rest>` variant) — where `PREFIX` is an action NAMESPACE (reserved default `DEFAULT`) and the namespace separator `::` carries NO surrounding spaces (distinct from the action-body `" :: "` and the arrow-body `" ---> "`, each one ASCII space on either side); the slash, arrow, and namespaced forms are honoured with the same expansion/execution, the bare slash form yielding to a colliding built-in/host slash command while the namespaced slash form is always unambiguous. A leading `\` escapes any of these forms (`\BACKGROUND :: x`, `\BACKGROUND ---> x`, `\/BACKGROUND x`).

Registry is the single source of truth + the extension contract. Adding a new action = adding ONE `actions[]` row (`name` , `version` , `summary` ≥6 words per §11.4.91, `expansion` , optional `rules` + `composes_with`) — no code change in either layer (the registry is data, not code). Both layers read the same file (§11.4.93-style single-source-of-truth). The first registry entry is `BACKGROUND` with the operator's verbatim expansion above; it composes §11.4.20/§11.4.70 (subagent-driven), §11.4.58/§11.4.103 (parallel streams), §11.4.89 (background execution), and §11.4.5/§11.4.69/§11.4.107 (captured physical evidence) + §11.4 (anti-bluff). The second registry entry is `REMINDER` — it re-surfaces previously-scheduled, CRITICAL, status-UNCERTAIN work: the agent FIRST verifies the ACTUAL current status from captured evidence (never assuming done or not-done — §11.4.6 no-guessing), THEN acts on the delta (report the captured proof if genuinely complete, resume from the exact point if partial, action it NOW if not started, or surface the block per §11.4.66 / §11.4.101), ALWAYS producing a status verdict; it composes §11.4.6 / §11.4.87 / §11.4.94 / §11.4.97 / §11.4.103 / §11.4.108 / §11.4.130 / §11.4.147. Three further registry entries are the **severity / handling-priority markers** `CRITICAL` (highest-priority / potentially release-blocking — maximum urgency + rigor ahead of lower-priority work, track it, auto-activate §11.4.102 systematic-debugging when an issue is involved), `IMPORTANT` (high-priority, above routine work but below CRITICAL), and `NOTE` (capture the remainder as durable CONTEXT — request-history §11.4.208 / §11.4.210 + persistent memory when a non-obvious project fact — applied when relevant, not an urgent action unless it contains an explicit action, §11.4.6 record only what the note says); each supports all six §11.4.140 forms incl. the single-colon `CRITICAL:` / `IMPORTANT:` / `NOTE:` , and registering `NOTE` promotes `NOTE:` from an ordinary-prose NO-OP to a registered action. They tag HOW the remainder is handled (priority + rigor), not a workable-item Type (§11.4.16). Decoupled + reusable (§11.4.28): the registry + hook + expander carry zero project-specific data and are consumed by reference (the §11.4.80 `codegraph_*` pattern); a consuming project ships its own registry or inherits the constitution default. Loads out-of-the-box via the §11.4.75 install seam.

Classification: universal (§11.4.17). Composes §11.4 / §11.4.5 / §11.4.6 / §11.4.17 / §11.4.20 / §11.4.28 / §11.4.35 / §11.4.58 / §11.4.66 / §11.4.69 / §11.4.70 / §11.4.75 / §11.4.80 / §11.4.89 / §11.4.91 / §11.4.93 / §11.4.103 / §11.4.105 / §11.4.107 / §11.4.109. Propagation gate `CM-COVENANT-114-140-PROPAGATION` (literal `11.4.140` across the consumer fleet)

- recommended gate `CM-ACTION-PREFIX-SYSTEM` (registry exists + parses + the grammar regex matches/no-matches the canonical fixtures + the LAYER-1 mirror block is present in CLAUDE.md/AGENTS.md/QWEN.md/GEMINI.md + the BACKGROUND expansion text matches the registry verbatim + the Claude `UserPromptSubmit` hook + expander script exist and are executable) + paired §1.1 meta-test mutation (corrupt the BACKGROUND expansion in the registry OR strip the `prefix_regex` OR remove the LAYER-1 mirror block from one carrier → the gate FAILs; strip the literal `11.4.140` → the propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.140. Non-compliance is a release blocker. No escape hatch — no `--skip-action-prefix`, `--ignore-prefix`, `--no-registry`, `--invent-expansion-OK`, `--single-layer-only` flag.

§ . . — Token-efficiency mandate (research-derived + operator mandate, - -)

Forensic anchor (operator mandate, 2026-06-09): the AI coding agent's token spend (input AND output) is dominated by an enormous always-loaded governance context — the consumer CLAUDE.md/AGENTS.md/QWEN.md plus the constitution submodule (measured ~170,000 tokens of static governance re-sent on EVERY turn: consumer `CLAUDE.md` ~90.5K + `constitution/CLAUDE.md` ~79.6K, a 112-row Applied-Fixes table plus verbatim §11.4.X anchor restatements) — compounded by 350–520K-token subagent transcripts, verbose status, full-file re-reads, and full-file loads where retrieval would do. The same pattern is externally documented as a scaling defect (Claude Code issue #24147 — "Cache read tokens consume 99.93% of usage quota - architectural scaling issue with CLAUDE.md re-

reads"). Operator mandate: cut token spend to **30–40% of current (a 60–70% reduction)** WITHOUT degrading quality, performance, or safety, and WITHOUT breaking any existing mechanism.

Every project worked on by AI coding agents MUST adopt a token-efficiency regime composed of the following measures, ranked by safety and impact, and MUST PROVE the reduction with a measured anti-bluff harness — never an asserted number (§11.4.6 / §11.4.123). The headline 60–70% is the warm-cache target; the rule requires the *measured* best-safe reduction with cited evidence, never the estimate.

The measures (composable, each preserving every existing rule):

1. **Prompt-cache the static governance prefix (PRIMARY — the single biggest lever).** The always-loaded governance MUST form a byte-stable prefix with a cache breakpoint at its end, with nothing volatile rendered ahead of it. Cache reads cost ~0.1× base input price (a 90% discount); writes 1.25× (5-min TTL) / 2× (1-hour). The discount applies to the dominant cost component. **Caching is transparent — the model sees byte-identical input cached or not — so it CANNOT remove a rule, weaken a gate, lower quality, or change a verdict; it changes only billing.** The only failure mode is a *silent invalidator* (timestamp/UUID/unsorted-JSON ahead of the breakpoint), which costs money but never corrupts, and is detectable via `cache_read_input_tokens`. The operative discipline: **batch governance edits and keep CLAUDE.md + the constitution byte-stable mid-session** — every governance edit busts the prompt cache (BASELINE.md finding), so all governance changes in a working window are landed together rather than dribbled across turns. Claude Code caches natively; SDK callers use `cache_control:{type:"ephemeral"}` ; Gemini/Qwen use their context cache via API-key auth.
2. **Subagent model-tiering + output-to-file (SECONDARY — biggest non-cache win).** Route MECHANICAL, NON-JUDGMENT subagent work (search, grep, status, doc-export, file-presence, read-only probes) to a cheaper/smaller model (Haiku-class); RESERVE the strong model for ALL reasoning, fix-design (§11.4.102), verdicts, code-review (§11.4.125), and demotion-evidence (§11.4.7). Subagents persist large output to a file and return a pointer instead of a 350–520K-token inline transcript. **The strong model owns every decision and every PASS/FAIL — the cheap model never emits a verdict — so**

§11.4.50 determinism and the anti-bluff covenant are untouched; quality cannot degrade. Bounded by §12.6 (60% memory) and the §11.4.58/§11.4.103 stream caps.

3. **Thin always-loaded INDEX + on-demand detail (progressive disclosure).** Restructure the consumer governance so the always-loaded file is a concise index (one line per fix / per anchor) and the full bodies are fetched on demand. **Three invariants keep every gate green and every rule reachable: (a) each index line carries the literal `11.4.N` anchor token so the `CM-COVENANT-114-N-PROPAGATION` literal-anchor scans still pass; (b) the canonical full text stays in the tracked, gate-scanned `constitution/Constitution.md` — no rule is deleted, only de-duplicated out of the consumer; (c) the full body is reachable in one hop.** This realises §11.4.35's consumer-thin / submodule-canonical split at the byte level — a de-duplication, never a deletion.
4. **Retrieval / CodeGraph-first over full-file loading.** Structural questions (where is X / what calls Y / what would break) → CodeGraph (§11.4.78/§11.4.79, already mandated + installed) or semantic search; region questions → grep-scoped read; never re-read a harness-tracked file. Read-only, already-trusted (§11.4.78) — no behaviour change.
5. **Output-token reduction.** Terse conductor status (no restatement of unchanged governance); `effort:"low"` on the mechanical-subagent allowlist only; structured output where a sink consumes it. Output is ~5× input price, so this is cost-leveraged. Presentation-only — captured-evidence artefacts (§11.4.5/§11.4.69) unchanged.
6. **Tool-call efficiency.** Batch independent tool calls in one block; reuse harness-tracked file state; reuse persisted tool outputs.
7. **Compaction / context-editing for long sessions (defensive).** Prune stale tool-results / old thinking so a 100+ turn cycle doesn't re-pay for an ever-growing transcript; never touches the governance prefix or on-disk evidence.

Mandatory measured proof (anti-bluff — §11.4.6/§11.4.50/§11.4.69/§11.4.123): the reduction MUST be proven by a token-accounting harness measuring tokens-per-development-cycle BEFORE vs AFTER on a frozen deterministic workload, split into `input_tokens` / `cache_read_input_tokens` / `cache_creation_input_tokens` / `output_tokens` from the authoritative

Anthropic `usage` object (NEVER `tiktoken`; NEVER the client-side `total_cost_usd` estimate — recompute cost from raw token fields with the published prices), reproduced N times to identical verdict (§11.4.50). Pass = AFTER ≤ 40% of BEFORE (target) OR the measured best-safe reduction with a cited cold-cache reason. The AFTER run MUST ALSO show ZERO regression on the pre-build full sweep, the meta-test mutation sweep (every gate still FAILs its paired mutation — proves M3 did not turn any rule into a bluff), every propagation gate (literal anchors intact), a strong-model reasoning-path probe (proves M2/M5 tiered nothing that decides), and a cache-warm proof (`cache_read_input_tokens > 0`). Cost reduction with quality/safety regression is a §11.4 FAIL, not a win.

No measure may break or degrade any existing mechanism — and the rule is structured so none can: caching is transparent (M1); tiering and low-effort are confined to mechanical non-judgment work (M2/M5); indexing preserves the full rules by reference with literal anchors so the propagation gates pass and the canonical text stays gate-scanned (M3); retrieval uses already-trusted read-only infrastructure (M4). The §11.4 anti-bluff covenant family, §11.4.50 deterministic consistency, §11.4.125 code-review, §11.4.40 full-suite retest, and §12.6 memory ceiling are all unconditionally preserved.

Composes with §11.4.5 / §11.4.6 / §11.4.20 / §11.4.40 / §11.4.50 / §11.4.58 / §11.4.69 / §11.4.70 / §11.4.78 / §11.4.79 / §11.4.80 / §11.4.103 / §11.4.106 / §11.4.123 / §11.4.125 / §11.4.128 / §12.6 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its model IDs, its mechanical-subagent allowlist, its governance files, its structural-index tool, and its frozen workload per §11.4.35. Propagation gate `CM-COVENANT-114-141-PROPAGATION` (literal `11.4.141` across the consumer fleet) + recommended gate `CM-TOKEN-EFFICIENCY` (the measured BEFORE/AFTER harness exists + produced a verdict ≥ target-or-cited-floor + the non-regression sweep is green + cache-warm proof present) + paired §1.1 meta-test mutation (inject a per-turn volatile token ahead of the governance cache breakpoint → `cache_read_input_tokens` collapses → measured reduction falls below the bar → gate FAILs; restore → PASSes — proves the harness measures real caching, not a hardcoded green). Gate-code = separate work item.

Canonical authority: constitution submodule `Constitution.md` §11.4.141. Research + design + measurement: `docs/research/token_efficiency/{RESEARCH,DESIGN,MEASUREMENT,TEST_PLAN}.md` . Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-token-`

`efficiency`, `--no-cache-governance`, `--assert-reduction-without-measuring`, `--tier-down-reasoning`, `--inline-all-governance`, `--tiktoken-estimate-OK` flag exists.

§ . . — All CI/CD automation (GitHub Actions / GitLab pipelines / equivalents) MUST be disabled (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-15):

> "Any GitHub actions or GitLab pipelines MUST BE disabled! Add this critical mandatory rule / mandatory constraint into the root constitution Submodule, commit and fetch all its changes to all upstreams and make sure we respect and follow this rule (we do apply it) ASAP!!!"

Every repository this Constitution governs — the main repo, this constitution submodule, and every owned + nested submodule we author and push — MUST ship with ALL server-side CI/CD automation DISABLED. No push to any owned upstream may trigger a GitHub Actions run, a GitLab pipeline, or any equivalent provider-side automation (Jenkins, CircleCI, Travis, Drone, Woodpecker, Bitbucket Pipelines, Azure Pipelines, or any `on: push` / `schedule` / `workflow_dispatch` workflow). This GENERALISES + makes ABSOLUTE the §11.4.75 Layer-5 posture ("Remote CI surfaces ... are DISABLED — the workflow file is preserved at `…disabled-local-only` (NOT `.yml`; GitHub Actions ignores it)"): what §11.4.75 did for the single constitution-compliance workflow, §11.4.156 mandates for ALL CI in ALL governed repositories. Enforcement migrates to the LOCAL §11.4.75 five-layer git-hook ritual + the §11.4.40 pre-tag sweep — never a remote runner.

The mandate (ALL hold): **(A) Zero active CI at the repository root.** No active `.github/workflows/*.yml|*.yaml`, no `.gitlab-ci.yml`, no `.gitlab/**` pipeline include, nor any equivalent provider config may exist at the ROOT of any governed repository/submodule — the only location a provider executes. **(B) Disabled means a push triggers ZERO runs.** Delete the config OR rename it to a non-active name (the §11.4.75 `.disabled` / `.disabled-local-only` convention — a provider ignores any name that is not its exact trigger filename); a workflow left with live `on:` triggers but `if: false` jobs still queues runs and is NOT

compliant. **(C) Scope = repositories we author + push.** Vendored / third-party source whose nested `.github/workflows` or `.gitlab-ci.yml` sit BELOW the repo root (e.g. AOSP `external/**`, `prebuilts/**`, `packages/modules/**`, vendored submodules) are INERT — a provider never executes a non-root config — so they are OUT of scope and MUST NOT be mass-edited (§11.4.29 vendor-name exemption + tree-integrity); the test is "does a push to one of OUR upstreams trigger a run?" — yes ⇒ disable, structurally-inert ⇒ document + leave (§11.4.6 — verify inertness as FACT, never assume). **(D) No new CI may be added.** Introducing any active workflow/pipeline into a governed repo is a release blocker. **(E) Pre-push verification.** Before any push, verify no root-level active CI exists in the pushed repo/submodule (`git ls-files | grep -E '^\.github/workflows/.*\.ya?ml$|^\.gitlab-ci\.yml$'` returns empty for authored repos); a §11.4.109-class PreToolUse guard + the recommended gate enforce the class mechanically. Honest boundary (§11.4.6): file-level disabling stops FILE-triggered runs; it does NOT disable provider-side server settings the agent cannot reach (org-default required workflows, branch-protection required checks, provider-side scheduled exports) — those MUST be turned off in provider settings by the operator, and the agent documents what it cannot reach rather than claiming a completeness it did not achieve.

Classification: universal (§11.4.17) — a platform-neutral CI-governance discipline reusable by ANY project; the consuming project supplies its concrete repository/submodule set + provider list per §11.4.35. Composes §11.4.75 (mechanical-enforcement Layer 5 — §11.4.156 promotes its CI-disabled posture to an absolute universal rule across all governed repos) / §11.4.29 (vendor/third-party name + tree exemption) / §11.4.6 (verify inertness, never guess) / §11.4.40 / §11.4.42 (the LOCAL pre-tag ritual replaces the remote runner) / §11.4.109 (PreToolUse guard blocks the class at the tool-call boundary) / §11.4.113 (same no-remote-surprise discipline applied to pushes) / §2.1 (multi-upstream push — the disabled state holds on every mirror). Propagation gate `CM-COVENANT-114-156-PROPAGATION` (literal `11.4.156` across the consumer fleet) + recommended gate `CM-NO-ACTIVE-CI` (no root-level active CI config in any authored repo/submodule, verified pre-push) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; add a root `.github/workflows/x.yml` to an authored repo → `CM-NO-ACTIVE-CI` FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.156.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--allow-ci`, `--enable-workflow`, `--keep-pipeline`, `--remote-ci-OK`, `--ci-exempt` flag.

§ . . — GEMINI.md maintained in lockstep with CLAUDE.md / AGENTS.md / QWEN.md (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-15):

> "Make sure with CLAUDE.md, AGENTS.md, QWEN.md we maintain GEMINI.md too! Add this mandatory fact / rule to root constitution Submodule we are inheriting / extending - CONSITUTION.md, CLAUDE.md, QWEN.md, AGENTS.md, GEMINI.md and other related relevant files!"

Forensic case study (FACT, 2026-06-15). When §11.4.156 + §11.4.157 were authored, `Constitution.md` / `CLAUDE.md` / `AGENTS.md` / `QWEN.md` carried the rule family through §11.4.155, but `GEMINI.md` had silently drifted to §11.4.141 — fourteen mandates (§11.4.142–§11.4.155) never propagated to it. A per-agent context carrier that lags is a §11.4 propagation-bluff: a Gemini-CLI agent reading the stale `GEMINI.md` operates under an out-of-date Constitution while the fleet believes the rule is universally in force.

`GEMINI.md` is a FIRST-CLASS governance context carrier, EQUAL to `CLAUDE.md` / `AGENTS.md` / `QWEN.md` — NEVER an optional or best-effort sibling. Every governance addition or edit MUST land in ALL FIVE carriers in lockstep: the canonical `Constitution.md` PLUS each per-agent mirror `CLAUDE.md` + `AGENTS.md` + `QWEN.md` + `GEMINI.md` (and any other relevant per-agent context file a runtime introduces), in the SAME change-window, each with its synchronized `.html` / `.pdf` / `.docx` exports per §11.4.65. The mandate (ALL hold): **(A) Five-carrier lockstep.** No governance change is complete until `GEMINI.md` carries it alongside the other three mirrors — `GEMINI.md` is added to the §11.4.26 constitution-update pipeline's stage-2 propagation set + stage-3 cross-reference validation explicitly. **(B) No silent drift.** A `GEMINI.md` whose highest rule number lags the other mirrors is a §11.4.157 violation (the same severity class as a §11.4.65 stale-export drift); the gap MUST be back-filled. **(C) Equal status, equal**

rules. `GEMINI.md` restates the SAME literal `11.4.N` anchors the §11.4.x propagation gates require (per §11.4.35 inheritance), so the propagation-gate fleet count INCLUDES `GEMINI.md`. **(D) Consumer projects too.** The lockstep binds the consuming project's repository-root context files (its own `CLAUDE.md` / `AGENTS.md` / `QWEN.md` / `GEMINI.md`) per §11.4.35 — a project that maintains the other three but not `GEMINI.md` is non-compliant. Honest boundary (§11.4.6): the pre-existing §11.4.142–§11.4.155 `GEMINI.md` back-fill is a tracked remediation work unit; this rule MAKES that drift a release-blocking debt rather than silently tolerating it — claiming `GEMINI.md` is "in sync" while the back-fill is incomplete is itself a §11.4.157 violation.

Classification: universal (§11.4.17) — a platform-neutral context-carrier-parity discipline reusable by ANY project that ships per-agent context files. Composes §11.4.26 (constitution-update workflow — `GEMINI.md` added to the propagation + validation set) / §11.4.35 (canonical-root inheritance — the per-agent mirrors restate the canonical anchors) / §11.4.17 (every new rule lands in all carriers) / §11.4.44 / §11.4.65 (revision header + synchronized exports for `GEMINI.md`) / §11.4.140 (`GEMINI.md` is one of the "every CLI agent" context carriers) / §11.4.156 (this rule's own first lockstep test) / §1.1. Propagation gate `CM-COVENANT-114-157-PROPAGATION` (literal `11.4.157` across the consumer fleet, GEMINI.md INCLUDED) + recommended gate `CM-GEMINI-MD-LOCKSTEP` (GEMINI.md's highest §11.4.N is not less than the other three mirrors' highest) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; let GEMINI.md's highest rule number fall behind the other mirrors → `CM-GEMINI-MD-LOCKSTEP` FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.157.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-gemini-md` (11.4.161), `--gemini-optional`, `--gemini-lag-OK`, `--four-carrier-suffices` (2026-06-21) flag.

§ . . — Rootless container runtime mandate (User mandate, - -)

Forensic anchor — direct user mandate (2026-06-21).

The mandate. Every project governed by this Constitution MUST use **Podman in rootless mode** (or an equivalent rootless container runtime) for ALL containerized workloads. Docker in rootful mode, `sudo`, or any escalation to root for container management is **STRICTLY FORBIDDEN** unless the target platform genuinely has no rootless container option AND that platform constraint is documented per §11.4.112 (structurally-impossible — with cited authoritative sources proving the absence of a rootless option on that platform). Rootless container execution eliminates the most common container-to-host privilege-escalation vector, aligns with the security-by-default posture of the §11.4.10 credentials-handling mandate, and is available and production-mature on Linux (Podman rootless since v2.0+, 2020) and macOS (Podman Machine with `--rootful=false`).

The `vasic-digital/containers` Submodule as the sole orchestration layer. Per §11.4.76 (Containers-submodule mandate), the `vasic-digital/containers` Submodule MUST be used as the **sole container orchestration layer** — every container operation MUST go through the Submodule's `pkg/boot` / `pkg/compose` / `pkg/health` primitives. No ad-hoc `docker` / `podman` commands outside the Submodule; if the Submodule's `pkg/boot` does not support a lifecycle or runtime the project needs, the missing capability MUST be extended in the Submodule itself via upstream PR (the §11.4.74 extend-don't-reimplement discipline), not worked around with a raw container command in the project.

The on-demand-infra invariant (refines §11.4.76). All container-related integration tests MUST boot infra on-demand via the Submodule (`pkg/boot` is the test entrypoint), not depend on a pre-running container daemon or operator-side manual `podman machine start`. Operators are NEVER required to start `podman machine` / `docker compose up` / `podman-compose up` manually — the boot is part of the test entry point. A test that claims to exercise containerized components but silently depends on an already-running daemon (skipping the boot) is a §11.4 PASS-bluff at the test-infrastructure layer.

Honest boundary (§11.4.6). Rootless container execution eliminates the container-to-root privilege-escalation vector and enforces containerization hygiene — it does NOT replace §11.4.10 credentials-handling (credentials stored in containers are still subject to the leak audit and `.gitignore` mandates) nor §12.3 container hygiene (memory limits, OOM policy, restart backoff still apply). A platform that genuinely

lacks rootless containers is documented per §11.4.112 and uses the Submodule's rootful mode with a documented extra risk acceptance — never uses ad-hoc rootful Docker.

Composition. §11.4.161 composes with §11.4.76 (Containers-submodule — §11.4.161 adds rootless-as-default + on-demand-infra invariant), §11.4.74 (extend-don't-reimplement — missing Submodule capabilities are extended upstream), §11.4.10 (credentials — rootless containers still require credential hygiene), §11.4.112 (structurally-impossible — the documentation path for platforms with no rootless option), §12.3 (container hygiene — memory limits/OOM/restart policies still apply), §11.4.6 (no-guessing — "the target platform probably has rootless" is not a finding).

Gates. Propagation gate `CM-COVENANT-114-161-PROPAGATION` (literal `11.4.161`) + recommended gate `CM-ROOTLESS-CONTAINER-RUNTIME` (every container operation is verified as Podman rootless OR has a §11.4.112 struct-impossible citation for that platform; no `sudo`/rootful Docker for container management; container integration tests boot on-demand via the `containers` Submodule) + paired §1.1 meta-test mutation (replace a rootless invocation with sudo or ad-hoc docker, OR bypass the Submodule's `pkg/boot` with a raw `podman` command → gate FAILs; gate-code = separate work item).

Classification: universal (§11.4.17) — the consuming project supplies its concrete rootless runtime (Podman rootless or equivalent), the `containers` Submodule integration, and any platform constraint documentation per §11.4.112 per §11.4.35. No escape hatch — no `--allow-rootful`, `--sudo-container-OK`, `--ad-hoc-docker-permitted`, `--no-containers-submodule`, `--pre-started-daemon-OK` flag exists.

11 4 164 2026 06 21

§ . . — Universal Constitution Auto-Propagation & Hook System (User mandate, - -)

Forensic anchor — direct user mandate (2026-06-21).

The mandate. Every fetch+pull of the constitution submodule MUST trigger an automatic post-update hook that installs/registers newly added or modified constitution components, so a pull of the constitution submodule is a COMPLETE governance-update transaction — not a partial one that leaves skills unregistered,

MCP servers unknown, hooks unwired, or scripts non-executable. The canonical hook script lives at `constitution/scripts/post_update_hook.sh` and is inherited by reference (§11.4.28) — NEVER copied locally.

(a) Change detection. The hook MUST detect which constitution submodule files changed in the latest pull: `Constitution.md`, `CLAUDE.md`, `AGENTS.md`, `QWEN.md`, `GEMINI.md`, and any file under `scripts/`, `hooks/`, `skills/`, `mcp/`, `plugins/`. It compares HEAD before vs after the pull via `git diff --name-only HEAD@{1} HEAD` (or the equivalent for the merge/pull operation).

(b) Skill registration. For any newly added or modified file under `skills/`, the hook MUST install/register it with the CLI agent's skill system (e.g. for Claude Code: add it to the project's skill configuration so the agent discovers and can invoke it per §11.4.102(B)). The registration mechanism is project-runtime-specific per §11.4.35 — the hook invokes a consumer-defined script at a canonical path (`scripts/register_skills.sh`) that the consumer project provides.

(c) MCP server registration. For any newly added or modified file under `mcp/`, the hook MUST register the MCP server with the agent's MCP configuration (e.g. for Claude Code: add an entry to `.mcp.json` or `settings.json`). The registration mechanism is project-runtime-specific per §11.4.35 — the hook invokes a consumer-defined script at a canonical path (`scripts/register_mcp.sh`).

(d) Hook installation. For any newly added or modified file under `hooks/`, the hook MUST install it into the project's `.git/hooks/` or runtime hooks directory, making it executable. The hook script sets executable permission and copies/symlinks it; existing hooks with the same name are overwritten only if the new file's mtime is newer (safe default per §11.4.122 — never overwrite a consumer's custom hook without a newer upstream version).

(e) Script validation. For any newly added or modified file under `scripts/`, the hook MUST (i) make it executable, (ii) validate its syntax — `sh -n` for POSIX-sh scripts, `bash -n` for bash scripts (per §11.4.67), (iii) validate any `helix-deps.yaml` or `.gitignore-meta/` entries per §11.4.31/§11.4.77.

(f) Summary report. The hook MUST emit a summary of what was installed/updated: number of skills registered, MCP servers registered, hooks installed, scripts made executable and validated. The summary goes to stdout AND to `.constitution/state/last_update.log` (gitignored per §11.4.30).

(g) Graceful failure. If any component installation fails (skill not registered, MCP server not wired, hook not installed, script not validated), the hook MUST log the failure per §11.4.6 with exact file:line and continue with remaining installations — one failure does not abort the entire post-update transaction. The failure is surfaced in the summary report and is a finding for §11.4.102 investigation.

(h) Anti-bluff. The hook is itself anti-bluff — tested by a paired §1.1 mutation that adds/removes a skill in the constitution submodule, pulls the submodule, and asserts the hook detects the change. A hook that reports "nothing to install" when a new skill was added is a §11.4 PASS-bluff at the auto-propagation layer.

Consumer obligation. Every project that fetches/pulls the constitution submodule MUST invoke the post-update hook after the fetch completes. The invocation mechanism is project-defined per §11.4.35 — either a `post-update` git hook wired via `scripts/install_git_hooks.sh`, or a post-pull script that calls `constitution/scripts/post_update_hook.sh` explicitly. A project that pulls the constitution and does NOT invoke the hook is non-compliant — it has an incomplete governance-update transaction, and skills/MCP/hooks/scripts it believes are present may not be.

Honest boundary (§11.4.6). The auto-propagation hook installs/registers constitution components — it does NOT guarantee the consumer's runtime accepts them (MCP server may have incompatible dependencies, skill may require a runtime version the consumer lacks) and does NOT replace §11.4.32 post-constitution-pull validation sweep (which verifies rules are enforced, not just that components are installed). Installation failures not investigated are the finding §11.4.102 captures.

Classification: universal (§11.4.17) — the consuming project supplies its concrete skill-registration script, MCP-registration script, hook-installation mechanism, and post-pull invocation method per §11.4.35. Composes §11.4.28/.32/.35/.67/.77/.102(B)/.122/.31/§1.1. Propagation gate `CM-COVENANT-114-164-PROPAGATION` (literal `11.4.164`) + recommended gate `CM-CONSTITUTION-AUTO-PROPAGATION` (post_update_hook.sh exists + executable, invoked after every pull, logs changes or failures, paired §1.1 mutation detects a new skill) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; pull the constitution and skip invoking the hook → `CM-CONSTITUTION-AUTO-PROPAGATION` FAILs; gate-code = separate work item).

§11.4.166 — REPEALED (operator decision, 2026-06-22). The Universal Semgrep static-analysis mandate is repealed: Semgrep is **NO LONGER a mandatory requirement** for projects governed by this Constitution. Static analysis / security scanning remains *encouraged* and may be re-adopted per project, but is NOT mandated, NOT a release blocker, and NOT cascaded. Removed with this repeal: `scripts/semgrep/`, `scripts/hooks/semgrep_precommit.sh`, the `submodules/semgrep` submodule, the MCP / pre-commit / PATH wiring, the consumer `.docs_chain/contexts/semgrep_status.yaml` context, and the `CM-COVENANT-114-166-PROPAGATION` / `CM-SEMGREP-WIRED` gates. Anchor number 11.4.166 is RETIRED (not reused). Repealed before full fleet cascade — consuming projects never received it as a binding mandate.

§11.4.173 — Containerized + distributed build mandate (User mandate, 2026-06-29). EVERY build of EVERY component (source compile, artifact/package/installer/container-image production, codegen, asset render — for ANY language/platform: Go, Android/Gradle, desktop, web, native, firmware) MUST run INSIDE a specialized build container provisioned via the `digital.vasic.containers` submodule (§11.4.76) — NEVER on the bare host. The build containers MUST be DISTRIBUTED to the designated remote build host(s) (e.g. `thinker.local`) via the SAME containers-submodule distribution mechanism the infra uses (§11.4.76 Distributor / remote compose over SSH, §11.4.161 rootless), so the build EXECUTES on the remote build host (offloading the developer/main host); once the build completes the produced artifacts MUST be brought BACK to the originating main host (scp/rsync/volume copy) for use/flashing/distribution. Building outside a container, or on the bare host, is FORBIDDEN — a release blocker (the "works on my machine" / unreproducible-build class §11.4.76 exists to prevent). The build-host target + build-container definitions are config-injected (§11.4.28, never hardcoded in the submodule); a missing build-container capability is added by EXTENDING the containers submodule upstream (§11.4.74), never by an ad-hoc host build. Honest boundary (§11.4.6): the containerized+distributed build guarantees reproducibility + host-isolation + capacity offload — it does NOT replace the §11.4.40 full-suite retest, §11.4.108 four-layer artifact→runtime verification, or §11.4.38 installable-asset evidence (those still run against the brought-back artifact). Classification: universal (§11.4.17). Composes §11.4.76 (containers submodule — sole orchestration layer), §11.4.161 (rootless runtime), §11.4.74 (extend-don't-reimplement), §11.4.28 (config injection / decoupling), §11.4.24 (build-

resource stats), §11.4.82 (iteration speedup — persistent caches in the container), §11.4.121 (no-commit-while-build-writes-artifacts), §11.4.38 (installable-asset evidence on the brought-back artifact), §11.4.108 (artifact→runtime verification), §12.6 (host memory ceiling — offloading the build preserves the main host). Propagation gate `CM-COVENANT-114-173-PROPAGATION` (literal `11.4.173`) + recommended gate `CM-CONTAINERIZED-DISTRIBUTED-BUILD` (every build runs via the containers submodule on the remote build host; a bare-host build is detected + FAILs) + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.173. Non-compliance is a release blocker. No escape hatch — no `--build-on-host`, `--skip-container-build`, `--local-build-ok`, `--no-distributed-build`, `--bare-host-build` flag.

§11.4.177 — Developer-tooling project-decoupling + invocation-directory operation (User mandate, 2026-07-04). No project-specific script, hook, alias, or binary may be wired — copied, symlinked, PATH-exported, or aliased — into a global or shared developer-tooling PATH; shared tooling MUST be project-agnostic and operate on the INVOCATION directory (the cwd, or an explicit path/config argument), NEVER on a hardcoded project path. A project-specific helper lives in and runs from its own repo; a genuinely reusable helper is promoted to the shared toolkit ONLY after it is stripped of every project-specific assumption (paths, device serials, package names, region endpoints) and takes its target from the invocation context. A project script reachable on the global/shared PATH is a decoupling violation of equal severity to importing project-specific code into a shared submodule (§11.4.28). The forensic case (FACT, 2026-07-04): a project cwd-hook symlinked into the global Claude-Toolkit PATH re-coupled every alias to one hardcoded checkout. Honest boundary (§11.4.6): decoupling guarantees the tool is project-agnostic — it does NOT by itself prove the tool is correct (that still needs the tool's own tests). Composes §11.4.28 (submodules-as-equal-codebase decoupling) / §11.4.29 (naming) / §11.4.35 (canonical-root layer split) / §11.4.11 (file-layout discipline) / §11.4.6 (no-guessing). Classification: universal (§11.4.17) — the consuming project supplies its concrete tool paths + invocation-config mechanism per §11.4.35. Propagation gate `CM-COVENANT-114-177-PROPAGATION` (literal `11.4.177`) + recommended gate `CM-TOOLING-PROJECT-DECOUPLED` (no tracked project script is symlinked/PATH-wired into a global toolkit dir; shared tools take their target from cwd/arg/config, never a hardcoded project path) + paired §1.1 meta-test mutation (wire a project script into the shared PATH, or hardcode a project path into a shared tool → gate FAILs; strip the literal → propagation gate

FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.177. Non-compliance is a release blocker. No escape hatch — no `--global-symlink-ok`, `--hardcode-project-path`, `--wire-into-shared-path` flag.

§11.4.184 — Mandatory SonarQube static-analysis CLI + local tooling installed and PATH-discoverable (User mandate, 2026-07-06). Forensic anchor — operator mandate (2026-07-06): "Make sure sonarqube is installed and configured locally same we did for Firebase, GitHub and GitLab CLIs through exported System PATH and .bashrc! Add all major points to the constitution so all projects do the same!" Every project/host that runs static analysis MUST have the SonarQube scanner CLI (`sonar-scanner`) installed AND durably discoverable on the system PATH via the shell rc (`.bashrc` / `.zshrc`) — exactly like the Firebase/ GitHub/GitLab CLIs — PLUS the shared `constitution/scripts/sonarqube/` tooling (`sonarqube_install_check.sh` / `sonarqube_lib.sh` / `sonarqube_run_scan.sh` / `sonarqube_container.sh` + `compose/`) consumed **by reference** (§11.4.28 decoupled + §11.4.177 project-agnostic + §11.4.80-style inherited, NEVER copied per project). The mandate (ALL hold): (1) **Installed + PATH-durable** — the scanner CLI resolves in every fresh shell (a durable rc export — e.g. a generic `Factory/software/*/bin` glob or an explicit line — not an ephemeral session-only PATH); (2) **Shared tooling present + GREEN** — `sonarqube_install_check.sh` exits 0 (scanner + rootless podman per §11.4.161 + podman-compose + curl + ES-ready host `vm.max_map_count`) before any scan-dependent work; (3) **Anti-bluff (§11.4.6)** — "installed/configured" is PROVEN by `command -v sonar-scanner` resolving AND the install-check exit 0, NEVER assumed; a claimed-installed-but-absent CLI is a §11.4 bluff at the tooling layer; (4) **Rootless (§11.4.161)** — the local SonarQube server runs via rootless podman through the container tooling, never rootful docker/sudo. **Honest boundary (§11.4.6):** this anchor names the SonarQube SCANNER CLI + the shared local-server tooling; per §11.4.166 the former universal Semgrep static-analysis mandate is REPEALED — SonarQube is a DISTINCT, operator-mandated tool here, not a Semgrep re-instatement, and this anchor mandates the TOOLING availability, not that every project must run a scan on every build. Classification: universal (§11.4.17) — the consuming project/host supplies its concrete install path + rc-export mechanism per §11.4.35. Composes §11.4.28 (decoupled shared tooling) / §11.4.36 (install-on-clone) / §11.4.74 (submodule-catalogue-first) / §11.4.75 (mechanical enforcement) / §11.4.109 (anti-forgetting) / §11.4.161 (rootless

container) / §11.4.166 (Semgrep-repeal boundary) / §11.4.177 (project-decoupled tooling) / §11.4.6 (no-guessing). Propagation gate `CM-COVENANT-114-184-PROPAGATION` (literal `11.4.184` across the consumer fleet) + recommended gate `CM-SONARQUBE-CLI-INSTALLED` (`command -v sonar-scanner` resolves + `sonarqube_install_check.sh` exit 0 + shared tooling present) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; move the scanner off PATH → the install gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.184. Non-compliance is a release blocker. No escape hatch — no `--skip-sonarqube-install` , `--sonar-cli-optional` , `--ad-hoc-sonar-path` , `--rootful-sonar-OK` flag.

§11.4.184(I) — HawkScan (DAST) + OWASP ZAP + gitleaks + Trivy: mandatory local security-tooling extension (operator mandate, 2026-09-18). Forensic anchor — operator mandate (2026-09-18): "Add HawkScan and all other missing utils as Git submodules fully available on local level through constitution Submodule and fully configured and optimized for high end use! ... Make sure everything is derived through constitution Submodule and accessible to all projects incorporating it." Following this anchor's own established pattern (this anchor's un-lettered SonarQube CLI clause above), every project/host that runs security scanning MUST have four additional tools installed and locally runnable through this constitution submodule's `scripts/<tool>/` tree, mirroring the `scripts/sonarqube/` five-file structure (`<tool>_lib.sh` shared functions, sourced never executed; `<tool>_install_check.sh` a three-tier presence/version verifier; `<tool>_container.sh` rootless-podman lifecycle where the tool needs a local server; `<tool>_run_scan.sh` drives the scan, polls for completion, and writes a Markdown report to `qa-results/<tool>/<ts>/` , never printing secrets; `compose/docker-compose.<tool>.yml` for containerized tools), consumed BY REFERENCE and never copied per project (§11.4.28 decoupled / §11.4.80-style inherited):

1. **gitleaks** (secrets scanner) — fully open-source, no account required. Installable via package manager or the upstream release binary; runs entirely locally with no network callout. Complements, and does NOT replace or conflict with, this constitution's own `credential_scan_lib.sh` pre-commit-hook-seam detector — no prior anchor declares that detector the exclusive secrets-scanning mechanism, so both run.

2. **Trivy** (dependency/container/IaC vulnerability scanner) — fully open-source, no account required, runs entirely locally against a filesystem, image, or repository target, with no network callout beyond its own vulnerability-database pull (cacheable, refreshable offline).
3. **OWASP ZAP** (DAST) — fully open-source, no account required, runs as a local rootless-podman container (`zaproxy/zap-stable` , §11.4.161) against a locally-reachable target, no external SaaS dependency.
4. **HawkScan** (DAST, StackHawk) — **the one tool in this set that is SaaS-backed, not self-contained.** The scanner image (`docker.io/stackhawk/hawkscan`) always authenticates against StackHawk's own backend; there is no fully local, no-account mode. Setup therefore has an unavoidable manual, interactive, operator-only component — a free-tier account signup at `app.stackhawk.com` (email + verification) and creation of one "Application" per scanned project to obtain an `applicationId` — that cannot be automated by any agent or script. `scripts/hawkscan/run.sh` MUST fail open, never closed: with no `HAWK_API_KEY` set, or the placeholder `applicationId` still present, it prints a clear, loud, non-blocking warning and exits 0. A missing third-party credential is an operator setup gap, never a build or gate failure.

Step-by-step key-acquisition guides for every tool that needs one (today: HawkScan only) live at `scripts/hawkscan/README.md` and propagate with the tool scripts themselves.

None of these four tools is declared in `helix-deps.yaml` — that manifest's schema (§11.4.31) is for git-clonable source dependencies, and none of the four is distributed as a clonable repository consumable that way (gitleaks/Trivy/ZAP ship prebuilt binaries or container images; HawkScan is SaaS-backed with no source repository to pin). Presence and configuration are governed by this clause plus the install-check scripts, exactly as SonarQube already is under this same anchor's un-lettered clause above. **Semgrep is deliberately NOT added by this clause** — §11.4.166's repeal of the former universal Semgrep mandate, in favor of §11.4.184's SonarQube mandate, stands unreversed.

Honest boundary (§11.4.6): this clause mandates the tooling's presence and local runnability; it does NOT itself run a scan, does NOT claim any project has been scanned, and does NOT claim any finding from these tools has been triaged — those are separate, per-project work items. HawkScan specifically can never be

claimed "configured" by an agent alone — the account and Application creation are irreducibly manual, and a project without them MUST show the non-blocking warning, never a fabricated pass.

Composes with: §11.4.184 (SonarQube, this anchor's own un-lettered clause) / §11.4.166 (Semgrep-repeal boundary, deliberately not reversed) / §11.4.31 (dependency-manifest schema, why these four are absent from it) / §11.4.28 / §11.4.80 (by-reference, never-copied inheritance) / §11.4.161 (rootless container runtime) / §11.4.10 (credentials — `HAWK_API_KEY` etc. are runtime-loaded env vars, never committed) / §11.4.6 (no-guessing / honest boundary). Propagation gate `CM-COVENANT-114-184-PROPAGATION` continues to cover this clause (same anchor, same literal `11.4.184` token — no new anchor number was minted). Recommended new gates: `CM-GITLEAKS-INSTALLED`, `CM-TRIVY-INSTALLED`, `CM-ZAP-CONTAINER-AVAILABLE`, `CM-HAWKSCAN-SCAFFOLDED` (each: `<tool>_install_check.sh` exit 0 — for HawkScan, "scaffolded" means the runner correctly detects and reports missing credentials, NOT that a scan ran) + paired §1.1 mutation (move/rename a tool off PATH → its install-check gate FAILs; neuter `hawkscan/run.sh`'s credential guard so it runs with no key → the mutation proof for that gate FAILs); gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.184(I). Non-compliance is a release blocker for the three open-source tools (gitleaks, Trivy, ZAP); HawkScan's compliance bar is "correctly scaffolded with a fail-open credential guard," never "scanned," until an operator completes the two manual account-creation steps. No escape hatch — no `--skip-gitleaks`, `--trivy-optional`, `--zap-off`, `--hawkscan-required` flag.

§11.4.196 (2026-07-14) — Native-alias-first priority + per-alias real-signal limit/subscription tracking + auto-rebind-on-recovery + resource-detection-by-real-identity-not-substring-match (User mandate, 2026-07-14)

Verbatim operator mandate: use ALL operational claude-NATIVE aliases FIRST — as long as ≥1 native alias is operational (NOT session-rate-limited, NOT weekly-limit-reached, NOT expired-subscription) — before ANY provider alias; track per alias the session rate-limit / weekly-limit / subscription expiry+renewal from REAL

signals (never faked, §11.4.6) and record when each becomes operational AGAIN; as soon as a higher-priority native recovers, rebind tracks to it; and fix the host-budget guard footgun where a `pgrep -f` substring-match false-REFUSED every worker spawn. The multi-track alias-binding + rate-limit/subscription-tracking layer of the §11.4.187 ruler orchestration MUST implement ALL of: **(A) NATIVE-ALIAS-FIRST PRIORITY** — every OPERATIONAL native (subscription-based `claudeN`) alias is selected before ANY provider alias, UNCONDITIONALLY, as long as ≥1 native is operational; the guarantee is STRUCTURAL — a native/provider CLASS partition scanned native-class-completely-before-provider-class (the §11.4.111 resolve-by-CLASS-not-by-file-position discipline), so a provider can NEVER win while a non-cooled native exists, REGARDLESS of roster file order; within a class the operator-mandated order (native equal-capability, then providers deepseek→xiaomi→opencode→kimi→…) is authoritative decision-DATA (§11.4.28) never re-invented (§11.4.6). **(B) PER-ALIAS REAL-SIGNAL LIMIT + SUBSCRIPTION TRACKING** — an "operational?" state per alias derived from REAL captured signals, NEVER faked/guessed (§11.4.6): the §11.4.187 captured 429 signature (`isApiErrorMessage` + `apiErrorStatus:429` / `api_error_status:429`) sub-classified into a reason-CLASS closed set `{session | weekly | subscription}` from real message markers (reset/weekly-limit/session-limit); each limited alias records its CLASS + an operational-again EPOCH, and the three classes have DIFFERENT windows — session (429 self-heals at the reset, short), weekly-limit-reached (~until the weekly reset, long), subscription-expired (INDEFINITE — operational-again is UNKNOWN until a REAL renewal signal, parked with a far-future sentinel; §11.4.6 NEVER guesses a renewal date, an operator/config supplies the real one). A limited alias is NEVER auto-selected until its epoch passes (session/weekly) or an explicit renewal/recovery signal clears it (subscription). **(C) AUTO-USE-ON-RECOVERY** — the moment a higher-priority native recovers (its window elapsed, or a renewal marked it operational), the mechanism rebinds tracks UPWARD to it (a `promote`-class operation keyed off a comparable priority-rank: native class before provider, then config order), PRESERVING each track's worktree AND device leases (the leases key on the STABLE track id — the alias changes, not the track; §11.4.119 / §11.4.187). Idempotent — a track already on the best operational alias is left untouched. **(D) RESOURCE-DETECTION BY REAL IDENTITY, NOT SUBSTRING-MATCH (the RB-02 guard footgun fix)** — a host-safety "heavy work in flight" guard MUST identify a real resource (a heavy build, a live worker) by its ACTUAL process command line read from the OS (`/proc/<pid>/cmdline` on

Linux), NEVER by a bare `pgrep -f REGEX` substring match that false-matches a process merely MENTIONING a token without BEING it — a pattern-CARRIER (a process whose args literally quote the whole alternation pattern; a `claude`/worker whose prompt embeds the token; a launcher/monitor/test), a same-tool ENGINE process, or the guard's own process/parent (the §12.12 self/sibling pgrep footgun — forensic FACT 2026-07-13: the guard false-REFUSED EVERY §11.4.187 spawn on a host with ZERO real builds because a live `claude` worker's prompt quoted the pgrep pattern). The guard re-reads each matched PID's real cmdline and excludes those carrier/engine/worker/self classes; an UNREADABLE/vanished cmdline is conservatively counted as the real resource (REFUSE — the safe, reversible default, §11.4.101 / §12.8), so a genuine build is still caught. **(E) ANTI-BLUFF (§11.4/§11.4.108)** — every piece carries four-layer coverage (pre-build gate + on-host test + paired §1.1 mutation + captured evidence per §11.4.69), each with a §11.4.115 RED-polarity reproduction of the defect on the pre-fix artifact + §11.4.50 determinism; **honest boundary (§11.4.6)** — this anchor mandates the MECHANISM + the REAL-signal derivation of "operational"; genuine live per-subscription quota-isolation proof still requires live tokens exercising real rate-limit boundaries (an operator-scheduled acceptance window), never conflated with a mock/hermetic contract GREEN. Classification: universal (§11.4.17). Composes §11.4.187 (the ruler orchestration this is the alias-binding + rate-limit layer of) / §11.4.182 (track+branch+alias identity label) / §11.4.176 (multi-track exactly-once claim + device-lock) / §11.4.111 (resolve-by-stable-name/CLASS not by index/file-position) / §11.4.101 (autonomous-decision-over-blocking — the bounded-park-when-all-cooled + the conservative-refuse safe default) / §11.4.103 (continuous parallel-stream routine) / §11.4.6 (no-guessing — REAL signals, never faked) / §11.4.108 (runtime-signature) / §11.4.115 (RED-polarity) / §11.4.119 (single-resource-owner — leases key on the stable track id) / §12.12 (RLIMIT_NPROC / process-footgun awareness — the pgrep self/sibling match is the same footgun class) / §12.6 / §12.8 (host-safety budget + concurrency — the guard the RB-02 fix protects). Propagation gate `CM-COVENANT-114-196-PROPAGATION` (literal `11.4.196`) + recommended gate `CM-ALIAS-PRIORITY-LIMIT-TRACKING` (native-first CLASS partition present; per-alias reason-class + operational-again tracking; auto-rebind-on-recovery; the build-guard resolves by real cmdline not bare substring-match) + paired §1.1 mutation (swap the native/provider priority bases so a provider outranks a native → the recommended gate FAILs; flip the guard's strict-

cmdline-filter to the pre-fix bare `pgrep` → its RB-02 test FAILs; strip the literal `11.4.196` from a carrier → the propagation gate FAILs; gate-code for the recommended gate = separate work item).

(F) CONFIGURED ≠ IN USE — the mechanism MUST BE WIRED TO THE ACTUAL WORKER SPAWNS, and its binding registry MUST REFLECT THE LIVE WORKER SET (extension, 2026-07-15). Forensic FACT (2026-07-15): the alias-binding orchestrator was fully installed and its configuration valid, yet its binding registry was EMPTY (`(none bound)`) while THREE live workers ran OUTSIDE it on provider-backed aliases — so the recovery `promote` operation rebound ZERO tracks. The mechanism existed and managed NOTHING. Therefore: (1) EVERY worker / agent spawn path MUST bind THROUGH the orchestrator (the §11.4.187 launcher) — a worker started outside it is UNMANAGED (never rebound on recovery, never rate-limit-tracked, never counted against the host budget) and is a §11.4.196 violation; (2) the mechanism's READINESS is defined by its RUNTIME SIGNATURE (§11.4.108), NOT by its configuration — **the set of rows in the binding registry MUST EQUAL the set of live workers** (every live worker appears with its bound alias; every registry row corresponds to a live-or-honestly-terminal worker); an empty / partial registry alongside live workers is a FALSE-READY state; (3) claiming the mechanism "ready" / "in use" on the strength of a valid config + an installed script, WITHOUT that registry↔live-worker equality captured as evidence, is a §11.4 / §11.4.6 bluff at the orchestration layer — "configured" is NOT "in use"; (4) the recommended gate `CM-ALIAS-PRIORITY-LIMIT-TRACKING` therefore ALSO asserts the registry↔live-worker equality, with a paired §1.1 mutation (leave a live worker unregistered → the gate FAILs). Composes §11.4.187 (the spawn / resume substrate this binds), §11.4.198 (the always-on default this makes REAL rather than nominal), §11.4.108 (runtime-signature-as-definition-of-done), §11.4.147 (a respawned agent re-binds through the orchestrator too).

Canonical authority: constitution submodule `Constitution.md` §11.4.196.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--provider-before-operational-native` , `--fake-limit-state` , `--guess-renewal-date` , `--no-rebind-on-recovery` , `--substring-match-build-guard` , `--skip-real-signal-classification` flag.

§ . . — Research / kicked-off-work completion mandate: every research effort MUST be driven to full, wired, verified completion or explicitly closed — never left un-wired in the backlog (User mandate, - -)

Forensic anchor — verbatim operator mandate (2026-07-15):

> "all research work we start MUST BE fully finished, completely confirmed, validated and verified! It MUST NOT happen for it to stay at the backlog uncompleted and un-wired! Loss of requirements, failure to incorporate them is FORBIDDEN!!! Everything MUST BE fully completed! No exceptions!"

Every research effort, kicked-off improvement, design doc, spike, prototype, proof-of-concept, or partially-landed feature that a project STARTS MUST be driven to FULL completion — OR explicitly, evidence-backed, CLOSED. There is no third "sitting un-wired in the backlog" state. "We researched it / designed it / started it" is NEVER a terminal state; a `docs/research/*` document, a design doc, a spike branch, or a half-wired feature that is neither COMPLETED-and-wired nor explicitly-closed is a §11.4.197 violation of the same severity class as a §11.4 PASS-bluff at the requirements-integrity layer — the requirement was accepted, work began, and then it silently evaporated. Loss of requirements and failure to incorporate started work are FORBIDDEN.

COMPLETED means (ALL must hold): (1) **implemented** — the code / artefact exists; (2) **WIRED** — integrated into the system AND used by default, not merely present in the tree behind a dead flag / an un-called function / an unreferenced module / a never-selected path (mere-presence-without-wiring is the §11.4.124 dead-code gap AND the §11.4.108 layer-2/3 SOURCE→ARTIFACT→RUNTIME gap — a change present in source that never reaches the running artifact, or reaches it but is never exercised, is NOT completed); (3) **confirmed + validated + verified** with captured physical evidence per §11.4.5 / §11.4.69 / §11.4.107 on a clean deployment per §11.4.108 runtime-signature — never a metadata-only / config-only / absence-of-error / grep-without-runtime claim (§11.4 / §11.4.1); (4) tracked to a terminal Status per §11.4.15 / §11.4.33.

Explicitly CLOSED means: the effort is deliberately terminated with a documented, evidence-backed reason drawn from the existing closed vocabulary — §11.4.90 `Obsolete` (superseded-by-design-change / superseded-by-later-mandate / duplicate-of / not-reproducible / feature-removed), §11.4.112 `structurally-impossible` (won't-fix, with cited impossibility proof), or §11.4.122 operator-confirmed drop — NEVER a silent abandonment. "No time" / "lower priority" is `Operator-blocked` per §11.4.21 (with the self-resolution-exhaustion audit) or a still-open tracked item, NOT a close.

Mechanical binding to the tracker (§11.4.93): every research effort / design doc / kicked-off improvement MUST map to at least one tracked workable item (ATM-NNN per §11.4.54) whose completion is enforced by the loop. A `docs/research/*` document (or equivalent design / spike artefact) with NO tracked implementing item, OR a tracked item that has been stalled un-wired past the project's cadence, is a §11.4.197 violation. The autonomous loop (§11.4.87 / §11.4.94 / §11.4.97 / §11.4.103 / §11.4.126 / §11.4.192) MUST treat every such un-wired research effort as an actionable queue item — driving it to COMPLETED or explicitly CLOSED is exactly the "keep working until the queue is genuinely empty" contract, never optional backlog.

Honest boundary (§11.4.6): "fully completed" is bounded by the completion + closure definitions above (wired + verified, OR evidence-backed closed) — it does NOT mean every speculative idea must ship; it means every STARTED effort reaches a documented terminal state, and the terminal state is proven, never assumed.

Classification: universal (§11.4.17) — a platform-neutral requirements-integrity discipline reusable by ANY project; the consuming project supplies its research-doc tree path + tracker binding per §11.4.35. Composes §11.4.8 (deep-web-research — the research this mandate insists on FINISHING, not merely starting) / §11.4.87 / §11.4.94 / §11.4.97 / §11.4.103 / §11.4.126 / §11.4.192 (the autonomous-loop queue-drain that drives un-wired efforts to completion) / §11.4.118 (discovery-pressure — enumerated coverage of what was started) / §11.4.123 (rock-solid-proof — completion PROVEN, never claimed) / §11.4.124 (dead/unwired-code investigate-before-remove — the wiring half) / §11.4.150 (deep multi-angle web research per change) / §11.4.90 / §11.4.112 / §11.4.122 / §11.4.21 (the explicit-close vocabulary) / §11.4.93 / §11.4.54 (the tracker binding) / §11.4.108 (wired-and-verified on a clean target) / §11.4.15 / §11.4.33 (terminal Status). Propagation gate

`CM-COVENANT-114-197-PROPAGATION` (literal `11.4.197`) + recommended gate `CM-RESEARCH-COMPLETION-TRACKED` (every `docs/research/*` effort / design doc maps to an open-or-closed tracked item; no research doc lacks an implementing item; no tracked item is stalled un-wired past the cadence) + paired §1.1 mutation (add a `docs/research/*` effort with no tracked implementing item, OR mark a research effort "done" while its feature is present-but-un-wired → the recommended gate FAILs; strip the literal → the propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.197.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--research-may-stay-unwired`, `--skip-completion-tracking`, `--abandon-without-close`, `--present-but-unwired-is-done`, `--backlog-research-OK`, `--lose-requirement-OK` flag.

§11.4.198 — Default working mechanisms: multi-alias native-first orchestration AND heavy token-optimization are ALWAYS-ON defaults for single- and multi-track work (User mandate, 2026-07-15)

Forensic anchor — operator mandate (2026-07-15): multiple claude-toolkit aliases (native-first) AND heavy token-optimization MUST be the DEFAULT mechanisms, always used with single- and multi-track development and with single or multiple working agents.

Two proven mechanisms are promoted to STANDING, always-on defaults — engaged automatically for EVERY working configuration (single-agent OR multiple-agent; single-track OR multi-track), never a per-request opt-in:

(A) Multi-alias native-first orchestration is the DEFAULT. The claude-toolkit multi-alias mechanism — every OPERATIONAL claude-NATIVE alias selected before ANY provider per §11.4.196 (native-alias-first priority + per-alias real-signal limit/subscription tracking + auto-rebind-on-recovery), driven by the §11.4.187 automatic multi-track ruler orchestration — MUST be the default execution substrate for ALL work, NOT only for the multi-track case. A SINGLE-track or SINGLE-agent session STILL binds its worker through the same native-first alias-priority mechanism (so it uses the best operational native alias + rebinds on

recovery), and a MULTI-track / multiple-agent configuration uses it across every track (§11.4.176 exactly-once claim + §11.4.182 track+branch+alias identity label). Running a session on an arbitrary / hardcoded / provider-first alias when an operational native exists — or bypassing the alias-priority mechanism because "it is just one agent" — is a §11.4.198 violation.

(B) Heavy token-optimization is the DEFAULT. The §11.4.141 token-efficiency discipline (compact-in-CLAUDE.md/AGENTS.md + full-one-hop-away anchor layout, targeted reads over whole-file dumps, subagent context isolation per §11.4.20 / §11.4.70, evidence-hash references not evidence dumps) MUST be applied ALWAYS — on every read, every dispatch, every doc edit — for single- AND multi-track work, with single OR multiple agents. Token-heavy working patterns (re-reading whole large files when a targeted slice suffices, dumping full evidence into context, monolithic non-subagent execution of parallelisable work) when a token-efficient path exists are §11.4.198 violations.

Honest boundary (§11.4.6): "default mechanism" means engaged automatically without an operator request; it does NOT weaken any host-safety bound — the multi-alias / multi-agent fan-out stays under the §12.6 60% memory ceiling + the §12.12 thread-headroom check + the §11.4.58 parallel-agent cap (spawning contending / redundant agents is NOT "using the default", per §11.4.183 maximum-USEFUL-throughput). §11.4.198 BINDS the two mechanisms as always-on; it does NOT redefine them — their mechanics live in §11.4.196 / §11.4.187 / §11.4.141.

Classification: universal (§11.4.17). Composes §11.4.141 (token-efficiency — mechanism B) / §11.4.187 (automatic multi-track ruler orchestration — the substrate) / §11.4.196 (native-alias-first priority + limit tracking — mechanism A) / §11.4.182 (track+branch+alias identity) / §11.4.176 (multi-track exactly-once claim) / §11.4.183 (maximal multi-agent utilization — the bounded-throughput sibling) / §11.4.103 (continuous parallel-stream routine) / §11.4.58 (parallel-development PWU) / §11.4.20 / §11.4.70 (subagent-driven default) / §11.4.126 (default autonomous-loop) / §12.6 / §12.12 (host-safety bounds). Propagation gate `CM-COVENANT-114-198-PROPAGATION` (literal `11.4.198`) + recommended gate `CM-DEFAULT-MECHANISMS-ALWAYS-ON` (the native-first alias-priority mechanism + the §11.4.141 token-optimization discipline are wired as the default for single- AND multi-track / single- AND multi-agent configurations) + paired §1.1 mutation (make

provider-first / bypass-alias-priority the single-agent default, OR strip the token-optimization default → the recommended gate FAILs; strip the literal → the propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.198.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--single-agent-skips-alias-priority`, `--provider-first-default`, `--token-optimization-optional`, `--heavy-tokens-OK`, `--not-default-mechanism` flag.

§ . . — Governance-corpus self-custody: the rule corpus is bound by the same custody it imposes — every named gate is implemented-or-registered-deferral under a monotone ratchet, and propagation gates count anchor BLOCKS (exactly-once + lockstep-identical), never bare literals (research-derived, - -)

Forensic anchor (measured on THIS corpus, 2026-07-22 — commands and control needles preserved in the root-cause analysis). The canonical rule file held 10,735 lines and 220 distinct §11.4.N anchors; **413 named CM-* gates, of which 241 (58%) had no implementation in either canonical gate site, plus 85 literal "gate-code = separate work item" deferrals**; the status-custody seam DESIGNED on 2026-07-17 was still prose on 2026-07-22 — the corpus's highest-value remediation fell into its own gap within five days; the two anchors that state the fix-confirmation discipline verbatim predate the failures they failed to prevent by five-plus weeks. The positive complement: **every fully-mechanised rule in the record either prevented or exposed its failure mode** (the suite PENDING-semantics + the release verdict-coverage seam landed within a day of diagnosis and now block) — the gap is not rule quality, it is the prose-to-seam conversion rate. The SHAPE defect: the propagation-gate family — the ONLY gate family that runs everywhere — asserts literal-presence ≥ 1, mechanically rewarding restatement over implementation ("the corpus measures its own health in words present, and so it grows words"); measured consequences: two anchor blocks duplicated verbatim-divergently in ALL FOUR mirrors while every presence-gate stayed green; one anchor NUMBER heading two DIFFERENT mandates (the

§11.4.140/§11.4.141 collision — independently re-detected by the block-integrity instrument's live run; resolution is an operator-owned re-mint, tracked); two canonical entry-point mandates contradicting each other (the mandated resumption file unreachable from the mandated README). External corroboration: a mandated-checklist adoption WITHOUT workflow embedding produced no measurable improvement across 101 hospitals (the Ontario null, cited in the external-research corpus) — prescription without a seam does not move outcomes at any scale.

The mandate (ALL hold):

(A) NAMED-GATE LEDGER + IMPLEMENTATION RATCHET. Naming a gate is a commitment with a same-commit cost: every `CM-*`-class gate token named anywhere in the governance corpus MUST, at every build (of the governance repo and of every wired consumer), be either IMPLEMENTED — the token present in an EXECUTABLE gate site; prose carriers NEVER count (§11.4.201(7)(a)) — or covered by a REGISTERED DEFERRAL row pointing at a tracked work item (§11.4.197). Silent gate debt is refused at the build seam. The unimplemented count is a MONOTONE-DECREASING ratchet, validated per §11.4.201(8) against the definition of done (at the correct end-state — every named gate implemented — the count is 0, so the metric reaches target and is valid as a necessary floor). A gate NAME that vanishes from the corpus without an explicit removal citation FAILs — the metric-gaming channel (deleting names to lower the count) is closed structurally; repeals stay legal and visible (the §11.4.166 pattern). **An anchor's "done" state is its SEAM landing, not its TEXT landing** — a rule authored without its seam is, by §11.4.205's own criterion, in the worse-than-nothing state, and this ledger makes that state visible, counted, and shrinking instead of silent. Brownfield adoption (ratchet down from the measured baseline vs hard-fail day one) is an operator §11.4.66 decision mirroring §11.4.224(E)'s adoption fence — never an invented ratchet.

(B) ANCHOR-BLOCK INTEGRITY — the propagation-gate SHAPE fix. Every propagation-class gate MUST count anchor BLOCK-STARTS (structural, line-anchored heading patterns), never bare literals: (i) exactly ONE block per anchor per governance file — 0 is the old absence check, > 1 is the DUPLICATION check, FAIL naming the anchor and file; (ii) content-hash EQUALITY across the declared lockstep mirror set (§11.4.157) — divergent copies FAIL naming both files (compact-vs-full variance ACROSS layers — canonical file vs mirror — remains

legitimate; equality binds only within the declared lockstep set, and canonical-vs-mirror semantic drift stays the §11.4.186 cross-doc family's job); (iii) mid-body citations of an anchor literal are CARRIERS and never count (§11.4.201(7)(a)); (iv) ZERO blocks extracted from a governance file is BLIND-OR-EMPTY, never clean (§11.4.201(6)); (v) one anchor NUMBER heading two different mandates is a COLLISION and FAILs (§11.4.54's never-reuse discipline applied to anchor ids); (vi) the block extractor MUST capture full dotted ids so a sub-anchor (`…10.A`) never prefix-matches its parent (`…10`) — a measured §11.4.201(1) false positive in the founding instrument itself, fixed RED-first.

(C) Consolidation pressure + honest boundary. Nothing mechanical decides SEMANTIC contradiction between two different anchors — only the decidable projections are mechanized here (block counts, lockstep hashes, anchor numbers; fenced verdict scopes stay §11.4.112(5), seam declarations stay §11.4.120); the remainder is §11.4.142/§11.4.194 review territory — stated, never claimed. This anchor adds the measured enforcement-ratio INSTRUMENT, not a growth ban: the counterweights to corpus growth remain §11.4.17 classification, §11.4.141 token-efficiency, and the visible-repeal precedent (§11.4.166).

Regular-cycle + review + pull binding (operator mandate 2026-07-23). The ledger regenerates and diffs on every governance edit and every build — regular development cycles, never an on-demand audit; a change that NAMES a gate is review-checked for its implementation-or-deferral per §11.4.194(6)(c); and on every constitution fetch/pull the §11.4.32 post-pull sweep + the §11.4.164 auto-propagation hook run the ledger + block-integrity checks against the post-pull tree, with violations tracked (§11.4.15) and cleared — no parallel mechanism is invented.

Reference mechanisms (proven, NOT wired — §11.4.6/§11.4.205). SOL-05 (gate ledger + ratchet) and SOL-06 (anchor-block integrity) under `docs/research/quality/solutions/` are runnable RED-first POCs with golden fixtures: SOL-05's live read-only run reproduced the 413-named-gate census exactly (234 unimplemented by its slightly-laxer structure rule — the 7-gate methodological delta stated in-doc); SOL-06's live run confirmed the four mirrors CLEAN post-F7 and independently re-detected the §11.4.140/§11.4.141 collision. Per §11.4.205 they carry NO force until wired by separate tracked work items (§11.4.197).

Classification: universal (§11.4.17) — no project literal; every consuming project (and this governance repo itself) supplies its gate-site list, deferral registry, and lockstep mirror set as DATA per §11.4.35. Composes §11.4.6 / §11.4.15 / §11.4.17 /

§11.4.32 / §11.4.54 / §11.4.66 / §11.4.75 / §11.4.141 / §11.4.157 / §11.4.164 / §11.4.166 / §11.4.186 / §11.4.194(6) / §11.4.197 / §11.4.201 / §11.4.205 / §11.4.224(E). Propagation gate `CM-COVENANT-114-227-PROPAGATION` (literal `11.4.227` — itself REQUIRED to be block-start-shaped per clause (B)) + recommended gates `CM-GATE-LEDGER-RATCHET` (every named gate implemented-or-registered-deferral; the unimplemented count monotone-decreasing; vanished names cite removals) and `CM-ANCHOR-BLOCK-INTEGRITY` (exactly-once per anchor per file + lockstep content-hash equality + carrier exclusion + BLIND-on-zero + collision detection) + paired §1.1 mutations (name a gate with no implementation and no deferral row → the ledger gate FAILs; delete a gate name without a removal citation → FAILs; duplicate an anchor block in one mirror, let two lockstep copies diverge, or re-mint an existing anchor number → the integrity gate FAILs; strip the literal → the propagation gate FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6).

Canonical authority: constitution submodule `Constitution.md` §11.4.227.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--name-gate-without-seam`, `--silent-gate-debt-OK`, `--delete-name-to-lower-count`, `--presence-gte-1-suffices`, `--duplicate-anchor-OK`, `--mirror-drift-OK`, `--reuse-anchor-number` flag exists.

§ . . — Cross-agent extension lifecycle mandate: per-platform compatibility declaration, source tracking, per-extension documentation with compatibility matrix, and auto-wiring of constitution extensions into all detected agent platforms on constitution pull (research-derived from Extensions Catalog forensics, - -)

Forensic anchor (FACT, 2026-07-24 — measured from the live Extensions Catalog at `docs/extensions/EXTENSIONS_CATALOG.md`). The project's extension ecosystem — skills, MCP servers, ACPs, LSPs, plugins, and supporting infrastructure (34 skills, 1 MCP, 38 provider aliases, 21 CCR providers, 4 daemons, plus a separate HelixSkills repo) — exhibited four structural gaps: **(G1)** of 18 active Claude Code project skills, only Claude Code and OpenCode platforms are explicitly supported — HelixCode, Gemini CLI, and Qwen Code have zero recorded

loading paths despite being governed by this constitution's own lockstep carriers (§11.4.157 — CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md); **(G2)** 4 of 18 active skills are symlinks into the constitution submodule (correct pattern) while 14 are local-only directories with NO traceability record to which original source/master each belongs to — a git-archive snapshot of a skill without its origin is a fork with no master; **(G3)** the Extensions Catalog is a single snapshot document with NO per-extension per-platform compatibility matrix and NO mechanism to re-detect drift when extensions change; **(G4)** the §11.4.164 `post_update_hook.sh` detects changed governance MDs + scripts/hooks/skills/mcp/plugins and registers them, but it does NOT auto-wire constitution skills into all detected agent platforms — 4 constitution skills (`media-validator` , `multitrack` , `scheduled-work-queue` , `session-sync`) remain un-wired on both Claude Code (not symlinked into `.claude/skills/`) and OpenCode (no loading path to `.opencode/skills/`), silently absent while their governance files claim they exist — the canonical "docs say present, runtime says absent" gap §11.4.108 forbids.

The mandate (ALL hold — four sub-clauses, one lifecycle):

(A) Cross-agent extension compatibility. Every extension governed by this constitution — Skill, MCP server, ACP bridge, LSP, plugin, provider alias, embedding, daemon, or any future extension class — MUST:

1. **Ship a per-platform compatibility declaration** listing every governed CLI agent platform at minimum: Claude Code, OpenCode, Gemini CLI, Qwen Code. The declaration states, per platform, one of the closed vocabulary: `SUPPORTED` (verified working with a recorded per-platform liveness probe), `UNTESTED` (present but not yet verified — honest gap, tracked per §11.4.197), `UNSUPPORTED` (platform genuinely cannot load this extension class — per §11.4.112 structural-impossibility, with cited evidence), or `CONFIG-ONLY` (declared in config, never verified as wired per §11.4.139).
2. **NEVER cause loops, hangs, crashes, or broken behavior** in any platform that attempts to load it. An extension that causes a platform malfunction is a §11.4.228(A) violation of PASS-bluff severity — it claims compatibility it does not provide, producing the exact "green suite but broken-for-user" failure mode §11.4 forbids.
3. **Cross-platform liveness testing is mandatory** — the §11.4.169 closed test-type set applies: the extension MUST ship a per-platform liveness test (does the agent load it + produce correct output?) that runs on every declared-

SUPPORTED platform at minimum. A platform where testing is genuinely infeasible (no runtime available on this host) is an honest §11.4.3 SKIP-with-reason recorded in the compatibility declaration, NEVER a faked PASS.

(B) Extension source tracking. Every extension MUST be traceable to its exact original source:

1. **Canonical source record** — each extension carries machine-readable provenance: originating repository URL, commit SHA (or tag+delta description when `UNTRACED`), canonical file path within that origin. This record lives in an `EXTENSION_SOURCE.yaml` in the extension's directory or in a consumer-maintained extensions registry. An extension with no provenance record is `UNTRACED` and a tracked §11.4.197 item — never silently trusted as correct.
2. **Symlinks preferred over copies** — a copy is a fork with no master; a symlink (or a git submodule pointer, or a reflink share) preserves the canonical source relationship and makes divergence mechanically detectable. Where a copy is genuinely required (platform cannot traverse symlinks, an out-of-repo directory), the SOURCE record MUST state the canonical origin + why a copy was necessary (§11.4.6 honest rationale). Blind copies without provenance are §11.4.124-class dead-code-in-waiting.
3. **Per-project extensions catalog required** — every governed project MUST maintain an extensions catalog (`docs/extensions/EXTENSIONS_CATALOG.md` or equivalent declared per §11.4.35) enumerating every active extension, its source, its per-platform compatibility status, and its last-verified timestamp. The catalog is §11.4.86 roster/corpus-backed (sha256 fingerprint of the sorted extension keyset, NOT mtime), re-syncs out-of-the-box on any extension change. The catalog is the SINGLE SOURCE OF TRUTH for "what extensions exist" — ad-hoc per-agent skill directories without catalog entries are `UNCATALOGUED` and a tracked §11.4.197 item.

(C) Mandatory extension documentation with per-platform compatibility matrix. The extensions catalog MUST:

1. Carry a **per-platform compatibility matrix** (rows = extensions, columns = governed platforms, cells = compatibility status from the closed vocabulary in clause (A)(1)) — readable at a glance. A cell with no recorded status is `UNTESTED` by default, never silently assumed compatible.

2. Be maintained per §11.4.65 (four-format export: `.md`, `.html`, `.pdf`, `.docx`), updated on every extension add/change/removal, with §11.4.44 revision header.
3. Include **liveliness testing evidence per platform** — per extension × platform, a per-platform probe run result (load-ok / load-failed-for-reason / skip-with-reason) with a timestamp and evidence path per §11.4.69 `feature_class=extension_loading`. A platform declared SUPPORTED with no recorded probe is `UNTESTED` overridden — a classification mismatch and a §11.4.6 violation.
4. Auto-sync via the §11.4.106 docs-chain engine or equivalent — a stale catalog where extension drift has occurred (catalog says SUPPORTED, extension was removed or renamed) is a §11.4 PASS-bluff at the extension-management layer.

(D) Auto-wiring of constitution extensions into all detected agent platforms on constitution pull (§11.4.228, STRENGTHENS §11.4.164). The §11.4.164 `post_update_hook.sh` seam is EXTENDED — it MUST, after detecting changed extension artifacts in the constitution submodule:

1. **Detect all governed agent platforms** on the consuming host (Claude Code via `.claude/`, OpenCode via `.opencode/` or `opencode.json`, Gemini CLI via `~/.gemini/` or project `GEMINI.md`, Qwen Code via `.qwen/` or project `QWEN.md` — the platform-specific detection logic is consumer DATA per §11.4.35, never a hardcoded list). A platform whose detection mechanism is absent is an honest `DETECTION_UNSUPPORTED` gap recorded in the catalog, NOT a reason to skip the remaining platforms.
2. **Wire constitution skills into each detected platform** — for each detected platform, for each constitution skill not already present:
   - Claude Code: symlink from `.claude/skills/<skill-name>/` → `constitution/skills/<skill-name>/`
   - OpenCode: symlink from `.opencode/skills/<skill-name>/` → `constitution/skills/<skill-name>/`
   - Gemini CLI, Qwen Code: per-platform equivalent wiring mechanism (consumer-supplied DATA, §11.4.35)
3. **Register new/changed MCP servers** into each platform's MCP configuration (`.mcp.json`, `opencode.json`, equivalents) — per the existing §11.4.164 MCP-registration seam, applied to every detected platform.

4. **Self-validate** — after wiring, probe each platform that supports a `--list-skills` / `--list-extensions` equivalent or fall back to a filesystem check (skill/MCP directory present + parseable). A failed wiring is logged as a §11.4.32 post-pull finding — never silently absorbed.
5. **The hook is idempotent** — re-running it does not duplicate symlinks, corrupt configs, or lose existing non-constitution extensions. Every operation is dry-run-capable (report what WOULD change without changing it).
6. **Honest boundary (§11.4.6)** — auto-wiring makes constitution extensions REACHABLE to each platform; it does NOT guarantee each platform's runtime ACCEPTS them (a platform may have a different skill format, an incompatible MCP protocol, or a missing runtime — all honest `UNTESTED` / `UNSUPPORTED` gaps recorded in the catalog, never faked pass), and it does NOT make the extensions correct (§11.4.108 + the extension's own tests). The wiring gap this clause closes is the REACHABILITY gap — a skill that exists in the constitution but is invisible to a governed platform has the gap §11.4.108 forbids (source-present, runtime-absent).

Composes with §11.4.164 (post_update_hook — the seam this anchor extends) / §11.4.157 (GEMINI.md lockstep — the platforms whose extensions this governs) / §11.4.65 (universal export — the catalog's four formats) / §11.4.86 (roster/corpus-backed auto-sync — the catalog's fingerprint discipline) / §11.4.106 (docs-chain — the catalog's sync engine) / §11.4.69 (sink-side evidence — `feature_class=extension_loading`) / §11.4.124 (dead-code — blind copies are dead-code-in-waiting) / §11.4.3 (SKIP-with-reason — untestable platforms) / §11.4.112 (structural impossibility — `UNSUPPORTED` platforms) / §11.4.139 (clean artifact — `CONFIG-ONLY` un-wired class) / §11.4.32 (post-pull sweep — wiring failures) / §11.4.108 (runtime signature — a platform load is the extension's runtime signature) / §11.4.197 (research completion — per-platform wiring gaps are tracked items) / §11.4.169 (test types — per-platform liveness tests) / §11.4.142 / §11.4.194 (code-review — extension changes cross the same quality gauntlet). Classification: universal (§11.4.17) — no hardware/vendor/project literal; the consuming project supplies its platform detection mechanism, per-platform wiring commands, extension registry path, and catalog path as DATA per §11.4.35.

Propagation gate `CM-COVENANT-114-228-PROPAGATION` (literal `11.4.228`).
Recommended gates:

- `CM-EXTENSION-PER-PLATFORM-COMPAT` (every governed extension carries a per-platform compatibility declaration using the closed vocabulary; a declared-SUPPORTED platform with no recorded liveness probe → FAIL)
- `CM-EXTENSION-SOURCE-TRACKED` (every extension carries provenance with origin repo + commit SHA or explicit `UNTRACED` rationale tracked per §11.4.197; a blind copy without provenance → FAIL)
- `CM-EXTENSION-CATALOG-DOCUMENTED` (the extensions catalog exists, carries a per-platform compatibility matrix, is four-format-exported per §11.4.65, and its sha256 fingerprint is fresh per §11.4.86; a stale catalog with extension drift → FAIL)
- `CM-CONSTITUTION-EXTENSIONS-AUTO-WIRED` (the §11.4.164 `post_update_hook.sh` auto-wires constitution skills into every detected governed platform, idempotently, and self-validates; a constitution skill not wired into a detected SUPPORTED platform → FAIL, with explicit `DETECTION_UNSUPPORTED` / `UNSUPPORTED` SKIP-with-reason — the §11.4.201(1) false-positive guard)

All paired §1.1 mutations (gate-code = separate work item, NOT claimed shipped §11.4.6).

Canonical authority: constitution submodule `Constitution.md` §11.4.228.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-per-platform-compat`, `--single-platform-OK`, `--extension-without-provenance`, `--copies-over-symlinks-OK`, `--skip-extension-catalog`, `--catalog-format-optional`, `--skip-auto-wire`, `--manual-wiring-suffices`, `--untraced-extensions-OK` flag exists.

§ . . 2026 09—07 Dynamic, on-demand skill/extension activation: the active capability surface is the MINIMUM needed now, everything else stays DISCOVERABLE and one command away (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-09-07): *"We MUST use actively only Skills we need at particular moment! Use of the Skills and other extensions MUST BE fully dynamic so token use is ALWAYS at the minimum and any of Skills and the extensions activated / deactivated as they are needed / used! Extend the constitution Submodule properly for this to work flawlessly and be a part of every project like all other heavy optimizations and improvements we MUST use!"*

Forensic case study (FACT, measured 2026-09-07, Claude Code 2.1.263). A host-level skill pool of 955 skills renders as **29,153 B (~7,288 tok) of skill NAMES in every turn of every session** that loads it; the same pool rendered with its `description:` fields is **101,279 B (~25,319 tok/turn)** — a **3.5x** cliff the consuming project neither controls nor observes. Both figures are exact byte counts; the token conversion is a bytes/4 ESTIMATE (no tokenizer on the measuring host) and is labelled as such per §11.4.6.

Every project whose agent platform exposes a capability surface (Agent Skills, plugins, extensions, tool sets) MUST keep the **ACTIVE** surface at the minimum genuinely needed, with everything else activatable ON DEMAND — never a static all-on set paid for in every turn. ALL of the following hold:

(A) MINIMAL DECLARED CORE, EVERYTHING ELSE ON DEMAND. The consuming project declares, as DATA (§11.4.35), a deliberately small `core` set that is always active, and the pool it may draw from. Anything not `core` is inactive by default and activated only when a task needs it. An entry added to `core` is paid for in EVERY turn of EVERY session and MUST earn that.

(B) DISCOVERABILITY IS NEVER TRADED FOR TOKENS. A deactivated capability MUST remain FINDABLE — an always-available index lists the whole catalogue with its active/inactive state. `superpowers:using-superpowers` (§11.4.102(B)) continues to bind unconditionally: if any skill could apply, even at 1% relevance, it MUST be found in that index and invoked rather than improvised from

memory. **Saving tokens by making a capability unfindable is FORBIDDEN** — that trades a measurable cost for an unmeasurable one and is a §11.4 bluff at the capability layer.

(C) FAIL SAFE, NEVER SILENT (§11.4.201). A failed activation MUST tell the agent explicitly that the capability is **NOT** available, and exit non-zero. An activation that appears to succeed while the capability is unusable leaves the agent believing it holds a tool it does not — the false-null class this constitution treats as a critical defect. A dangling or unreadable activation MUST be rolled back, not left behind.

(D) CONCURRENCY-SAFE BY REFERENCE COUNTING (§11.4.119 / §11.4.176). Where several agent sessions share one checkout, one session's deactivation MUST NOT strip a capability another session is using. Claims are reference-counted and released only when no other claimant is **PROVEN** live (§11.4.180 liveness, §11.4.174 ownership-verified) — a claim whose holder cannot be proven live is reported as such, never silently assumed dead OR alive.

(E) HONEST ASYMMETRY — MEASURE WHAT THE PLATFORM ACTUALLY DOES (§11.4.6). The engine MUST be built on the platform's PROVEN behaviour, not its documented behaviour, wherever the two are not both established. Measured first-hand on this platform: **activation hot-loads** (a newly-linked skill appears in the ALREADY-RUNNING session with no restart) but **deactivation does NOT hot-unload** (the same session still invokes a deleted skill from cache). The saving is therefore a **minimal session-start baseline plus on-demand growth**; claiming that deactivation frees tokens in the live session is a bluff. Equally measured: the platform's own `permissions.deny` `Skill(name)` rule **blocks invocation without removing the entry from the rendered list**, so it saves ZERO tokens — a first-party mechanism must be TESTED for the property being relied on, never assumed from its documentation (§11.4.74 reuse-before-reimplement is satisfied by evidence, not by the existence of a similarly-named feature).

(F) A BUDGET GATE GUARDS THE RENDER-MODE CLIFF. The per-turn cost depends on a render mode the project does not control. A gate MUST budget the active surface in BOTH the name-only and the description-eager render, so an upstream switch to eager rendering breaks a gate LOUDLY instead of silently costing tokens in every turn thereafter.

(G) BOTH MODES — WITH AND WITHOUT THE CONSTITUTION SUBMODULE (REQ-NOTE-0001). The engine lives in the constitution submodule (§11.4.28, inherited by reference, never copied — §11.4.80/§11.4.177), but a project that does NOT vendor the submodule MUST still get a working command or an HONEST, LOUD refusal — never a silent no-op. Resolution order is explicit: an environment override, the in-project submodule, any ancestor's, an operator-configured host checkout, a documented host fallback, then honest degradation naming WHAT is unavailable, WHY, WHERE it looked, and WHAT still works. Degradation MUST use an exit code distinct from "operation failed", and MUST state that the existing static capability set is untouched so the agent does not conclude a capability exists when it does not.

(H) PRUNE-TO-CORE IS THE DEFAULT, AND IT IS NOT A §11.4.122 REMOVAL (operator decision, 2026-09-07). Session initialisation MUST **prune by default** — demoting every non-core capability so a session pays only for the declared `core` — rather than merely raising `core` and leaving whatever else accumulated. A raise-only baseline satisfies clause (A) in letter and voids it in substance: the surface grows monotonically as capabilities accumulate, and the operator's standing requirement that token use be at the minimum is never actually met (measured on the reference project: 19 active / 422 B / ~105 tok-per-turn under raise-only, against 3 active / 65 B / ~16 tok-per-turn under prune-by-default). **Pruning is NOT the silent capability removal §11.4.122 forbids**, and the distinction is structural, not a promise: §11.4.122 forbids removing an end-user CAPABILITY without asking, and pruning removes none, because all three properties that make it reversible and visible hold BY CONSTRUCTION — (1) **LISTED**: the always-core catalogue index of clause (B) still shows every pruned capability, so nothing becomes unfindable; (2) **ACTIVATABLE**: one command returns it, hot-loading into the running session per clause (E); (3) **NOT DELETED**: a link-style activation loses only the LINK (its target untouched), and a capability whose only copy lives in the active location is **MOVED** to a preserved store that is itself a searched source — never removed, zero bytes destroyed. A pruned capability is **dormant and one command away**, which is materially different from dropping a component from a build. A demotion that cannot be performed safely — another live session holds a claim per clause (D), the preserved store already holds an entry of that name (never clobber preserved content, §9.2), the move fails or lands unreadable (it is REVERTED), or the capability would not be re-activatable afterwards — MUST refuse LOUDLY and leave the capability ACTIVE per clause

(C); refusing is always preferable to risking content. Relocating a capability MUST be relocation-SAFE: path-relative references inside it that would break at the new location are re-pointed at the identical resolved target before the move, and where that is not sufficient the post-move readability check refuses and reverts (§11.4.6 — never leave a broken capability behind). An **explicit, documented opt-out** MUST exist at both session scope (an environment override) and project scope (a manifest flag), with session scope winning; a mechanism with no way out gets worked around destructively, and the opt-out keeps the engine in play rather than abandoned. Where the engine is unreachable (clause (G)), pruning inherits that clause's honest refusal — a session is NEVER left silently holding fewer capabilities than it believes it has.

Honest boundary (§11.4.6). This anchor reduces the per-turn cost of the capability LIST; it does not reduce the cost of a skill's BODY once invoked (that is already lazy), and it does not make any capability more correct. It is a token-efficiency and capability-hygiene discipline (§11.4.141), not a substitute for §11.4.102(B) skill discovery, and never a licence to hide a capability the task needed. Clause (H)'s prune-by-default inherits clause (E)'s measured asymmetry: the saving lands at the next session start for any entry the platform has already cached, so pruning is run AT session initialisation and its in-session effect is never overclaimed.

Classification: universal (§11.4.17) — platform-neutral; the consuming project supplies its manifest, pool sources, preserved-store location, budget limits, opt-out names, and host fallback as DATA per §11.4.35. Composes §9.2 / §11.4.6 / §11.4.28 / §11.4.35 / §11.4.74 / §11.4.80 / §11.4.102(B) / §11.4.119 / §11.4.122 (clause (H) — pruning is structurally NOT the removal §11.4.122 forbids) / §11.4.141 / §11.4.164 / §11.4.174 / §11.4.176 / §11.4.177 / §11.4.180 / §11.4.201 / §1.1. Propagation gate `CM-COVENANT-114-272-PROPAGATION` (literal `11.4.272`) + recommended mechanism gates `CM-SKILL-SURFACE-BUDGET` (active surface within budget in BOTH render modes), `CM-SKILL-ACTIVATION-FAIL-SAFE` (a failed activation exits non-zero with an explicit reason and leaves nothing dangling) and `CM-SKILL-PRUNE-PRESERVES-CONTENT` (clause (H): after a prune every demoted capability is still LISTED, still ACTIVATABLE, and its content byte-identical to its pre-prune state; an unsafe demotion refuses and leaves the capability ACTIVE) + paired §1.1 mutation (squeeze the budget below the real surface → the gate MUST FAIL; break an activation target → activation MUST refuse and leave no dangling link; remove the engine → the shim MUST exit with its distinct engine-absent code, never 0; make the demotion DESTRUCTIVE, or strip the relocation-

safety step, or strip the other-live-session claim check → the prune gate MUST FAIL). Gate-code beyond the shipped scripts = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.272. Non-compliance is a release blocker. No escape hatch — no `--all-skills-always-on`, `--skip-activation-engine`, `--hide-inactive-skills`, `--silent-activation-failure`, `--ignore-surface-budget`, `--assume-skill-loaded` flag.

---

§. . — Measuring-instrument verification: a census that INFORMS a decision must be control-needled before its result is believed (research-derived, - -)

Forensic anchor (FACT, 2026-09-08 — FIFTEEN instances in a single session). Across one working session, fifteen separate ad-hoc measurements each returned a confident, plausible, ACTIONABLE-looking result that was an artefact of the instrument rather than a property of the system. Every one was indistinguishable, at the moment it was read, from a real finding:

1. A governance-anchor census matched only one of two block-opener forms (`^### §N` , missing `^**§N`), under-counted the corpus by ~24 mandates and **manufactured 24 phantom dangling anchors** that were nearly reported as defects.
2. `comm` on inputs sorted numerically rather than lexicographically reported **all 34 anchors missing** from a file that contained none of them missing.
3. `$?` read after a pipeline reported the **pipeline's last stage**, turning a script's exit 1 into a reported exit 0 — a failure read as success.
4. A field-name typo (`CMA_PROVIDER_KEY_VAR` for `CMA_PROVIDER_KEYVAR`) returned `<none>` for every record, reading exactly like "the records have lost their key bindings".
5. `grep -w` treated `-` as a word boundary, so `zai` matched inside `zai-coding-plan` and moved a record from the NOT-PINNED to the PINNED column — corrupting the classification a recommendation was being built on.

6. A summary-line grep assumed `passed, failed` ordering; on failure the tool prints `failed, passed`, so the match came back EMPTY and the run's outcome was unknown while looking merely quiet.
7. `while read` silently dropped a final line with no trailing newline, so a one-element list iterated zero times and printed nothing.
8. A mutation was applied with the wrong environment knob (`CMA_TEST_SCRIPTS_DIR` where the suite reads `CMA_SCRIPTS_UNDER_TEST`), so the mutation tree was never loaded, the REAL tree was tested, and the resulting green was **meaningless while looking like proof the guard held**.
9. A "does the guard block this?" probe used a command the guard does not target, so it ran unblocked and proved nothing either way — an inconclusive result that reads as a negative.
10. A test fixture named each alias identically to its provider id, so two columns held the same string and a real column-swap mutation **survived at 52/0** under a comment asserting no two cells could share a value.
11. A positive control was chosen OUTSIDE the instrument's declared scope — the needle was added by a different layer than the one under test — so a working instrument reported as broken.
12. A positive control was chosen that WAS the very question under test, so it could not discriminate between "the instrument works" and "the answer is yes".
13. An `awk` range extracting a table cell used an escaping that matched nothing, reporting **838 rows where the live system held 134** — a number wrong by 6x that was about to gate a DESTRUCTIVE database recreate, caught only because it contradicted a count taken seconds earlier.
14. **A denylist with TWO needles was searched with ONE.** Both literals were loaded and both lengths were even PRINTED, then only the second was passed to the search. Every "zero remaining" was blind to the first — and the artefacts it cleared were regenerations of a CREDENTIAL SCRUB, so a live credential was reported eliminated while it was still being handed over in four copy-pasteable lines per file. The report was internally consistent, cited a positive control, and was wrong.
15. **`$?` clobbered by a command substitution in the same statement.** `printf '... exit=%s' "$(basename "$f")" "$?"` expands the substitution FIRST, so `$?` carried `basename`'s status, not the command's: a `pandoc` run that

FAILED with exit 64 was reported as `exit=0` while writing nothing. Distinct from instance 3 (exit code after a PIPELINE) — here the clobber happens inside the argument list of the very statement that reports it.

The gap this closes. §11.4.201 governs GUARDS AND GATES — instruments that BLOCK, where the failure modes are a false refusal and a false pass. §11.4.107(10) governs ANALYZERS, which must be calibrated golden-good / golden-bad. NEITHER covers the far more common instrument: the **ad-hoc census that INFORMS** — a `grep`, a set difference, an exit-code read, a field lookup, a row count — whose output is not a verdict but a FACT that a decision, a fix, a classification, or a report is then built upon. Those instruments are written inline, used once, trusted implicitly, and never themselves tested.

The mandate. Any measurement whose result will drive a decision, a claim, a classification, or a report MUST be control-needled BEFORE its result is acted on or communicated:

- **(a) POSITIVE control** — a value KNOWN to be present must be found by the instrument. This catches a pattern too narrow, a wrong field name, a wrong path, a wrong knob, and a stage that silently produced nothing.
- **(b) NEGATIVE control** — a value KNOWN to be absent (a fabricated needle) must NOT be found. This catches a pattern too broad, an accidental substring or word-boundary match, and a default that fabricates a hit.
- **(c) An EMPTY result is never a finding on its own.** Absence of output is one of: the thing is genuinely absent; the instrument could not look; the instrument looked in the wrong place; the instrument's output shape differed from what was matched. Those are FOUR different states and the raw empty result distinguishes none of them (§11.4.6). An empty result may be reported only once (a) has established the instrument can see a positive.
- **(d) A result that CONTRADICTS directly-observed evidence is presumed to be an instrument fault until proven otherwise.** When a census reports absent something that was just read on screen, the instrument is the first suspect, not the system.
- **(e) The instrument used to establish a FACT is itself part of the evidence.** A captured claim cites the command that produced it, so a reader can re-run it and a later reader can see what was actually measured (§11.4.5 / §11.4.2).

- **(f) When the criterion is a SET, the control must cover EVERY member of it.** A denylist, a literal list, a taxonomy, a multi-column key: needling ONE member proves only that member is searchable, and a search that silently drops the others still reports a clean, internally-consistent, positively-controlled zero. This is the failure mode instance 14 records, and it is the most dangerous on this list precisely because (a) and (b) both PASS while the answer is wrong: the control and the search shared the same single needle, so the instrument proved itself against exactly the case it was already handling. Enumerate the set, assert the search covers it, and state the member count in the captured claim.
- **(g) Capture an exit status BEFORE any other expansion can overwrite it.** `$?` is the status of the LAST command to run, which in a compound statement is frequently not the command being reported on — a pipeline's last stage (instance 3) or a command substitution sitting earlier in the same argument list (instance 15). Assign it to a variable on its own line, immediately after the command, and report the variable.

Honest boundary (§11.4.6). Control needles prove an instrument can distinguish PRESENT from ABSENT. They do NOT prove it measures the RIGHT property — a correctly-needled census of the wrong thing is still the wrong answer, and no amount of needling substitutes for asking whether the question is the right one. Nor do they make an instrument correct for inputs unlike the needles; a needle chosen to resemble the expected data will not reveal a failure on data that does not.

Classification: universal (§11.4.17) — platform-neutral; the consuming project supplies its concrete instruments and needle values per §11.4.35. Composes §11.4.5 / §11.4.6 (an un-needled census is a guess wearing a number) / §11.4.2 / §11.4.107(10) (the analyzer analogue) / §11.4.118 (an enumerated coverage claim rests on the instrument that enumerated it) / §11.4.201 (the guard/gate analogue) / §1.1.

Propagation gate `CM-COVENANT-114-273-PROPAGATION` (literal `11.4.273`) + recommended gate `CM-CENSUS-CONTROL-NEEDLED` (a decision-bearing census cites its positive and negative control) + paired §1.1 mutation (strip the literal → propagation gate FAILs; accept a decision-bearing census with no control → `CM-CENSUS-CONTROL-NEEDLED` FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6 / §11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.273. Non-compliance is a release blocker. No escape hatch — no `--skip-control-needle`, `--empty-is-absent`, `--trust-the-grep`, `--census-needs-no-proof`, `--obvious-result-exempt`, `--one-needle-covers-the-set`, `--read-status-after-substitution` flag exists.

2026 09 08

§ . . — Mechanical work belongs in a script, not in an agent's context: extract it, test it, and re-scan for it on a cadence (User mandate, - -)

Forensic anchor (FACT, 2026-09-08). In one working session, five independent review agents consumed roughly 400,000–450,000 tokens EACH. Reading their transcripts, the dominant cost was not judgment — it was MECHANICS: copy a tree to a scratch directory, apply a one-line mutation, run a suite, read the summary line, restore, verify the restore byte-identical, compare counts against a previous round, repeat for seven mutations. Every step of that loop is deterministic, has a fixed input and output, contains no decision, and produces the same result every time. It is a shell script that was instead executed one tool call at a time by a language model, at roughly four orders of magnitude the cost, with a fresh opportunity for a transcription error at every step — and this session recorded ten such instrument errors (§11.4.273).

The rule. Work that is DETERMINISTIC, REPEATABLE and contains NO JUDGMENT MUST be delegated to a properly written script or small compiled utility, and invoked — not performed turn-by-turn inside an agent's context. Concretely, the following belong in a tool and not in a transcript: mutation apply / run / restore / verify loops; suite fan-outs and their result tabulation; before-and-after comparisons; scratch-tree setup and teardown; residue and quiescence scans; census and inventory queries; artifact fingerprinting; evidence collection and layout; and any step already written down as a fixed recipe.

(a) EXTRACT. When an agent performs the same mechanical sequence more than twice, that sequence is a defect in the tooling, not a task. It is extracted into a script or utility, given a name, and invoked thereafter. The bar is behavioural, not aesthetic: if the steps can be written down exactly, they can be executed exactly, and an agent executing them by hand is the wrong instrument.

(b) THE EXTRACTED TOOL IS ITSELF GOVERNED. It ships with documentation a reader can act on (§11.4.18), a user guide or manual entry where it is operator-facing, and tests that validate its behaviour and produce MACHINE evidence deterministically (§11.4.50 / §11.4.5) — including its failure paths, its exit-code semantics (§11.4.201), and the control needles that prove its own measurements (§11.4.273). An extraction that replaces a slow correct process with a fast unproven one is a regression, not an optimisation.

(c) RE-SCAN ON A CADENCE. Every project periodically scans itself for mechanical work still being performed by agents — recurring command sequences in transcripts and logs, repeated inline recipes across scripts, and steps documented as "run these commands in order" — and refactors the findings out. This is a standing, recurring obligation, not a one-off cleanup: new mechanical loops accrete continuously as features land, and a codebase that was clean last quarter will not be clean this one.

(d) HONEST BOUNDARY — what MUST NOT be extracted (§11.4.6). This rule is about MECHANICAL work, and misapplying it is worse than not applying it at all. JUDGMENT MUST NOT be scripted: root-cause determination (§11.4.102), review verdicts (§11.4.125 / §11.4.142 / §11.4.194), the decision that a fix is correct, the choice of what to measure, the reading of whether captured evidence actually supports a claim, and any assessment requiring the question "is this the right thing to be checking?" A script that renders a verdict is a bluff gate (§11.4.201), and automating a review into a green checkmark is precisely the PASS-bluff this constitution exists to prevent. The correct division is: the SCRIPT gathers and executes, deterministically and cheaply; the AGENT decides what it means. Cost is never a reason to move a decision into a script, and a script's output is evidence for a judgment, never a substitute for one.

Classification: universal (§11.4.17) — platform-neutral; the consuming project supplies its concrete scripts, utility language, scan cadence, and transcript/log sources per §11.4.35. Composes §11.4.18 (script documentation) / §11.4.50 (deterministic results) / §11.4.5 (captured machine evidence) / §11.4.141 (token

efficiency — this is its mechanical-work instance) / §11.4.201 (exit-code and real-condition semantics) / §11.4.273 (the extracted instrument is control-needled) / §11.4.124 (investigate before deleting the thing you replaced) / §1.1.

Propagation gate `CM-COVENANT-114-274-PROPAGATION` (literal `11.4.274`) + recommended gates `CM-MECHANICAL-WORK-EXTRACTED` (a mechanical sequence performed by an agent more than twice has an extraction item) and `CM-EXTRACTED-TOOL-DOCUMENTED-AND-TESTED` (every extracted tool carries docs + tests + deterministic machine evidence) + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6 / §11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.274. Non-compliance is a release blocker. No escape hatch — no `--skip-extraction`, `--agent-can-do-it`, `--one-off-is-fine`, `--script-the-review`, `--extract-without-tests` flag exists.

§ . . — Semantic (Lumen) index + indexing-efficiency gate + universal agent/subagent accessibility: every indexed space reaches every agent and subagent, is proven COMPLETE before it is called ready, and earns its place by a MEASURED, honest efficiency result (research-derived, - -)

Forensic anchor (FACTS, measured in one cycle on a consuming project with a multi-hundred-thousand-file tree; genericised). (1) A semantic code-search server was configured, yet its own status reported 0 files, no embedding model, and "never" indexed — configured is not indexed (§11.4.196(F)); the constitution had no mandate for it at all. (2) A fixed-fixture benchmark measured a 69.2% token reduction for the structural index over grep+read — computed only over the 8 queries BOTH routes answered correctly — while the same structural index answered 5 of 15 fixture queries WRONG (grep+read: 2 of 15); the semantic route had 0 comparable answers because it was unindexed. A token saving reported without its wrong-answer count would have been a bluff. (3) One per-call, uncached, full-table file-list query accounted for 93.4% of the reference-resolution wall time (hundreds of calls at roughly 350 ms each on a ~585k-file index). (4) A foreign-key cascade on bulk re-insert stalled the bulk window on large tables. (5) A launcher classified `sync` by rows already stored instead of by pending backlog

and ran a bulk job of tens of millions of references on the stock runner, foreground and unsupervised. (6) The vendor deletes a lock by age, not by holder liveness, and a writable open recreates dropped indexes without the lock — a concurrent-writer corruption hazard. (7) An empty result from an instrument that had errored or been silenced was nearly read as "absent" (§11.4.273).

The mandate (ALL hold).

(A) UNIVERSAL ACCESSIBILITY — every indexed space, every agent, every subagent, no exception. Every code index the project maintains (the structural CodeGraph index of §11.4.78 AND the semantic Lumen index, and any future code index) MUST be reachable by EVERY AI agent the project uses AND by every subagent those agents dispatch, through a server DEFINITION the agent or subagent can actually resolve (project-scoped committed configuration or an installed plugin — never a per-user toggle only the main session sees). A configuration that ENABLES a server which nothing DECLARES (no definition resolvable where the agent or subagent runs) is a violation, not a partial setup. Proof is a real tool call FROM A DISPATCHED SUBAGENT returning an index-only fact (the §11.4.78(4) unforgeable challenge), per agent; an agent that genuinely cannot be driven is a documented §11.4.3 SKIP, never a faked PASS.

(B) SEMANTIC INDEX PROVEN, NOT CONFIGURED. The embedding model is provisioned (rootless per §11.4.161) and proven by a REAL embed call returning the declared dimensionality — a model listing alone is not evidence. The semantic index's scope is GENERATED from the SAME class source as the structural scope (§11.4.78(7)–(8), via the constitution's `scripts/lumen/gen_lumenignore.py`), so third-party and secret classes are excluded identically. Health = files AND chunks non-zero + a known-symbol search hit + a negative control + freshness within a consumer-declared TTL.

(C) COMPLETE BEFORE "READY". An index is called ready only when its PENDING work is zero (unresolved-reference backlog, files still to parse, embedding queue), its state reports complete, its file count is within the consumer tolerance of the accepted expected count, and its required schema indexes exist. Calling a partial index ready is a §11.4 PASS-bluff. Every count and every empty result behind this verdict is control-needled (§11.4.273); an instrument that errored or was silenced proves nothing.

(D) EFFICIENCY IS MEASURED DETERMINISTICALLY AND REPORTED HONESTLY. A benchmark harness runs a FIXED query set with GOLDEN answers against the index route AND a grep+read baseline, N ≥ 3 iterations with an identical canonical result hash (§11.4.50), recording per query: bytes, estimated tokens (declared estimator), p50/p95 latency and correctness against the golden answer. Token reduction is computed ONLY over queries BOTH routes answered correctly; wrong answers are reported separately, BY QUERY ID, and never averaged away. The harness is self-validating (a tampered golden fixture is rejected; a mutated estimator is caught). The thresholds — minimum reduction, maximum wrong-answer count, latency ceilings — are consumer DATA (§11.4.35), never constitutional literals. A route that saves tokens but is wrong more often than the baseline is NOT an efficiency win, and a report that shows only the saving is a bluff.

(E) ROUTING FOLLOWS THE MEASUREMENT. Structural questions go to the structural index and conceptual questions to the semantic index BEFORE grep+read (§11.4.141), EXCEPT for query classes the benchmark shows the route answers wrongly — those fall back to grep+read and the gap is tracked (§11.4.197), never hidden.

(F) PERFORMANCE DEFECTS ARE ROOT-CAUSED, PATCHED AND PROVEN. A measured hot spot or stall (clauses (3)–(4) of the anchor) is root-caused (§11.4.102) and fixed in the version-keyed patched runner with RED→GREEN, a paired mutation and an equivalence proof on a byte copy, then gated on every upgrade per §11.4.80(4).

(G) SAFETY. Writes obey §11.4.80(5) (one writer entry, liveness not lock age, bulk classified by pending backlog); live reads are read-only/immutable; experiments run on fixtures or copy-on-write copies.

Latent binding (§11.4.96 pattern). A project that maintains NO semantic index today is bound by clauses (A)–(G) only for the indexes it does maintain; the semantic-index clauses bind the moment it adds one — absence of a semantic index is never evidence that (B)–(E) were satisfied.

Honest boundary (§11.4.6). This anchor proves an index is REACHABLE, COMPLETE, correctly SCOPED and MEASURED — it does NOT prove every answer correct (the benchmark samples a fixed set; wrong-answer classes stay tracked), does NOT make the benchmark representative of every query shape, and does NOT license narrowing the index scope to hit a threshold (scope narrowing is

a §11.4.66 operator decision). The reference benchmark harness exists today as a prototype OUTSIDE the constitution tree; landing it under `<constitution>/scripts/` is owed work, not claimed done. **[2026-09-25 STATUS UPDATE — see the EXTENSION below: the harness has since landed INSIDE `<constitution>/scripts/lumen/`; this sentence's "outside the tree" premise is superseded, tracked-work-remaining is narrowed to the gate CODE only.]**

EXTENSION — clause (D) benchmark run live on this consuming project + harness-location correction (research-derived, 2026-09-25). Clause (D)'s deterministic-benchmark requirement was exercised end-to-end via the sanctioned `constitution/scripts/lumen/lumen_verify.sh` tool. This corrects the honest-boundary sentence above: as of this extension the harness is no longer "a prototype OUTSIDE the constitution tree" — it now LIVES inside `constitution/scripts/lumen/` with its own test file `constitution/scripts/lumen/tests/test_lumen_verify.sh`; only the recommended `CM-CODE-INDEX-BENCH` gate CODE remains owed, unchanged (§11.4.6/§11.4.227 — a fact stated as fact, not claimed beyond what landed). A 13-query golden set (9 conceptual, 2 structural, 2 deliberately-unsupported) was pre-declared via `Grep` / `Read` /direct `sqlite3` queries against the index database BEFORE any Lumen query ran, then benchmarked 3 times back-to-back producing byte-identical `results.tsv` / `summary.txt` on every run (§11.4.50 determinism, satisfying clause (D)'s N≥3-identical-canonical-hash requirement for the WHOLE benchmark, not one query). Result: recall 9/11 = 0.818 over in-scope indexable queries; both FAILs are near-misses at rank 7 (one place past k=5); token reduction computed ONLY over the 9 correctly-answered queries (mean-of-per-query 30.1%), the 2 negative-reduction outliers reported honestly rather than averaged away — satisfying clause (D)'s reduction-over-both-correct-only + wrong-answers-listed-by-ID requirements. The Q7 `.sh`-file mystery (zero results for a query whose gold file is a `.sh` script) was root-caused with NON-GUESSED evidence per §11.4.6: a direct `sqlite3` query of Lumen's own index database showed ZERO `.sh` files indexed anywhere (0 of 2625 in-scope `.sh` files on disk), confirmed authoritative by Lumen's own source (`internal/chunker/languages.go`'s `supportedExtensions` has no `.sh`, `.kt`, or `.txt` entry) — a genuine chunker-coverage gap, not a scope or freshness defect, tracked per clause (E) + §11.4.197 rather than silently worked around. Evidence: `docs/research/lumen_benchmark_20260925/RESULTS_v2.md`

(+ `.html` / `.pdf` / `.docx` siblings per §11.4.65), commit `ac532a21df78a447c2aceec30f63476348e93077`. Composes §11.4.6 / §11.4.28 / §11.4.50 / §11.4.65 / §11.4.177 / §11.4.197 / §1.1.

Classification: universal (§11.4.17) — no hardware/vendor/project literal; the consumer supplies its own-org list, scope DATA, accepted counts, tolerances, TTL, benchmark queries/golden answers and thresholds as DATA per §11.4.35; all mechanism is inherited by reference (§11.4.28 / §11.4.80 / §11.4.177). Composes §11.4.3 / §11.4.6 / §11.4.10 / §11.4.35 / §11.4.50 / §11.4.66 / §11.4.78 / §11.4.79 / §11.4.80 / §11.4.99 / §11.4.102 / §11.4.141 / §11.4.161 / §11.4.196(D)(F) / §11.4.197 / §11.4.201 / §11.4.224 / §11.4.227 / §11.4.232(C) / §11.4.273 / §1.1.

Propagation gate `CM-COVENANT-114-275-PROPAGATION` (literal `11.4.275`, block-start exactly once per governance file, lockstep across mirrors per §11.4.227(B)) + recommended mechanism gates `CM-CODE-INDEX-AGENT-ACCESSIBLE` (every indexed space answers an unforgeable challenge from a dispatched subagent; an enabled-but-undeclared server FAILs) / `CM-CODE-INDEX-COMPLETE-BEFORE-READY` (pending == 0, state complete, count within tolerance) / `CM-LUMEN-MODEL-PROVEN` / `CM-LUMEN-SCOPED-FRESH` / `CM-CODE-INDEX-BENCH` (N≥3 identical hashes, reduction over both-correct only, wrong answers listed by id, thresholds from consumer DATA) / `CM-CODEGRAPH-SCOPE-PROOF` / `CM-CODEGRAPH-UPGRADE-PROBE-GATED` / `CM-CODEGRAPH-WRITER-EXCLUSIVE` / `CM-CODEGRAPH-BULK-BY-BACKLOG` / `CM-CODEGRAPH-STALL-WATCHDOG` + paired §1.1 mutations (enable a server with no resolvable definition → accessibility gate MUST FAIL; report ready with pending > 0 → completeness gate MUST FAIL; drop the wrong-answer list or compute reduction over all queries → bench gate MUST FAIL; unload the embedding model → model gate MUST FAIL; classify bulk by stored rows → bulk gate MUST FAIL; drop the liveness check → writer gate MUST FAIL; golden-FALSE per §11.4.201(1): an unchanged, complete, reachable index MUST NOT fire any gate). Gate CODE is a separate work item, NOT claimed shipped (§11.4.6 / §11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.275. Non-compliance is a release blocker. No escape hatch — no `--index-partial-ready`, `--enabled-is-declared`, `--main-session-only-index`, `--skip-model-proof`, `--reduction-without-wrong-answers`, `--bulk-by-row-count`, `--lock-age-is-liveness` flag.